# LLVM-C2RTL ユーザーリファレンス

**Version 1.0.0 : 2017/01/20**

# 目次


用語解説 ..... 5
1. C2RTL 設計フレームワーク ..... 6
1.1 LLVM-C2RTL ツール環境 ..... 6
1.2 C/C++言語仕様 ..... 7
1.3 C++記述への対応 ..... 7
1.4 データ型 ..... 7
1.5 __attribute__によるハードウエア属性定義 ..... 7
1.5.1 ビット幅属性・符号属性・レジスタ属性・メモリ属性 ..... 8
1.5.2 RTL 合成対象トップ関数名指定 ..... 8
1.5.3 その他のハードウエア属性指定 ..... 9
1.6 RTL 合成対象トップモジュールの入出力ポート判定 ..... 9
1.7 C/C++記述から RTL 構造への変換の基本原理 ..... 9
1.7.1 グローバル変数の RTL 変換 ..... 9
1.7.2 ローカル変数（一時変数）の RTL 変換 ..... 10
1.7.3 メモリ変数アクセスの RTL 変換 ..... 10
1.7.4 代入式の RTL 変換 ..... 10
1.7.5 条件付き実行ブロックの RTL 変換 ..... 10
1.7.6 C/C++記述上の変数参照方向と RTL 記述上の参照タイミング ..... 11
1.7.7 C/C++記述上のメモリ変数アクセスと RTL 構造 ..... 12
1.8 RTL 合成用 C/C++記述方式 ..... 14
1.8.1 データフロー型単純パイプライン記述方式 ..... 14
1.8.2 フロー制御付きパイプライン記述方式 ..... 15
1.9 RTL 合成用 C/C++記述のコーディング制約 ..... 16
1.10 C2RTL フレームワーク内部処理 ..... 16
1.11 C2RTL フレームワーク RTL 合成処理 ..... 17
1.12 C2RTL フレームワーク RTL 動作検証フロー ..... 18
1.12.1 LLVM_C2RTL_Applications（C2RTL サンプルコード） ..... 18
1.12.2 clang オプション ..... 18
1.12.3 出力ファイル ..... 19
1.12.4 C2RTL 設計検証フロー ..... 19
1.12.5 C2RTL オプションファイル（c2r_options.log） ..... 21
2. C2RTL によるプロセッサモデル記述 ..... 23
2.1 TCT プロセッサ概要 ..... 23
2.1.1 TCT プロセッサ命令セット ..... 24

2.1.2 TCT プロセッサ C モデル ..... 26
2.2 RISC-V プロセッサ C モデル ..... 26
2.3 プロセッサ C モデルの検証手順 ..... 27
2.4 RTL ファイル出力情報 ..... 27
2.4.1 Verilog 出力ファイル構造 ..... 27
2.4.2 論理合成対象モジュール記述構造 ..... 28
2.4.3 RTL 合成結果情報 ..... 30
2.4.4 RTL 論理遅延クリティカルパス情報 ..... 31
2.5 RTL 等価 C モデル記述構造 ..... 32
2.5.1 RTL 等価 C コア関数記述構造 ..... 34
2.6 C2RTL 合成ログファイル ..... 35
2.7 C2RTL ツールエラーと対処法 ..... 35
3. C2RTL フレームワークによる ASIP 開発メソドロジー ..... 36
3.1 設計入力モデル ..... 36
3.2 ASIP 開発フロー概略 ..... 36
3.3 アプリ C リファレンスモデルの TCT プロセッサ実行検証環境の構築（アプリ C 実装モデルの構築） ..... 38
3.3.1 コンソール IO 処理のプロセッサ IO ポート移植 ..... 38
3.3.2 ファイル IO 処理のメモリ組込み変換実装 ..... 40
3.3.3 ファイル IO 処理の外部 FIFO モデル変換実装 ..... 41
3.4 TCT プロセッサ C モデルの ASIP 拡張命令用リソース定義 ..... 42
3.5 TCT プロセッサ C モデルの ASIP 拡張命令実装 ..... 42
3.6 ASIP 拡張命令実行動作記述のパイプライン設計手法 ..... 43
3.6.1 ASIP 拡張命令のパイプライン動作記述 ..... 43
Appendix ..... 46
A. TCTProcFIFO の C モデル ..... 46
A.1 ISS テストベンチ関数（TCTProc.c） ..... 46
A.2 ISS コア関数（TCTProc.c） ..... 48
A.3 プロセッサリソース定義記述（TCTProcArch.h） ..... 48
A.4 FE ステージ動作記述（TCTProc.c） ..... 51
A.5 命令フォーマット定義記述（TCTProcArch.h） ..... 52
A.7 EX ステージ動作記述（TCTProc.c） ..... 59
A.8 WB ステージ動作記述（TCTProc.c） ..... 62
A.9 IO デバイスモデル ..... 62
A.9.1 IO デバイスリソース定義記述（TCTProc_io.h） ..... 63
A.9.2 IO 命令デコード記述（TCTProc_io.c） ..... 65

A.9.3 IO 制御 FSM 動作記述（TCTProc_io.c） ..... 65
A.9.4 FIFO 制御 FSM 動作記述（TCTProc_io.c） ..... 67
A.10 外部 FIFO テストベンチモデル動作記述（TCTProc_io.c） ..... 68
B. TCTProcJPG_FIFO の C モデル ..... 73
B.1 ASIP 拡張命令用リソース定義（TCTProc_jpg.h） ..... 73
B.2 ASIP 拡張命令デコード記述 ..... 76
B.3 ASIP 拡張命令実行動作記述 ..... 78
C. C2RTL オプションファイルパラメータ ..... 81
D. C2RTL ツールエラーメッセージ ..... 81

# 用語解説

- モデル：本書では、特に断りがない限り「設計記述」や「記述」という意味を指す。
- RTL（Register-Transfer Level）モデル（RTL 記述）：レジスタ転送式によって回路動作を表現する回路設計記述。一般に「論理合成可能 RTL モデル」と「テストベンチ RTL モデル」に分かれる。
  - 論理合成可能 RTL モデル：論理合成ツールによって回路ネットリストが自動生成できる RTL 記述。
  - テストベンチ（testbench）RTL モデル：論理合成可能 RTL モデルをシミュレーション検証するための動作記述（信号生成、出力信号比較・アサーション等）であり、一般的に論理合成可能でない。
- C/C++モデル（C/C++記述）：C/C++言語のプログラム。本書で説明する C2RTL フレームワークでは、様々な位置付けの「C/C++モデル」が存在する。
  - RTL 合成可能 C/C++モデル：C2RTL フレームワークによって RTL モデルが自動生成できる C/C++記述（本書では、この C/C++記述を「C/C++データフロー記述」と呼んでいる）。
  - テストベンチ C/C++モデル：RTL 合成可能 C/C++モデルをシミュレーション検証するための動作記述（信号生成、出力信号比較・アサーション等）であり、一般的に RTL 合成可能でない。
  - アプリ C/C++モデル：プロセッサ用実行プログラムを生成するための C/C++ソースプログラム。
    - アプリ C/C++リファレンスモデル：アルゴリズム検証等において、ターゲットプロセッサの実行環境に移植する前に、ホストコンピュータでの計算機シミュレーションを行う目的で開発されたアプリ C/C++モデル。
    - アプリ C/C++実装モデル：ターゲットプロセッサの実行環境（メッセージ出力、データ入出力等）に移植されたアプリ C/C++モデル。
- ISS（Instruction-Set Simulator）：命令セットシミュレータ。本書では、C/C++記述による ISS 実装が前提であり、さらに「ISS コア関数」と「ISS テストベンチ」から構成される。
  - ISS コア関数：プロセッサの「RTL 合成可能 C/C++モデル」であり、プロセッサの 1 サイクルの動作を記述する。
  - ISS テストベンチ：ISS コア関数以外の ISS 機能を記述した C/C++モデルであり、プログラムロードや ISS ループ制御を行う。
  - プロセッサ実行ファイル：前記「アプリ C/C++実装モデル」（プロセッサ用実行プログラム）を C/C++コンパイラによって機械語命令に変換された情報を保存したファイル。本書で紹介する「TCT プロセッサ」用実行ファイルを「TCT プロセ

ッサ実行ファイル」と呼ぶ。

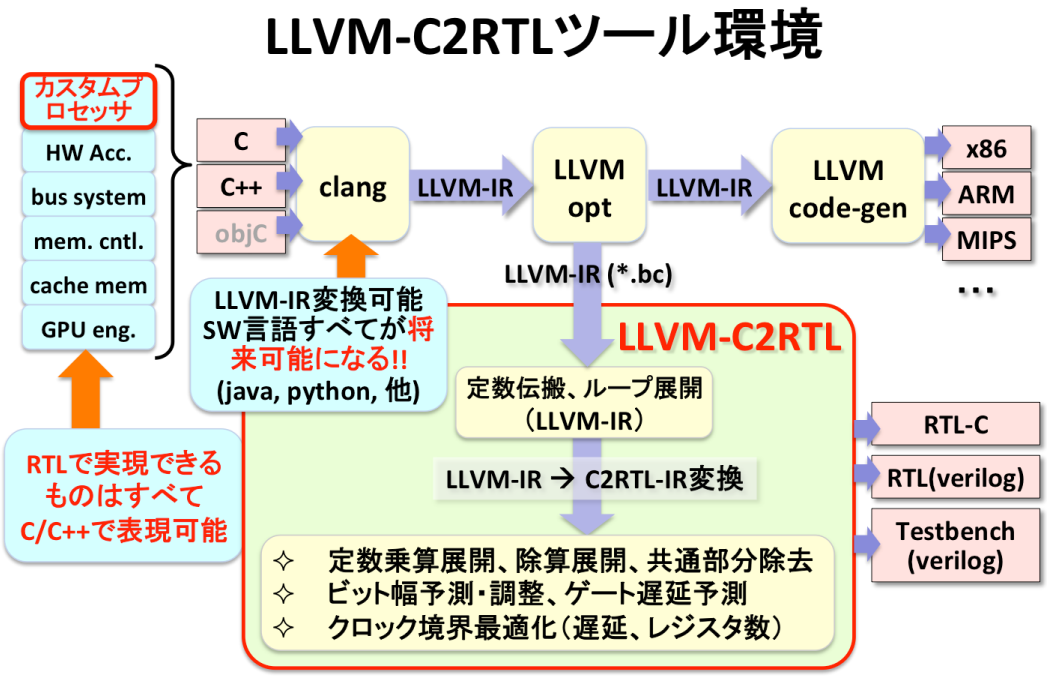
- LLVM：llvm.org によって公開されているオープンソースコンパイラ基盤である。本書で説明する C2RTL 設計フレームワークは、この LLVM コンパイラ基盤上に実装されている。

# 1. C2RTL設計フレームワーク

本手順書は、東京工業大学一色研究室において開発された C/C++記述ベースの RTL（Register-Transfer Level）設計フレームワーク（C2RTL）の使用法を解説するものである。前提知識として、C/C++言語プログラミング、プロセッサアーキテクチャ、論理回路設計手法、RTL 検証手法等を持ち合わせていることが望ましい。
本節では、C/C++記述ベースの RTL 設計フレームワーク（C2RTL）における C/C++記述上の特徴を中心に解説する。

## 1.1 LLVM-C2RTLツール環境

LLVM は、llvm.org によって公開されているオープンソースコンパイラ基盤である。C2RTL 設計フレームワークは、この LLVM コンパイラ基盤を利用し、LLVM 中間言語（LLVM-IR）を入力とした最適化処理系（LLVM optimizer : opt）エンジンとして、LLVM コンパイラ基盤上に実装されている。



## 1.2　C/C++言語仕様

C2RTL フレームワークにおける C/C++言語仕様は、clang（LLVM の C/C++言語処理系）がサポートする C/C++言語規格（C99, C++11 等）からの独自の言語拡張は一切なく、標準的な C/C++開発環境（標準コンパイラツールチェイン）で RTL 合成用 C/C++記述を開発することを前提としている。RTL 合成のためのハードウエア属性（ビット幅、レジスタ属性、メモリ属性等）は__attribute__記述によって clang に内部情報として取り込まれるが、これらハードウエア属性は、C/C++言語規格外であり、C/C++記述が定義する処理アルゴリズムの動作そのものを変更するものではなく、RTL 回路構造を生成するときのみ有効となる情報である。つまり、C/C++記述は、RTL 回路の動作を忠実に表現し、加えて、正しい RTL 回路構造を生成するためのこれらハードウエア属性を正確に C/C++記述上で反映させる必要がある。これら C/C++記述上の注意点については、以降の節で説明する。

## 1.3　C++記述への対応

C2RTL フレームワークは、当初 C 言語（ANSI-C）による記述体系を第一のターゲットとして実装され、その後、少しずつ C++言語への機能拡張を進めている。現状では、下記の C++記述に対応でしている。

- **クラスメソッド関数**: ただし、仮想関数は試験的実装を進めている段階である。また、constructor や deconstructor は今後実装予定である。
- **テンプレート関数、テンプレートクラス**：**実装済み**
- **クラス継承**：**今後実装予定**

## 1.4　データ型

RTL 合成用 C/C++記述では、現状では整数型、整数型メンバーからなる構造体（階層化可能）、及びこれらのポインタ型のいずれかの変数のみを対象とする。浮動小数点型は、将来的に導入を検討している。

## 1.5　__attribute__によるハードウエア属性定義

__attribute__による属性定義は、その対象を変数名または typedef 名とする。属性定義の対象が typedef 名の場合、対象の typedef を継承した typedef 名についても同じ属性が付与される。例えば、

```
typedef int A;
typedef A B, C;
```

という記述の場合、A は int 型の別名（typedef）であり、B と C は A を継承した typedef である。ここで、A に付与した属性は自動的に B と C にも付与される。次節に、この typedef 継承を利用した属性付与の例を挙げる。

### 1.5.1 ビット幅属性・符号属性・レジスタ属性・メモリ属性

```
#define _BW(N)           __attribute__((C2RTL_bit_width(N)))
#define _T(N)            __attribute__((C2RTL_type(N)))
#define _TOP(N)          __attribute__((C2RTL_top_function(N)))

typedef unsigned short UINT16 _BW(16), UINT10 _BW(10);
typedef signed short   SINT16 _BW(16), SINT10 _BW(10);
typedef UINT16         ST_UINT16 _T(state), M_UINT16 _T(memory);
typedef SINT16         SINT10;
typedef UINT10         ST_UINT10 _T(state), M_UINT10 _T(memory);
```

上記の例で、UINT16 と UINT10 は unsigned short の別名（typedef）であり、SINT16 と SINT10 は signed short の別名である。さらに、ST_UINT16, M_UINT16 はそれぞれ UINT16 の別名であり、ST_UINT10, M_UINT10 は UINT10 の別名である。これら、typedef 継承によって定義された型名について、_BW(N)と_T(N)マクロを介して、以下のハードウエア属性を指定している。

- **__attribute__((C2RTL_bit_width(N)))**：ビット幅属性を typedef 名に付与する。上記の例では、UINT10, SINT10 に対して 10 ビット幅属性を付与しており、これらを継承する ST_UINT10, M_UINT10 についても同じ 10 ビット幅属性が付与される。また、UINT16, SINT16 に対しては 16 ビット幅属性を付与しており、これらを継承する ST_UINT16, M_UINT16 についても同じ 16 ビット幅属性が付与される。なお、ビット幅 N が C++テンプレートの整数パラメータの場合、C2RTL_bit_width(“N”)と表記する。
- **__attribute__((C2RTL_type(state)))**：レジスタ属性を typedef 名に付与する。レジスタ属性を持つ typedef 名の変数は、スカラ型変数と配列型変数いずれも可能である。RTL 記述では、レジスタ属性を持つ変数は「レジスタ」として生成される。
- **__attribute__((C2RTL_type(memory)))**：メモリ属性を typedef 名に付与する。メモリ属性を持つ typedef 名の変数は、配列型である必要がある。RTL 記述では、メモリ属性を持つ配列型変数は「同期型 SRAM」として生成される。

なお、レジスタ属性・メモリ属性のいずれも付与されていない typedef の変数は、RTL 記述では「wire」として生成される。以降では、レジスタ属性・メモリ属性のいずれも付与されていない C 変数を「ワイヤ変数」と呼び、レジスタ属性の変数を「レジスタ変数」、メモリ属性の変数を「メモリ変数」とそれぞれ呼ぶことにする。

### 1.5.2 RTL合成対象トップ関数名指定

RTL 合成対象となる C/C++記述トップ関数定義前に以下の__attribute__を挿入する。

- **__attribute__((C2RTL_top_function(N)))**

ここで、N はパイプライン段数である。また、RTL 合成対象トップ関数の引数及び戻り値（ある場合）は、生成される RTL トップモジュールの入出力ポートに置き換わる（後述）。

### 1.5.3 その他のハードウエア属性指定

- **__attribute__((C2RTL_type(direct_signal)))**: 生成される RTL トップモジュールの入力ポートは、通常はパイプラインの初段ステージに入力し、後段ステージで参照される場合はパイプラインレジスタが挿入される。また、RTL トップモジュールの出力ポートは通常はパイプラインの最終段ステージでラッチされた後に出力され、中間段ステージで生成された出力信号はパイプラインレジスタが（最終段まで）挿入される。**direct_signal** 属性の入力ポートは、パイプラインレジスタを挿入せずに直接入力されるようになり、**direct_signal** 属性の出力ポートはパイプラインレジスタを挿入せずに直接出力されるようになる。

## 1.6 RTL合成対象トップモジュールの入出力ポート判定

前述の **C2RTL_top_function 属性**によって指定される RTL 合成対象トップ関数において、その関数引数と戻り値（void 型関数でない場合）が生成される RTL トップモジュールの入出力ポートに対応する。RTL 合成対象トップ関数の引数は、整数型やポインタ（整数型または最下層メンバー変数がすべて整数型の構造体）を前提とする。また、戻り値は整数型を前提とする。

- **入力ポート**：RTL 合成対象トップ関数以下で参照され、なおかつ一度も代入されることがない変数引数がすべて入力ポートとなる。構造体ポインタがトップ関数の引数として与えられた場合、代入されないすべての最下層メンバー変数が入力ポートとして定義される。整数型ポインタ引数の場合は、内部で代入されない変数であれば入力ポートとして定義される。
- **出力ポート**：RTL 合成対象トップ関数以下で代入されるポインタ型引数と戻り値が出力ポートとして定義される。構造体ポインタ引数において、代入される最下層メンバー変数もすべて出力ポートとして定義される。

## 1.7 C/C++記述からRTL構造への変換の基本原理

C2RTL フレームワークにおいては、RTL 合成対象トップ関数の 1 回の実行が、生成 RTL の 1 クロック動作を反映するように回路合成される。C/C++記述が RTL 構造に変換される際の具体的特徴は以下の通りである。

### 1.7.1 グローバル変数のRTL変換

C/C++記述のグローバル変数（関数内部にローカルな static 変数を含む）は、ワイヤ変数、レジスタ変数、メモリ変数のいずれにも成り得る。グローバルなレジスタ変数やメモリ変数は、RTL 構造上の物理的なレジスタ・メモリのインスタンスと一対一に対応する。レジスタ変数の場合、finite state machine (FSM)の「状態変数」(state variable)を実現できる。

### 1.7.2 ローカル変数（一時変数）のRTL変換

C/C++記述のローカル変数（static 変数は除く）は、ワイヤ変数として使用することが自然であるが、レジスタ属性を持たすこともできる。ローカル変数がレジスタ属性を持つ場合は、変数へ代入される値は 1 クロック後に参照可能になるが、FSM の状態変数を実現することはできない。なぜならば、RTL 合成対象トップ関数を 1 回実行させた後（1 クロック動作後）、一時変数の値はすべて失われるからである。同様の理由から、ローカル変数はメモリ属性を持たすこともできない。一方で、後述の説明にあるように、関数呼出しはすべてインライン展開されるため、その関数の呼出し回数に応じた分だけ RTL 上で複製される。なお、上記のローカル変数の性質は、厳密には RTL 合成対象トップ関数及びその内部で呼び出される関数だけに適用され、RTL 合成対象トップ関数よりも上位の関数のローカル変数は、前節のグローバル変数の性質を持つ。例えば、ある関数のローカル変数として構造体変数が定義され、このポインタを RTL 合成対象トップ関数の引数として渡した場合、この構造体変数のメンバー変数を FSM の状態変数として定義することは可能である。

### 1.7.3 メモリ変数アクセスのRTL変換

メモリ変数への代入式（代入式の左辺式がメモリ変数）は、メモリ書込み回路が合成され、メモリ変数への参照式（代入式の右辺式）はメモリ読出し回路が合成される。

### 1.7.4 代入式のRTL変換

代入式の右辺式に出現する演算子はすべて組合せ回路として合成される。さらに左辺式がレジスタ変数の場合は、レジスタの書込み回路として合成される。

### 1.7.5 条件付き実行ブロックのRTL変換

if-else ブロックのように条件付きで処理が分岐する場合、原則としては、レジスタ変数代入式とメモリ変数代入式を除くすべての演算式を「投機的」に実行し、すべての演算結果を、条件式によりマージする回路が生成される。

例：下記コードにおいて、Z がレジスタ変数で、Z 以外はすべてワイヤ変数とする。

```
if (A) {
  x = a + b;
  y = x * Z;
  Z ++;
}
else{
```

```
  x = a - b;
  y = x + Z;
}
```

verilog による RTL 記述は以下となる。（実際の出力 RTL 記述は、assign 文の計算式の粒度が細かくなる）

```
assign x = (A) ? a + b : a - b;
assign y = (A) ? x * Z : x + Z;
always@(posedge clk) if(A) Z <= Z + 1;
```

### 1.7.6 C/C++記述上の変数参照方向とRTL記述上の参照タイミング

C/C++言語は逐次処理プログラミング言語であることと、C/C++言語上ではレジスタ属性やメモリ属性は無効であることから、C/C++記述上での実行順序と生成される RTL 構造の関係について十分な理解が必要となる。

- 変数の順方向参照：ある変数の参照式がその変数への代入式から「到達可能」（代入された後に参照される）な場合、これを「順方向参照」と呼ぶ。レジスタ変数やメモリ変数への順方向参照は、その変数の代入式から 1 クロック後に実行されるような RTL 構造を生成する。
- 変数の逆方向参照：ある変数の参照式に「到達可能」な同一変数への代入式が存在しない場合、これを「逆方向参照」と呼ぶ。レジスタ変数への逆方向参照は、レジスタが更新される直前の値（current state）を参照することに対応する。

前節の「グローバル変数」「ローカル変数」の分類で説明すると、グローバル変数のみが「逆方向参照」可能であり、ローカル変数は「順方向参照」のみとなる。

例：下記コードにおいて、Z がレジスタ変数で、Z 以外はすべてワイヤ変数とする。

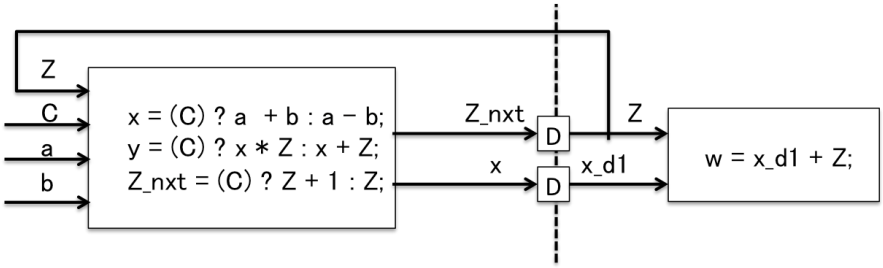
```
if (C) {
  x = a + b;
  y = x * Z; // x:順方向参照、Z:逆方向参照
  Z ++;      // Z:逆方向参照
}
else{
  x = a - b;
  y = x + Z; // x:順方向参照、Z:逆方向参照
}
w = x + Z; // x:順方向参照、Z:順方向参照
```

ここで、w への代入式に Z の順方向参照が含まれているため、この代入式はその他の代入式よりも 1 クロック後に発生する。この RTL 構造を実現するために、w への代入式における x の参照値を 1 クロック遅延挿入した回路構造が生成される。下記が、生成される RTL 記述となる。

```
////////////// pipe [j] wire assignments //////////
assign x = (C) ? a + b : a - b;
assign y = (C) ? x * Z : x + Z;
assign Z_nxt = (C) ? Z + 1 : Z;
////////////// pipe [j] state update //////////
always@(posedge clk) Z <= Z_nxt;
always@(posedge clk) x_d1 <= x;
```

```
//////////// pipe [j+1] wire assignments //////////
assign w = x_d1 + Z;
```

上記では、RTL パイプライン境界を明示しており、さらに説明の便宜上 Z の代入式を Z_nxt（Z の next state：ワイヤ変数）への代入式に分解して記述している。下図にこの RTL 構造を図示する。




### 1.7.7 C/C++記述上のメモリ変数アクセスとRTL構造

前節の「順方向参照」と「逆方向参照」の概念は、メモリ変数のアクセスを記述する上でも重要となる。

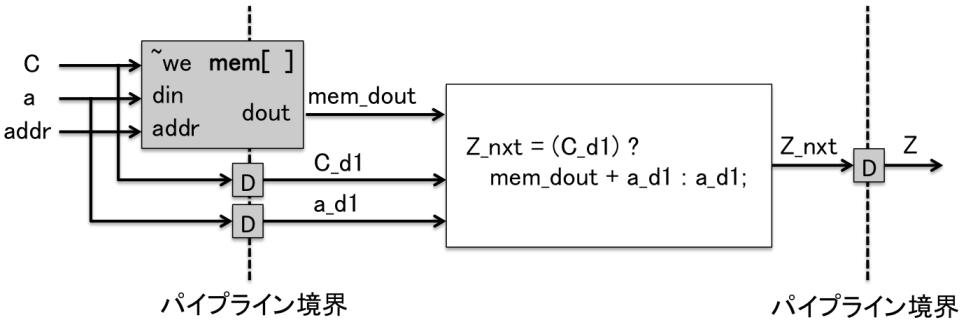
例 1：下記コードにおいて、Z がレジスタ変数、mem がメモリ配列変数で、これら以外はすべてワイヤ変数とする。

```
if (C) {
  Z = mem[addr] + a; // mem[addr]: 順方向メモリ参照
}
else{
  mem[addr] = a;
  Z = a;
}
```

この C 記述に対応する同期型 SRAM を前提とした RTL 記述は以下となる。

```
//////////// pipe [j] state update //////////
always@(posedge clk) begin
  addr_d1 <= addr;
  if(~C) mem[addr] <= a;
  a_d1 <= a;
  C_d1 <= C;
end
//////////// pipe [j+1] wire assignments //////////
assign mem_dout = mem[addr_d1];
//////////// pipe [j+1] state update //////////
always@(posedge clk)
  Z <= (C_d1) ? mem_dout + a_d1 : a_d1;
```

ここで、レジスタ Z への代入は mem[addr]の読出しの 1 サイクル後に実行されるため、mem が配置されるパイプラインステージよりも一つ後段のステージにレジスタ Z が配置される 2 段パイプライン構造になる。このため、レジスタ Z の更新式で必要となるワイヤ変数 C, a を 1 クロック遅延挿入した回路構造になる（下図）。




例 2 : 前記例から、メモリ mem とレジスタ Z を同じパイプラインステージに配置することを意図した C 記述を下記に示す。ここで、Z はレジスタ変数で、Z_tmp はグローバルワイヤ変数（初期値 0）とする。

```
Z = Z_tmp; // Z_tmp: 逆方向ワイヤ参照
if (C) {
  Z_tmp = mem[addr] + a; // mem[addr]: 順方向メモリ参照
}
else{
  mem[addr] = a;
  Z_tmp = a;
}
```

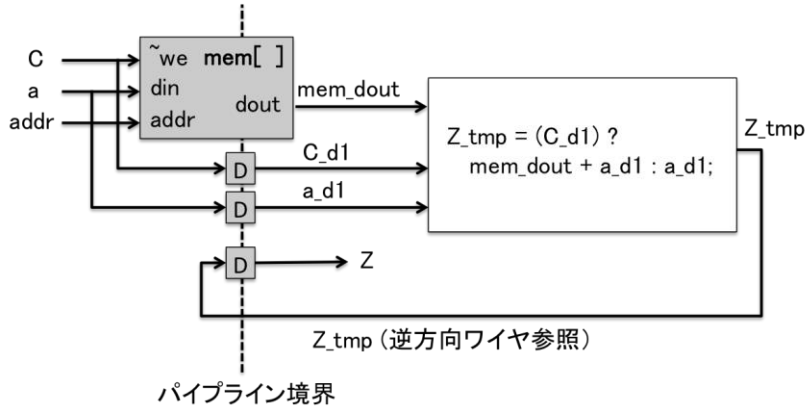
ここで、グローバルワイヤ変数 Z_tmp は、レジスタ Z の代入式で逆方向参照されていることに注目する。Z_tmp への代入式は順方向メモリ参照を伴うため、その値は 1 クロック後に確定するが、その Z_tmp の値が参照可能になるのは、Z_tmp への代入が実行された後に、再び RTL 合成対象トップ関数が呼び出されたときである（1 クロック後の動作に対応する）。この C 記述上の動作の関係を以下に示す。

```
---------- 時刻 T (C = 1 とする) ---------
Z = Z_tmp; // T = 0 の場合は、Z_tmp = 0（初期値）とする
if (C) { // C = 1 なので以下のメモリ読出しを実行
  Z_tmp = mem[addr] + a;
}
else{ // C = 1 なので以下をスキップ
  mem[addr] = a;
  Z_tmp = a;
}
---------- 時刻 T + 1 (C = 0 とする) ---------
Z = Z_tmp; // Z_tmp = mem[addr] + a;がこの時点で参照可能になる
if (C) { // C = 0 なので以下をスキップ
  Z_tmp = mem[addr] + a;
}
else{ // C = 0 なので以下実行
  mem[addr] = a;
  Z_tmp = a;
}
```

この C 記述に対応する同期型 SRAM を前提とした RTL 記述は以下となる。

```
////////////// pipe [j] state update //////////
always@(posedge clk) begin
  Z <= Z_tmp;
```

```
  addr_d1 <= addr;
  if(~C) mem[addr] <= a;
  a_d1 <= a;
  C_d1 <= C;
end
////////////// pipe [j+1] wire assignments //////////
assign mem_dout = mem[addr_d1];
assign Z_tmp = (C_d1) ? mem_dout + a_d1 : a_d1;
```



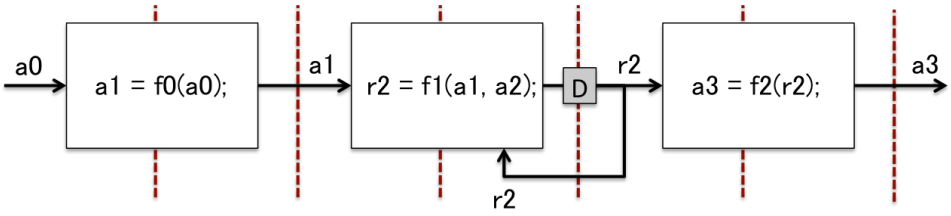

上記 2 例では、異なる C 記述方式によって生成される RTL 記述構造（具体的にはパイプライン構造）が異なることを示している。実際は、これら異なる RTL 記述構造も論理回路上は厳密には等価であるが、レジスタ Z のパイプライン上の配置が異なっているため、Z を参照するその他の代入式においては、Z を参照するタイミングが変わることになる。

## 1.8 RTL合成用C/C++記述方式

ここでは、RTL 合成用 C/C++記述における 2 つの C 記述方式について解説する。

### 1.8.1 データフロー型単純パイプライン記述方式

この記述方式は、単純なパイプライン動作を表現するための記述方式であり、FSM の状態更新にのみレジスタ変数の逆方向参照を行い、その他のデータ処理はすべて順方向参照で記述するコーディングスタイルである。パイプライン動作がストールを伴わない場合（例えば Image Signal Processing Pipeline など）には、この単純な記述方式が有効である。この記述方式の場合、パイプライン段数を自由に設定することが可能なため、高スループット回路を実現しやすいが、パイプラインストールを伴わない演算回路のみで実現する必要があり、回路規模が必要以上に大きくなる場合もある。




パイプライン境界：任意の「回路カット」上に配置可能 → 高スループット化
（C2RTLツール：パイプライン境界最適配置自動化機能付き）

データフロー型単純パイプライン構造
（r2：レジスタ変数、その他：ワイヤ変数）

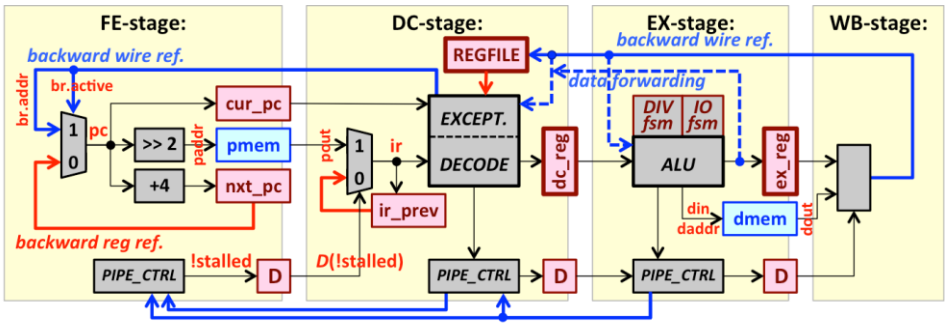
### 1.8.2 フロー制御付きパイプライン記述方式

この記述方式は、パイプライン型プロセッサのようにパイプラインストール制御（フロー制御）が必要な場合の記述方式であり、前節の「グローバルワイヤ変数の逆方向参照」を頻繁に使用する記述スタイルである。「グローバルワイヤ変数の逆方向参照」は以下のケースで必要となる。

- 後段パイプラインステージにおいてストール状態になった場合、この状態を前段パイプラインに通知する。
- 後段パイプラインステージで生成された演算結果を前段パイプラインに data forwarding する。
- 同期型メモリ読出しデータを次のクロックで参照する（前節参照）

一般には、パイプライン型プロセッサのフロー制御ロジックは非常に複雑になることもあり、プロセッサのパイプライン数を変更するためには、かなりの設計工数を要する傾向にある。

本書の後半部分で説明する機能特化型プロセッサの開発においても、この「フロー制御付きパイプライン記述方式」は必要となる。プロセッサのアプリ特化拡張命令を実装するとき、回路遅延の都合上パイプライン回路方式を採用せざるを得ないケースもあり、元のプロセッサパイプライン数を変更せずに、拡張命令のパイプライン実装を行うためには、「逆方向参照」記述を駆使して、RTL パイプライン構造を詳細に調整する必要が出てくるためである。




フロー制御付きパイプライン構造
（4段パイプラインRISCプロセッサ）

## 1.9　RTL合成用C/C++記述のコーディング制約

RTL 合成を可能とする C/C++記述は以下の制約を満たす必要がある。

【コーディング制約 1】 RTL 合成対象トップ関数以下に含まれるすべてのループ処理は、その繰返し数がコンパイル時に定数回であることが推測可能である必要がある。

C2RTL フレームワークでは、すべてのループ処理は完全なループ展開ができることが前提となっているためである。「コンパイル時に定数回であることが推測可能」とは、繰返し数が直接定数で与えられる場合以外に、以下のように「定数伝搬」によって定数回であることが推測可能である場合も含む。

```
loop_func(10, 2);
void loop_func(int m, int n)
{
  int i;
  for(i = 0; i < m; i += n){…}
}
```

ここでは、loop_func 関数内部のループ処理の繰返し数は、変数 m, n によって与えられるが、これら m, n が loop_func の呼び出し側でそれぞれ 10, 2 と与えられているため、繰返し数が 5 回であることがコンパイル時に推測できる。

【コーディング制約 2】 RTL 合成対象トップ関数以下に含まれるすべての関数呼出しは、再帰的関数であってはならない。

C2RTL フレームワークでは、すべての関数呼出しは完全なインライン展開ができることが前提となっているためである。

【コーディング制約 3】RTL 合成対象トップ関数以下では、同一のメモリ変数に対するアクセス（読出しまたは書込み）は、高々 1 回でなければならない。

この制約は、シングルポート同期型 SRAM を想定しているための制約である。

【コーディング制約 4】RTL 合成対象トップ関数以下では、同一のレジスタ変数またはメモリ変数に対し、2 回以上の代入が起こらないこと。

つまり､C 記述上で同一のレジスタ変数またはメモリ変数への代入文が複数存在する場合は、これらはすべて排他的である必要がある。

## 1.10 C2RTLフレームワーク内部処理

RTL 合成処理のための内部処理は、主に以下のステップから成る。

- ループ展開 :【コーディング制約 1】を満たすすべてのループ処理は、「完全ループ展開」される。このとき、ループインデックス変数はすべて定数に置き換わる。

例：

```
int i, sum = 0;
for(i = 0; i < 5; i ++){ sum += a[i]; }
```

→ 完全ループ展開後

```
sum = 0;
sum += a[0];
sum += a[1];
sum += a[2];
sum += a[3];
sum += a[4];
```

- 関数のインライン展開：【コーディング制約 2】を満たすすべての関数呼出しは、「完全インライン展開」される。
- 中間 wire 型変数のビット幅推定：内部の wire 型変数のビット幅は、その変数への代入式から自動的に推定される（#pragma によるビット幅属性は無視される）。
- 中間レジスタ変数のビット幅推定：内部のレジスタ型変数のビット幅は、#pragma によるビット幅属性よりも必要なビット幅が小さい場合は、その必要最小ビット幅に自動調整されるが、ビット幅属性よりも多くのビット幅が必要である場合は、#pragma によるビット幅属性を採用する。
- 定数乗算の加算展開：乗算オペランドの一つが定数の場合、もう一方の変数オペランドに関するシフト・加算へ展開する。例えば、y = x * 5 は y = (x << 2) + x に変換される。
- 除算・剰余算のシフト・減算展開：除算（/）と剰余算（%）は、long division 法（古典的な 2 進除算法）によるシフト・条件付き減算の繰返しによって求まるが、これら一連のシフト・条件付き減算を組合せ回路として展開する。
- 冗長演算除去：上記の記述変換後に冗長演算部分を除去する。具体的には、「共通部分式除去」(common subexpression elimination)、「定数伝搬」(constant propagation)、「無効演算除去」(dead-code elimination)などの処理が中心となる。

## 1.11 C2RTLフレームワークRTL合成処理

RTL 合成処理は、主に以下のステップから成る。

- 演算器ゲートモデル生成：演算種別や入力・出力ビット幅によりゲート規模見積りとゲート遅延見積りを行う。
- パイプライン配置・パイプライン境界最適化：レジスタ変数やメモリ変数の順方向参照（代入後に参照）によって定義されるクロック境界（代入から 1 クロック後に参照可能になるため、代入式と順方向参照式の間にクロック遅延が発生する）を基に RTL 回路構造のパイプライン境界を配置し、クロック境界を交差する信号にパイプラインレジスタを挿入する。さらに、各パイプライン段の論理遅延を可能な限り均等化し、

パイプラインレジスタが最小になるようにパイプライン境界（パイプラインレジスタの挿入位置）が調整される。

- RTL モデル出力:RTL モデル出力ファイルは､RTL 等価 C モデル､論理合成可能 verilog ファイル、verilog シミュレーション用テストベンチが含まれる。

## 1.12 C2RTLフレームワークRTL動作検証フロー

### 1.12.1 LLVM_C2RTL_Applications（C2RTLサンプルコード）

詳細については、別資料「LLVM_C2RTL_Applications_mm_dd_yy.docs」を参照。

### 1.12.2 clangオプション

LLVM-C2RTL の起動方法は以下のコマンドによって、C/C++記述から RTL モデル生成までを実行する。

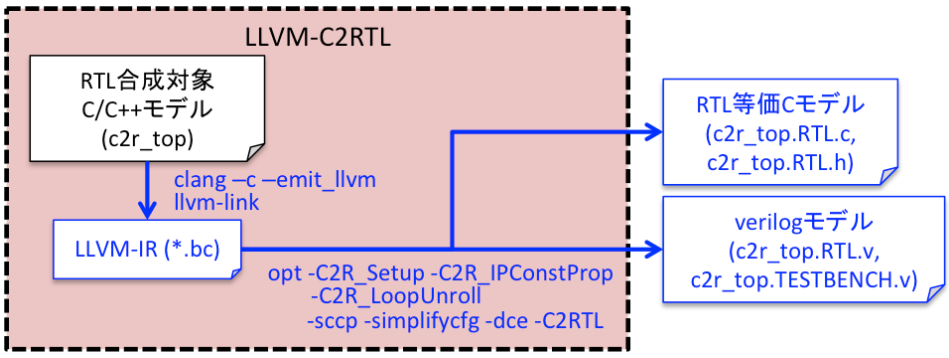
- clang -c2rtl < src_c_cpp >

ここで、< src_c_cpp >は、RTL 合成対象トップ関数から到達可能なすべての関数を含んだ１つ以上の C/C++ソースファイル名である。

上記-c2rtl オプションは、clang ドライバーによって以下の一連の処理を実行する。

1) LLVM-IR 変換：clang -c -emit-llvm -O1 -D"__LLVM_C2RTL__" –g < src_c_cpp >
   -c -emit-llvm によって、C/C++ファイルは*.bc ファイル（LLVM-IR 中間ファイル）に変換される。-g は dwarf デバッグ情報生成のためであり、構造体のメンバー変数名等の抽出で使用される（出力 RTL モデルの可読性のため)。-D”__LLVM_C2RTL__”は、C/C++記述中で、RTL 合成から除外したいコード（デバッグ表示など）を埋め込む時に使用するマクロである。#if !defined(__LLVM_C2RTL__) ～ #endif のコード領域は、RTL 合成から除外される。
2) LLVM-IR 中間ファイル結合：llvm-link < src_bc > -o < link_bc >
   ここで、< src_bc >は< src_c_cpp >の LLVM-IR ファイル（*.bc）であり、これら複数の*.bc ファイルを一つの*.bc （< link_bc >）に結合する。
3) C2RTL 実行：opt -C2R_Setup -C2R_IPConstProp -C2R_LoopUnroll -sccp -simplifycfg -dce -C2RTL < link_bc > -o < opt_bc >
   LLVM optimizer (opt)の各最適化エンジンを適用後、opt のエンジンとして登録されている C2RTL を実行する。RTL モデル出力の他、C2RTL へ入力される LLVM-IR 中間ファイルを < opt_bc >に出力する。C2RTL が起動するまでに適用される最適化エンジンの処理内容は以下の通りである。
   a) -C2R_Setup：RTL 合成対象トップ関数から到達可能な関数を抽出
   b) -C2R_IPConstProp：関数間の定数伝搬 (interprocedural constant propagation)
   c) -C2R_LoopUnroll：完全ループ展開

d)　-sccp -simplifycfg -dce：完全ループ展開後、冗長プログラム構造を除去




### 1.12.3 出力ファイル

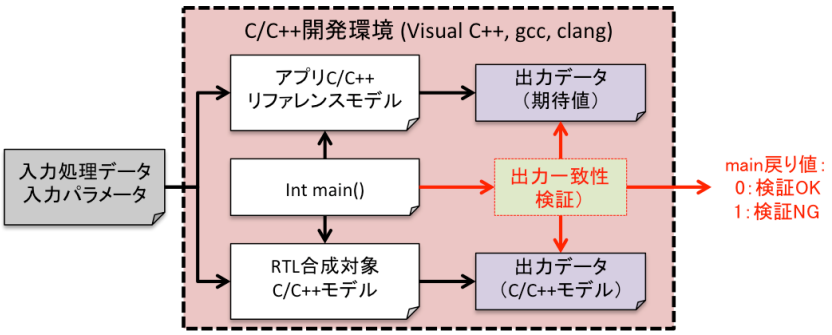
RTL 合成対象トップ関数の定義を含むソースファイルが置かれているディレクトリを c2r_src とし、RTL 合成対象トップ関数名を c2r_top_func とした場合、LLVM-C2RTL 実行後、以下の出力ファイルが生成される。

- LLVM-IR 中間ファイル：c2r_src/に各種中間ファイルが出力される。ただし、clang -c2rtl で起動した場合は、中間ファイルは c2r_src には出力されない。なお、LLVM_C2RTL_Applications で提供している Makefile には、c2r_src/obj/rtl/にこれら中間ファイルが出力されるように設定されている。
- RTL 等価 C モデル：c2r_src/C2R_ROOT/RTL_C/c2r_top.RTL.c, c2r_top.RTL.h
- 論理合成可能 verilog ファイル：c2r_src/C2R_ROOT/RTL_V/c2r_top.RTL.v
- verilog テストベンチ：c2r_src/C2R_ROOT/RTL_V_TB/c2r_top.TESTBENCH.v

### 1.12.4 C2RTL設計検証フロー

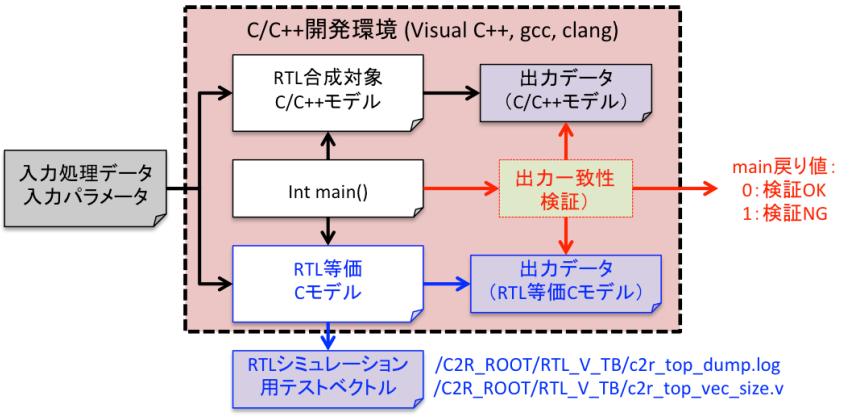
C2RTL フレームワークにおける各種モデル検証は、以下の手順で行う。

1)　RTL 合成対象 C/C++モデル動作検証：最初の検証ステップは、RTL モデルの元となる C/C++モデルの動作検証である。このためには、RTL 合成対象 C/C++モデルを駆動制御するテストベンチ関数が必要となる。下図において、main 関数が内部のテストベンチ関数を介して、RTL 合成対象 C/C++モデルを駆動する。



RTL 合成対象 C/C++モデルのベースとなるリファレンスモデルがある場合は、このリファレンスモデルとの出力データの比較により、RTL 合成対象 C/C++モデルの動作確認を行う。リファレンスモデルが存在しない場合は、RTL 合成対象 C/C++モデルの出力データの正当性を何らかの方法で確認する必要がある（画像データの視覚確認、ノイズ測定など）。

2) RTL 等価 C モデル動作検証： RTL 合成対象 C/C++モデルの動作検証後、LLVM-C2RTL によって生成される RTL 等価 C モデルの動作検証を行う。下図において、RTL 等価 C モデルは、RTL 合成対象トップ関数と同じ関数インターフェースを持つため、前記テストベンチ関数から RTL 等価 C モデルを駆動制御することで、同じ動作検証環境を流用でき、スムーズな動作検証が可能となる。また、RTL 等価 C モデルの動作検証を実行する際に、各サイクルの入出力値をテストベクトルとしてファイルにダンプする仕組みも RTL 等価 C モデルに組み込まれている。




なお、テストベクトルのダンプ機能を無効化するためには、

#define C2R_DISABLE_VECTOR_OUTPUT

を RTL 等価 C モデルをインクルードする前の箇所に挿入する。テストベクトルのダンプ機能を有効にすると、RTL 等価 C モデル検証時間が数倍遅くなるため、モデル開発・チューニングの途上で、verilog シミュレーションを行わない場合は、テストベクトルのダンプ機能を無効化することで、RTL 等価 C モデル検証時間を短縮することができる。

3) RTL 等価 C モデル動作検証における注意点： RTL 等価 C モデル動作検証における注意点は、RTL 合成対象 C/C++モデルは、それ自体パイプライン処理が反映されないのに対し、RTL 等価 C モデルは、正確なサイクル精度の回路動作を反映するため、パイプライン遅延が発生する（パイプライン段数のクロック数だけ出力が遅延する）ことである。例えば、以下の関数を RTL 合成対象トップ関数とし、3 段パイプラインで RTL 合成することを考える。

---

**/// RTL 合成対象トップ関数：**

```
int top_func(int a, int b) { return func(a, b); }
/// RTL 等価 C 関数
int top_func_RTL(int a, int b) { return top_func_RTL_core(a, b);}
```

---

**ここで、**top_func の戻り値はそのまま返されるが、RTL 等価 C 関数である top_func_RTL の戻り値は、3 段パイプラインが埋まる 3 クロック後である（つまり、top_func_RTL を 3 回呼んだ時の戻り値が 1 回目の本来の戻り値となる）。なお、パイプラインが埋まるまでの戻り値は 0 になるように RTL が合成される。このようにパイプライン遅延を反映した検証を行うためには、例えば以下の方法が可能である。

---

```
/// RTL 合成対象トップ関数：
int top_func(int a, int b, int *out) { *out = func(a, b); return 1;}
/// RTL 等価 C 関数
int top_func_RTL(int a, int b, int *out) {
    return top_func_RTL_core(a, b, out);
}
```

---

**この場合、**top_func の戻り値は常に 1 であり、処理結果の出力は*out に書き込まれる。一方、top_func_RTL の戻り値は、3 段パイプラインが埋まる 3 クロック後に戻り値 1 となり、それまでは戻り値 0 となるため、戻り値が 1 になった時から*out 値のサンプルを開始することで、RTL 等価 C モデルのパイプライン遅延を吸収することができる。

4) verilog 動作検証： RTL 等価 C モデルの動作検証によって、verilog 検証用テストベクトルが生成（verilog テストベンチと同じディレクトリに出力）されるので、C2R_ROOT/RTL_V/c2r_top.RTL.v と C2R_ROOT/RTL_V_TB/c2r_top.TESTBENCH.v を verilog シミュレータを使った verilog 動作検証を行う。RTL 等価 C モデルと出力結果が全て一致するか否かを確認するメッセージが出力される。（RTL-VERIFY OK!!!というメッセージが verilog シミュレータで出力されれば、出力が完全一致したことを示す）

### 1.12.5 C2RTLオプションファイル（c2r_options.log）

RTL 合成対象トップ関数を含んだソースファイルが置かれたディレクトリ（例：c2r_src/）に c2r_options.log（C2RTL オプションファイル）が存在する場合、このオプションファイルに記述された各種パラメータを読み込み、C2RTL 内部処理にこれらパラメータを反映させる。大部分のオプションは、C2RTL 内部処理のログ出力制御（コンソール出力、ファイル出力）に関するものであるが、RTL モデル出力に直接影響を与えるパラメータを以下に示す。

- プローブ信号定義：RTL 等価 C モデルと verilog モデルにデバッグ用のプローブ信号を定義することができる。

```
BEGIN_PROBE
    ＜C/C++変数名＞ ＜C/C++変数名＞ …
END_PROBE
```

定義されたプローブ信号は、verilog シミュレーションにおいて、入出力信号とともにテストベクトルに埋め込まれる。

- 乗算器構造指定：C/C++記述で c = a * b;という乗算の記述を、ゲートレベルの回路構造記述に詳細化することができる。

```
MultType    C2R::MULT_direct
```

  - C2R::MULT_direct：ゲートレベルの構造詳細化を行わず、c = a * b;の形式のまま verilog に出力する。
  - C2R::MULT_array：部分積（a と b[i]の論理積）の総和の形式に詳細化する。
  - C2R::MULT_booth：radix-4 Booth recoding 手法による部分積の総和の形式に詳細化する。
  - C2R::MULT_array_wallace：部分積（a と b[i]の論理積）の総和計算を Wallace-tree 加算器で詳細化する。
  - C2R::MULT_booth：radix-4 Booth recoding 手法による部分積の総和計算を Wallace-tree 加算器で詳細化する。

（今後、オプション説明項目を追加予定）

# 2. C2RTLによるプロセッサモデル記述

C2RTL フレームワークでは、前節の説明の通り、極めて多様な RTL 構造を C/C++記述で表現可能である。特に「フロー制御付きパイプライン記述方式」の場合、一般的な「同期型論理回路構造」はほぼすべて表現可能であると考えられる。プロセッサ設計に関しては、以下の構成要素・構造的特徴はすべて C2RTL フレームワークにおいて C/C++記述ベースで設計可能である。

- メモリサブシステム：
  - プログラムメモリ・データメモリ分離型構成（Harvard Architecture）
  - プログラムメモリ・データメモリ共有型構成（OS サポート用、load/store 命令で structural hazard が発生）
  - キャッシュメモリ（プログラム・データ）：外部メモリとのインターフェース回路含む（LLVM-C2RTL 前の旧 C2RTL システムにおいて実装済みであり、今後現在の LLVM-C2RTL 環境用に移植予定）
  - Memory Management Unit、SDRAM コントローラ：開発中
- IO サブシステム：
  - UART：ピン精度（rx, tx: 1-bit ずつ）、baud-rate 等含めた転送方式パラメータを SW で設定可能
  - FIFO：データ幅変更可能（8-bit～64-bit 以上）→画像処理等のストリーミング処理を簡単に実装可能（制御方式：strobe_out/ready_in のハンドシェーク方式）
  - VGA：ビデオ出力
  - その他：I2C, I2S（オーディオ入出力）
- バスシステム：現段階で未実装であるが、AMBA/AXI 等の標準バスインターフェースの C 記述ベースの設計は可能と考えられる。

本章では、C2RTL フレームワークと合わせて提供されるテンプレートプロセッサ（TCT プロセッサ）と、TCT プロセッサ記述を元に実装した RISC-V プロセッサ記述について説明する。なお、これら 2 つのプロセッサ記述は、LLVM_C2RTL_Applications の下記ディレクトリを参照のこと。

- TCT プロセッサ：/TCT_Appl/Applications/TCT_ProcModel/Src/CORE/TCT.v1/
- RISC-V プロセッサ：/TCT_Appl/Applications/TCT_ProcModel/Src/CORE/RISC_V.v1/

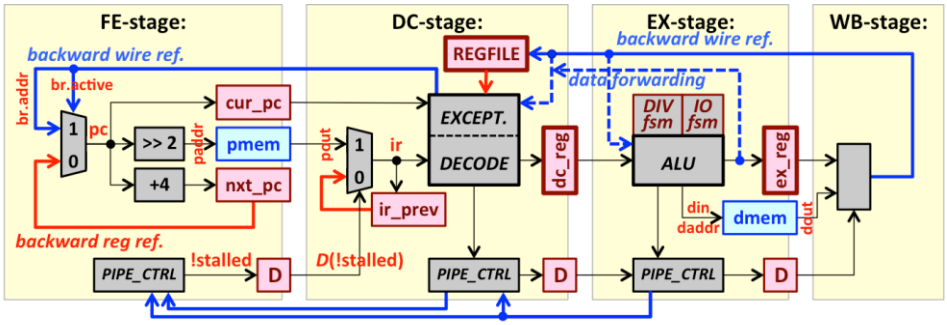
（本章については、今後詳細情報を追加予定。。。）

## 2.1 TCTプロセッサ概要

TCT プロセッサ（TCT: Tightly-Coupled Thread）は、東工大開発の RISC プロセッサであり、マルチコア SoC アーキテクチャにおける高速プロセッサ間通信機構を持つプロセッサエレ

メントとして当初 RTL 設計で開発された。その後、Synopsys 社 Processor Designer の LISA 言語に基づくプロセッサ記述に移植され、様々な ASIP 開発（ASIP: Application-Specific Instruction-set Processor）のベースプロセッサとして利用されてきた。最近では、Synopsys 社 IP Designer（nML 言語）への移植も完了しており、Computer Vision 系の ASIP 開発が進められている。

TCT プロセッサアーキテクチャは以下の特徴を持つ。




フロー制御付きパイプライン構造
（4段パイプラインRISCプロセッサ）

- 命令・データ幅：32 ビット
- パイプライン段数：4 段（fetch, decode, execute, writeback）
- メモリ：プログラム・データ分離型（Harvard Architecture）
- 32 個汎用レジスタ：そのうち R26 – R31 は特殊用途として使われる。
  - LKR (R26)：リンクレジスタ（関数リターンアドレス格納）
  - R27：（プロセッサ通信制御用：reserved）
  - SP (R28)：スタックポインタ
  - BOS (R29)：グローバルオフセット（データメモリアドレシング用）
  - R30, R31：（プロセッサ通信制御用：reserved）
- RISC 型命令セット：
  - ロード・ストア
  - 算術演算、シフト演算、論理演算、比較演算
  - 分岐、コール・リターン、HALT、NOP

### 2.1.1 TCTプロセッサ命令セット

| type | instruction | operation |
|---|---|---|
| LOAD | **LDM<dtype>** Rd, Ra, #imm17 | Rd ← (dtype) DMEM[Ra + #imm17] |
| | **LDMR<dtype>** Rd, Rs1, #imm12 | Rd ← (dtype) DMEM[Rs1 + #imm12] |
| | **LDMR2<dtype>** Rd, Rs1, Rs2 | Rd ← (dtype) DMEM[Rs1 + Rs2] |
| STORE | **STM<dtype>** Rs2, Ra, #imm17 | DMEM[Ra + #imm17] ← (dtype) Rs2 |
| | **STMR<dtype>** Rs2, Rs1 | DMEM[Rs1] ← (dtype) Rs2 |

| | | |
|---|---|---|
| COMP | ***OP*** Rd, Rs1, Rs2 | Rd ← Rs1 *OP* Rs2 |
| | ***OP*** Rd, Rs1, #imm12 | Rd ← Rs1 *OP* #imm12 |
| | ***OP*** Rd, #imm17 | Rd ← Rd *OP* #imm17 |
| | **ADDI** Rd, Rs1, #imm17 | Rd ← Rs1 + #imm17 |
| BRANCH | **JMP** #imm27 | PC ← PC + #imm27 |
| | **CALL** #imm27 | DMEM[SP – 4] ← LKR, LKR ← PC + 4, PC ← PC + #imm27 |
| | **BEQ** #imm27 | if(R0 == 0) PC ← PC + #imm27 |
| | **BNEQ** #imm27 | if(R0 != 0) PC ← PC + #imm27 |
| | **JMP_REG** | PC ← R0 |
| | **CALL_REG** | DMEM[SP – 4] ← LKR, LKR ← PC + 4, PC ← R0 |
| | **RTN** | PC ← LKR, LKR ← DMEM[SP – 4] |
| MISC | **HALT** | halt |
| | **NOP** | nop |
| IO | **IOread** Rd, devID, mode, param | Rd ← IO[devID](mode, param) |
| | **IOwrite** Rs1, devID, mode, param | IO[devID](mode, param) ← Rs1 |

- dtype：SC(signed byte), UC(unsigned byte), SS(signed half-word), US(unsigned half-word), W(word) → “W”は命令ニューモニックでは省略
  - 例（LDM 命令）：LDMSC（Load signed byte）、LDMUC（Load unsigned byte）、LDMSS（Load signed half-word）、LDMSS（Load unsigned half-word）、LDM（Load word）
  - リターン命令：LKR レジスタのロード命令として実装（dtype = RTN）
- OP：演算命令（算術・シフト・論理・比較）
  - Unsigned 演算のみ：+, –, ==, !=, !, ~, &, |, ^, <<, =
  - Signed/Unsigned 演算：*, /, %, <, >, <=, >=, >>
- 即値：imm12（12 ビット）, imm17（17 ビット）, imm27（27 ビット）→すべて符号拡張（2 の補数形式）
- IO 命令：実装される IO デバイスに依存
  - devID：デバイス ID（0, 1：UART, 6：FIFO, その他：reserved）
  - mode：0（データ入出力）、1（制御入出力）
  - param：制御入力時のみ有効（IO デバイスパラメータ設定用）

下表に TCT 命令セットの 32 ビットバイナリ形式を示す。

| 命令 | 31-27 | 26-22 | 21-19 | 18-17 | 16-12 | 11--7 | 6---0 |
|---|---|---|---|---|---|---|---|

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| STM/LDM | 1?010 | Rs2/Rd | dtype | Ra | #imm17 | | |
| STMR/LDMR | 1?100 | Rs2/Rd | dtype | 00 | Rs1 | #imm12 | |
| LDMR2 | 11101 | Rd | dtype | 00 | Rs1 | Rs2 | 0000000 |
| OP:COMP | 01100 | Rd | OP | | Rs1 | Rs2 | 0000000 |
| OP:COMP_I1 | 01101 | Rd | OP | | Rs1 | #imm12 | |
| OP:COMP_I2 | 01110 | Rd | OP | | #imm17 | | |
| ADD_I | 01010 | Rd | Rs1 | | #imm17 | | |
| JMP/CALL | 0010? | #imm27 | | | | | |
| BEQ/BNEQ | 0011? | #imm27 | | | | | |
| JMP_REG/CALL_REG | 10111 | ? | 00...00 | | | | |
| NOP/HALT | 000?0 | 00...00 | | | | | |

“?”：0 or 1（命令種指定）

### 2.1.2 TCTプロセッサCモデル

- TCTProcArch.h：プロセッサリソース定義
- TCTProc.h：関数プロトタイプ等
- TCTProc_io.h：IO デバイスリソース定義
- TCTProc.cpp：プロセッサパイプライン動作記述、命令セットシミュレータ（以降 ISS と呼ぶ）テストベンチ動作記述
- TCTProc_io.cpp：IO デバイス動作記述
- TCTProcUtil.cpp：ISS 用メモリ設定関数等
- ../CPP_utils/：タイマー定義、ELF ファイル読み込み機能等

CPP_utils 以外のソースは基本的に C 記述であるが、現在プロセッサ C++記述への移行作業を進めている。

## 2.2 RISC-Vプロセッサモデル

RISC-V プロセッサは、UC Berkeley で開発されたオープンソースの RISC アーキテクチャであり、RISC-V Foundation（riscv.org）によって命令セット、アーキテクチャモデル、コンパイラツールチェインなどが公開されている。ここでは、前述の TCT プロセッサの C 記述モデルをベースに RISC-V 命令セット（RV32I base instruction set、RV32M standard extension）を実装している。

- RVProcArch.h：プロセッサリソース定義
- RVProc.h：関数プロトタイプ等
- RVProc_io.h：IO デバイスリソース定義

・　RVProc.cpp：プロセッサパイプライン動作記述、ISS テストベンチ動作記述
・　RVProc_io.cpp：IO デバイス動作記述
・　RVProcUtil.cpp：ISS 用メモリ設定関数等
・　../CPP_utils/：タイマー定義、ELF ファイル読み込み機能等（TCT.v1 と共通機能）

パイプライン構造は、TCT プロセッサと同じ 4 段パイプライン（fetch, decode execute, writeback）であり、命令デコード部（decode）と命令実行部（execute）以外は、ほぼ同じコードを再利用している。

## 2.3　プロセッサCモデルの検証手順

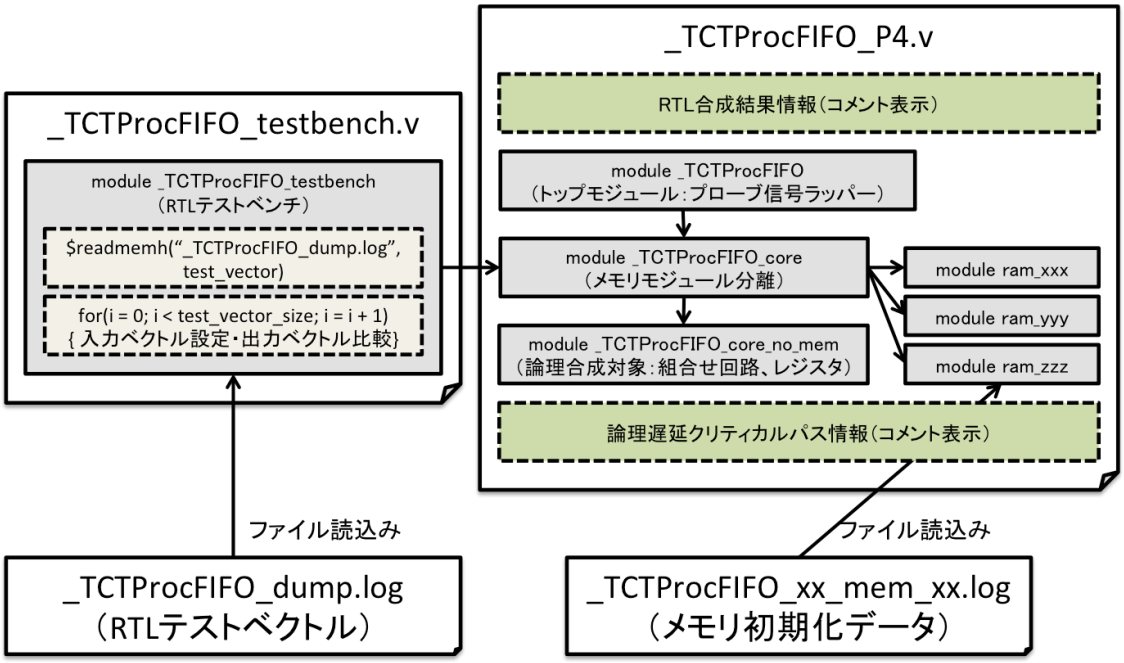
＜今後追加予定＞

## 2.4　RTLファイル出力情報

＜以下 2.4 各節は、旧 C2RTL ツールに基づく説明のため、今後 LLVM-C2RTL 用に情報更新予定＞

### 2.4.1　Verilog出力ファイル構造

Verilog ファイル出力は、論理合成可能 RTL モジュールを含むモデルと RTL 検証を行うための RTL テストベンチを含む。下図にこれらの RTL 出力の記述構造を示す。




・　RTL テストベンチ（_TCTProcFIFO_testbench.v）：内部で、論理合成可能 RTL モジュール（例：_TCTProcFIFO_core）を読込む。
　➢　テストベクトル読込み：$readmemh によって、テストベクトル（HEX 形式）を

レジスタ配列 test_vector に読込む。

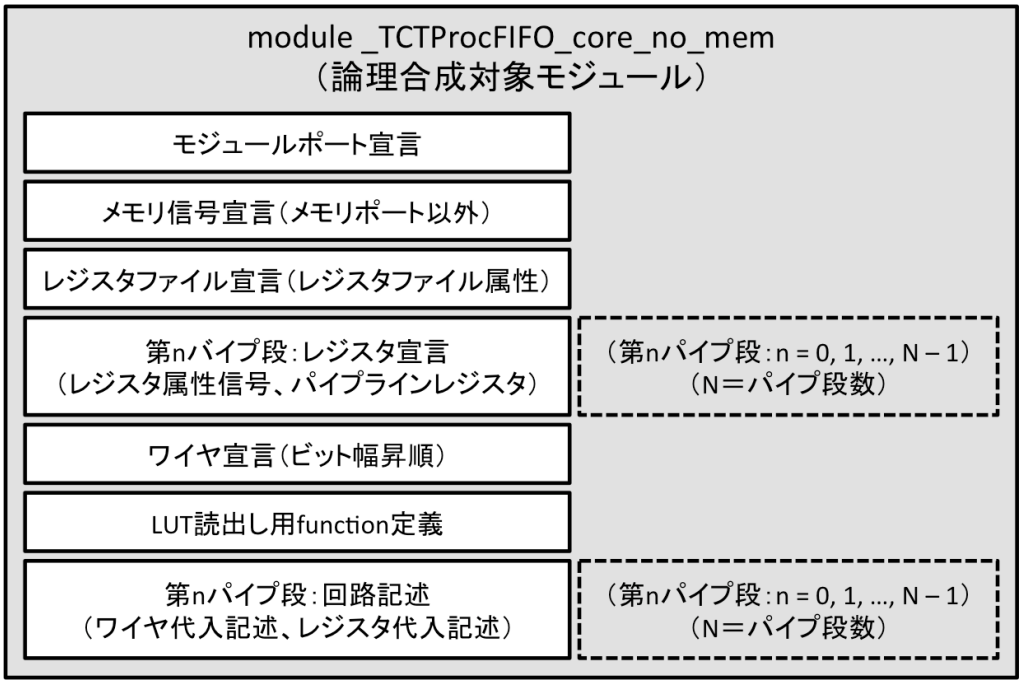
- test_vector_size：テストベクトルのサイズ（シミュレーションサイクル数と同じ）
- 入力ベクトル設定・出力ベクトル比較：各サイクルの入力ベクトルを設定し、出力ベクトルを期待値と比較する。
- RTL 合成対象コアモジュール（module _TCTProcFIFO_core）の読込み（下記説明の通り、コアモジュールにはプローブ信号がポートリストに追加されており、これらプローブ信号の出力ベクトル比較をする仕組みが提供されている）

・ 論理合成可能 RTL モデル（_TCTProcFIFO_P4.v）：RTL 合成対象トップ関数の動作を RTL で記述

- RTL 合成結果情報：ファイル先頭部分に、RTL 合成対象トップ関数の合成結果に関する情報がコメントとして埋め込まれる。
- トップモジュール（module _TCTProcFIFO）：「1.4 RTL 合成対象トップモジュールの入出力ポート判定」で説明した入出力ポート判定方法によって決定した入出力ポートリストを持つトップモジュール。その直下のコアモジュールのプローブ信号ポートをラッピングしている。
- コアモジュール（module _TCTProcFIFO_core）：「1.3.3 RTL 検証用プローブ信号指定」で説明したプローブ信号を入出力ポートに追加したモジュールであり、RTL テストベンチに読み込まれる。その内部モジュールには、メモリモデルを取り除いた論理合成対象モジュール（_TCTProcFIFO_core_no_mem）と各種メモリモデルがある。
- メモリモジュール（module ram_xxx）：同期型 SRAM の動作記述（一般的な FPGA コンパイラによって同期型 SRAM と認識される）である。「1.3.4 その他のハードウエア属性指定」で説明したメモリ初期値ファイル指定があるメモリ配列変数については、そのメモリ初期値ファイルを$readmemh で読込む。
- 論理合成対象モジュール（module _TCTProcFIFO_core_no_mem）：各メモリ信号（addr, din, dout, we, ce）をすべてモジュールポートリストに持ち、すべての組合せ回路とレジスタがパイプラインステージ毎に出力される。
- RTL 論理遅延クリティカルパス情報：ファイル末尾部分に、RTL 合成対象トップ関数の合成結果における論理遅延クリティカルパス情報がコメントとして埋め込まれる。

### 2.4.2 論理合成対象モジュール記述構造

本節では、C データフロー記述の主要部分を含む論理合成対象モジュールの記述構造（下図）を説明する。




- モジュールポート宣言：モジュールポートとして以下を含む。
  - 「1.4 RTL 合成対象トップモジュールの入出力ポート判定」で説明した入出力ポート判定方法によって決定した入出力ポートリスト（本来回路として必要なポートリスト）
  - 「1.3.3 RTL 検証用プローブ信号指定」で説明したプローブ信号
  - 各メモリポート
    - addr：アドレス信号（ビット数は、メモリのワード数で決定される）
    - din：データ入力（書込み用）
    - dout：データ出力（読出し用）
    - we：書込みイネーブル（バイト単位の書込み制御記述がある場合は、バイトイネーブルとなる）
    - ce：チップイネーブル
- メモリ信号宣言とメモリポート assign 文：論理合成対象モジュール内部では、メモリポートと異なる以下の信号が用いられる。
  - re, we：読出しイネーブル、書込みイネーブル
  - raddr, waddr：読出しアドレス、書込みアドレス

  なお、メモリアクセス動作は、読出しと書込みが排他的となることを前提に RTL 合成される。モジュール内部で使用されていないメモリポートについては、以下の assign 文が生成される。
  - ce = (re) | (|we);：チップイネーブルは、we と re の OR 結合で与えられる
  - addr = (we) ? waddr : raddr;：アドレス信号は書込みアドレスと読出しアドレスの

いずれか（we で選択）で与えられる

- レジスタファイル宣言：プラグマによるレジスタファイル属性が指定されたレジスタ配列
- レジスタ宣言（各パイプラインステージでまとめて列挙）：以下の 2 種類のレジスタを含む
  - 状態レジスタ：プラグマによるレジスタ属性が指定されたレジスタ
  - 状態レジスタの「現状態」信号（ワイヤ）：Verilog では重要ではないが、RTL 等価 C モデルのデバッグを想定して導入された信号である。レジスタ名 xxx に対して現状態信号名 xxx_prev が対応する。
  - パイプラインレジスタ：レジスタ属性のないワイヤ信号（ツールが生成する中間信号を含む）がパイプライン境界を交差するときに挿入されるパイプラインレジスタである。ワイヤ信号名 xxx とパイプラインステージ ID 番号 y（y = 0, 1, …, N – 1）（N：パイプライン段数）に対して、パイプラインレジスタ名 Py_xxx が対応する。
- ワイヤ宣言（ビット幅数の昇順で列挙）：レジスタ属性のないワイヤ信号（ツールが生成する中間信号を含む）
- LUT 読出し用 function 定義：LUT（Look up table：定数値の配列）を読出す組合せ回路用の function を case 文で定義
- パイプラインステージ回路記述（各パイプラインステージでまとめて列挙、パイプラインステージの順番は後方から前方の順番で記述）：
  - ワイヤ信号代入記述：ワイヤ信号への assign 文（右辺式の信号はワイヤ信号、レジスタ信号、メモリ信号のいずれか）
  - レジスタ信号代入記述：always@(posedge clk or negedge rst_n)の手続き文におけるリセット時のレジスタ代入式と通常クロックエッジ時のレジスタ代入式

### 2.4.3 RTL合成結果情報

RTL 合成結果情報は、論理合成対象 RTL モデルのファイル先頭部分にコメントとして埋め込まれており、RTL 合成対象トップ関数の合成結果に関する以下の情報を示している。

- c2v_options：RTL 合成処理の各種オプション設定値（後述）
- C ソース行数：各 C ソースファイルで RTL 合成で引用された行数
- RTL 合成処理時間統計：RTL 合成処理の主要工程の処理時間
- パイプラインステージゲート数・論理遅延：各パイプラインステージのゲート数（組合せ回路、レジスタ）と論理遅延のツールによる見積り値を示す。

【例】TCTProc_FIFO_P4.v で表示されるゲート数・論理遅延情報を以下に示す。

```
(41,807 total gates) 33,099 comb.gates + 2,177 registers (521 pipe-reg + 600 state-reg + 32 feedback-reg + 1,024 reg-file), maxDist = 150
pipe-stage (0) :       284 gates,    98 pipe-reg bits,    96 state bits,    0 feedback-reg bits,         0 reg-file bits, maxDist =    40, 1 memory port units
pipe-stage (1) :    8,959 gates, 178 pipe-reg bits, 262 state bits,    0 feedback-reg bits,         0 reg-file bits, maxDist =    74, 0 memory port units
pipe-stage (2) : 21,199 gates, 174 pipe-reg bits, 242 state bits,    0 feedback-reg bits,         0 reg-file bits, maxDist = 150, 1 memory port units
```

```
pipe-stage (3) :    2,657 gates,    71 pipe-reg bits,      0 state bits, 32 feedback-reg bits, 1,024 reg-file bits, maxDist =    24, 0 memory port units
```

- total gates：レジスタ 1 つを 4 ゲート換算した回路全体のゲート数
- comb. gates：組合せ回路ゲート数
- pipe-reg bits：パイプラインレジスタビット数
- state bits：レジスタ属性信号のビット数
- feedback-reg bits：逆方向ワイヤ参照において挿入される値保持用レジスタ
- reg-file bits：レジスタファイル属性信号のビット数

・ メモリ統計：使用されるメモリのワード数、データビット長

・ レジスタファイル統計：レジスタファイル属性信号のワード数、データビット長

・ operation statistics：RTL 合成の諸工程における演算種とビット長の統計

- (initial)：C データフロー記述の回路展開（ループ展開・インライン展開）後の演算量統計
- (after transformation)：定数乗算展開・除算展開処理後の演算量統計
- (after CSE)：共通部分式除去後の演算量統計
- (after optimization)：その他冗長演算除去後の演算量統計

・ C function hierarchy statistics : RTL 合成対象トップ関数以下の各関数の RTL 合成情報

【例】TCTProc_FIFO_P4.v で表示される合成情報を以下に示す。

```
(_TCTProcFIFO) : 33157 gates, 492 pipe-reg bits + 600 state bits + 32 feedback-reg bits
  (excl. sub-modules: 177 gates, 377 pipe-reg bits + 34 state bits + 32 feedback-reg bits)
  (_1_set_fifo_input_ports) : 0 gates, 0 pipe-reg bits + 0 state bits + 0 feedback-reg bits
    (excl. sub-modules: 0 gates, 0 pipe-reg bits + 0 state bits + 0 feedback-reg bits)
    (_1_1_set_fifo_FW) : 0 gates, 0 pipe-reg bits + 0 state bits + 0 feedback-reg bits
    (_1_2_set_fifo_BW) : 0 gates, 0 pipe-reg bits + 0 state bits + 0 feedback-reg bits
  (_2_fetch) : 182 gates, 33 pipe-reg bits + 96 state bits + 0 feedback-reg bits
  (_3_decode) : 8953 gates, 2 pipe-reg bits + 229 state bits + 0 feedback-reg bits
    (excl. sub-modules: 5448 gates, 2 pipe-reg bits + 158 state bits + 0 feedback-reg bits)
    .....
```

- (_TCTProcFIFO)：RTL 合成対象トップ関数以下（すべて）の RTL 合成結果のゲート数とレジスタビット数を示す。
- (excl. sub-modules: . . . )：その関数から呼び出される子関数のゲート数とレジスタビット数を除いた情報（子関数がない場合には表示されない）
  - ✧ (_1_set_fifo_input_ports)：_TCTProcFIFO 以下で呼ばれる子関数 set_fifo_input_ports の RTL 合成結果情報

→ 以降、同じレベルの子関数の RTL 合成結果情報は同じインデントレベルで表示される。

### 2.4.4 RTL論理遅延クリティカルパス情報

論理遅延クリティカルパス情報は、論理合成対象 RTL モデルの末尾部分にコメントとして埋め込まれている。論理遅延クリティカルパス情報は、特定の論理遅延量を持つ信号を列挙する「第 1」の表示形式と、各関数内部の最大論理遅延量をすべてのインスタンス化された（インライン展開された）関数について列挙する「第 2」の表示形式とで出力している。

下記は、第 1 の表示形式である特定の論理遅延量を持つ信号の列挙の例を示す。

```
/************* CriticalNodes (maxDistPipe = P2(D=150), 2nd-maxDistPipe = P1(D=74)) ***************

(P: 1)(D: 74, 74)        (_3_decode:TCTProc.c:L.301)[ID:    829]   U32[0,4294967295]   (COND:   _3_sig_2828.1)
_cpu.dc_reg.src[0] <-- _cpu.dc_stt.src[0]
(P: 2)(D: 74, 74)        (_4_execute:TCTProc.c:L.592)[ID:2812]  U1[0,1]  (COND:  _tmp_2949.0)  _4_o315_unsigned_lt
<-- (_4_o152_src0) < (_4_o162_src1)
. . .
. . .
(P: 2)(D:146,105)        (_4_execute:TCTProc.c:L.627)[ID:4149] U32[0,4294967295] (COND: _tmp_2946.1) _tmp_4149
<-- (_4_sig_3097) ? _tmp_2903 : _cpu.ex_reg.out.prv..
(P: 2)(D:148,107)        (_4_execute:TCTProc.c:L.627)[ID:2899] U32[0,4294967295] (COND: _tmp_2945.1) _tmp_2899
<-- (_tmp_2946) ? _tmp_4148 : _tmp_4149
(P: 2)(D:150,109)        (_4_execute:TCTProc.c:L.518)[ID:2688]   S33[-2147483648,4294967295]   _tmp_2688   <--
(_tmp_2945) ? _4_o64_ex_out : _tmp_2899
(P: 2)(D:150,109)        (_4_3_div_st_2:TCTProc.c:L.391)[ID:3325] U32[0,4294967295] _cpu.ex_reg.out <-- _tmp_2688
(P: 2)(D:150,109)        (_TCTProcFIFO:)[ID: 273] state__cpu.ex_reg.out
```

- (maxDistPipe = P2(D=150), 2nd-maxDistPipe = P1(D=74))：クリティカルパス情報の出力対象は、パイプラインステージの論理遅延のうち最大のもの（maxDistPipe）と 2 番目に大きいもの（2nd-maxDistPipe）の間の論理遅延を持つ信号としている。
- (P: 2)(D:146,105) (_4_execute:TCTProc.c:L.627) . . . ：各行は、パイプラインステージ ID（P: 2)、論理遅延値（D:146, 105)、C データフロー記述のソース行番号の他、信号値の範囲、組合せ回路代入式などが表示されている。
  - 論理遅延値：クロックエッジ（0→1 立上り）からの信号伝搬時間であり、ツール内部で規格化された単位時間である（1 単位時間 = 1 ゲート遅延：ゲート遅延は AND/OR など単純 2 入力ゲートを想定)。さらに、信号伝搬時間は、信号の MSB（最上位ビット）と LSB（最下位ビット）の両方で見積もられている。(D:146, 105) の表記の場合、146 が MSB 信号伝搬時間で 105 が LSB 信号伝搬時間である。

下記は、第 2 の表示形式であるすべての関数における最大遅延信号の列挙の例を示す。

```
*************** CriticalNodes @ PixPipes *****************

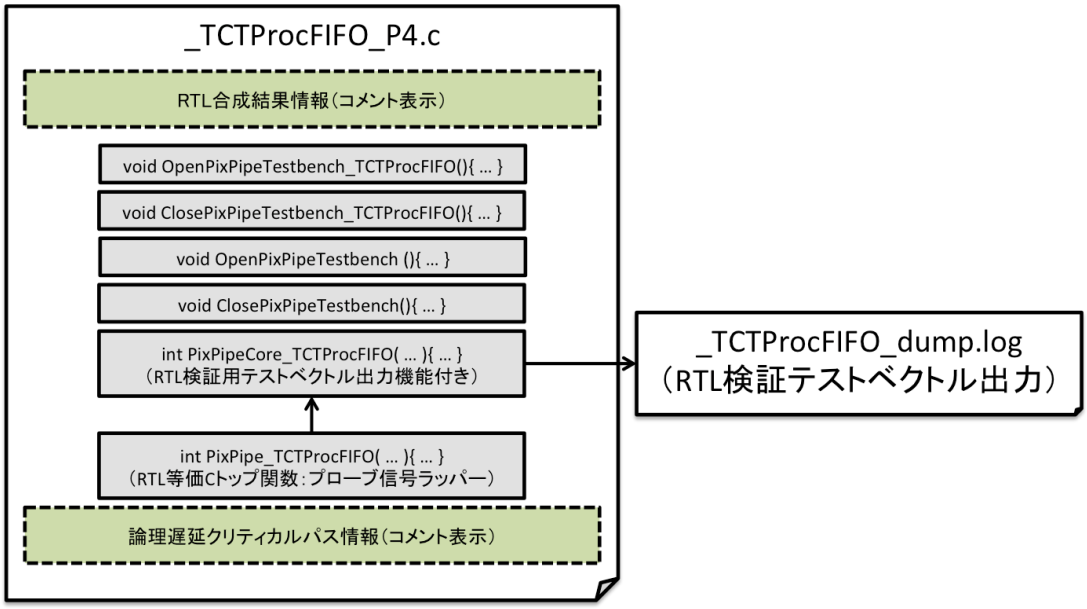
(P: 0)(D: 40,   9)       (_2_fetch:TCTProc.c:L.18)[ID:  440]  U32[0,4294967295]  (COND:  _2_sig_2483.0)  _2_sig_2496
<-- (_2_o45_pc) + (4)
(P: 2)(D: 40, 40)        (_4_11_zol_ex:TCTProc.c:L.200)[ID:3838]   U32[0,4294967295]   (COND:   _4_11_sig_2003.0)
_cpu.zol.total_count_EX <-- _4_o31_src0bp
(P: 1)(D: 41, 41)        (_3_8_dec_c2:TCTProc.c:L.60)[ID:1801]   U5[0,31]   (COND:   _3_sig_2672.1)   _tmp_1801   <--
(_3_sig_2689) ? _cpu.insn.rf1 : _tmp_1884
. . .
. . .
(P: 2)(D: 96, 96)        (_4_10_div_st_1:TCTProc.c:L.360)[ID:3676]        U64[0,9223372036854775800]        (COND:
_tmp_2945.1) _tmp_3676 <-- (_tmp_2946) ? _4_3_o33_f_next : _tmp_4186
(P: 2)(D:150,109)        (_4_3_div_st_2:TCTProc.c:L.391)[ID:3325] U32[0,4294967295] _cpu.ex_reg.out <-- _tmp_2688
(P: 2)(D:150,109)        (_4_execute:TCTProc.c:L.518)[ID:2688]   S33[-2147483648,4294967295]   _tmp_2688   <--
(_tmp_2945) ? _4_o64_ex_out : _tmp_2899
(P: 2)(D:150,109)        (_TCTProcFIFO:)[ID: 273] state__cpu.ex_reg.out
```

## 2.5 RTL等価Cモデル記述構造

＜以下 2.5 各節は、旧 C2RTL ツールに基づく説明のため、今後 LLVM-C2RTL 用に情報更新予定＞

RTL 等価 C モデルの出力ファイルは、論理合成可能 Verilog 記述出力と似た記述構造で出力

される。また、RTL 検証テストベクトルを出力する機能も組み込まれている。下図に、RTL 等価 C モデルの記述構造を示す。



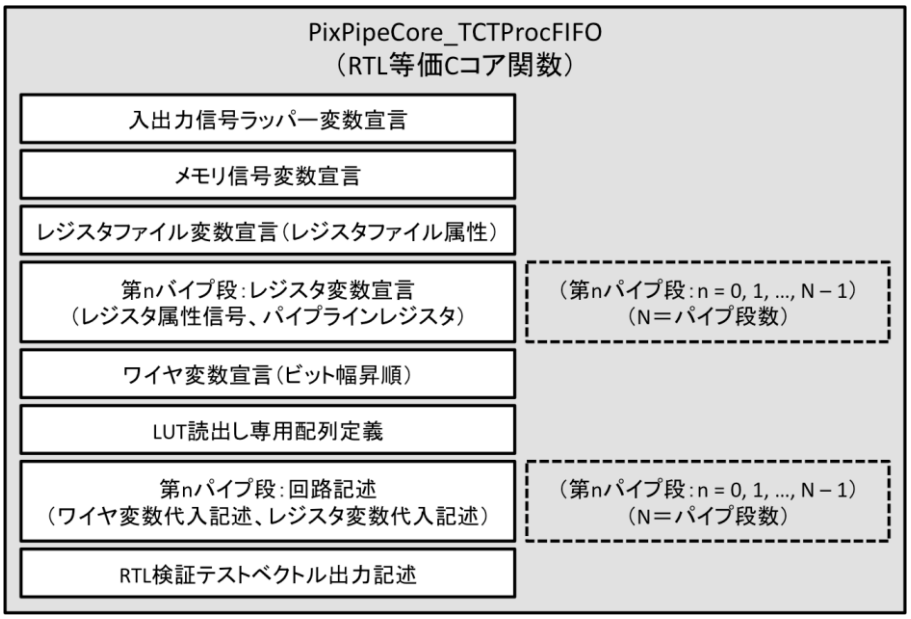

- RTL 合成結果情報（Verilog 出力と同じ情報）：ファイル先頭部分に、RTL 合成対象トップ関数の合成結果に関する情報がコメントとととして埋め込まれる。
- RTL テストベクトル出力ファイル管理：
  - OpenPixPipeTestbench(){…}：RTL テストベクトル出力ファイルをオープンする。
  - ClosePixPIpeTestbench(){…}：RTL テストベクトル出力ファイルをクローズする。

  【補足】特殊な使用法として、main 関数から複数の RTL 合成対象トップ関数（プラグマで指定）を呼び出すことも可能である。このような場合では、RTL 検証はそれぞれのトップ関数から合成される RTL トップモジュールを単体検証することを前提に、個別の RTL 検証テストベクトルが生成される。そのため、OpenPixPipeTestbench と ClosePixPipeTestbench は、すべての RTL トップモジュール用 RTL 検証テストベクトル出力ファイルを管理し、これら関数から個別 RTL 検証テストベクトルファイルの管理用関数を呼出す構造になっている（OpenPixPipeTestbench_TCTProcFIFO(), ClosePixPipeTestbench_TCTProcFIFO()等）。
- RTL 等価 C コア関数（PixPipeCore_TCTProcFIFO()）：論理合成対象モジュール記述（Verilog）と同じ記述構造を持つ。また、RTL 検証テストベクトル出力機能も組込まれている。RTL 等価 C コア関数の引数は、プローブ変数も含まれる。詳細は後述。
- RTL 等価 C トップ関数（PixPipe_TCTProcFIFO()）：RTL 合成対象トップ関数と同じ引数型を持ち、内部で RTL 等価 C コア関数を呼出す。RTL 等価 C コア関数のプローブ変数引数のラッパー機能を持つ。
- RTL 論理遅延クリティカルパス情報（Verilog 出力と同じ情報）：ファイル末尾部分に、RTL 合成対象トップ関数の合成結果における論理遅延クリティカルパス情報がコメン

トとして埋め込まれる。

### 2.5.1 RTL等価Cコア関数記述構造

RTL 等価 C コア関数は、論理合成対象モジュール記述（Verilog）と同じ記述構造を持ち、RTL 検証テストベクトル出力機能を組込んでいる（下図）。




- RTL 等価 C コア関数の引数：RTL 合成対象トップ関数の引数とプローブ変数を含む。
- 入出力信号ラッパー変数宣言：RTL 等価 C コア関数の内部は、すべて整数型データのみで記述されており、これら内部変数と関数引数（構造体ポインタ型を含む）とのラッピングを行うための入出力ラッパー変数を宣言する。
- メモリ信号変数宣言：メモリ配列変数は、RTL 等価 C コア関数定義の直前にグローバル変数として定義されており、これらメモリ配列変数へのアクセスのためのアドレス変数、データ変数、イネーブル変数が宣言される。
  - re, we：読出しイネーブル、書込みイネーブル
  - raddr, waddr：読出しアドレス、書込みアドレス
  - dout, din：読出しデータ、書込みデータ
- レジスタファイル変数宣言：プラグマによるレジスタファイル属性が指定された static 型レジスタ変数配列
- レジスタ変数宣言（各パイプラインステージでまとめて列挙）：以下の 2 種類のレジスタを含む
  - 状態レジスタ変数（static 型）：プラグマによるレジスタ属性が指定されたレジスタ変数
  - 状態レジスタの「現状態」信号（auto 型：非明示）： RTL 等価 C モデルのデバッグを想定して導入された信号であり、レジスタ名 xxx に対して現状態信号名

xxx_prev が対応する。

- パイプラインレジスタ（static 型）：レジスタ属性のないワイヤ信号（ツールが生成する中間信号を含む）がパイプライン境界を交差するときに挿入されるパイプラインレジスタである。ワイヤ信号名 xxx とパイプラインステージ ID 番号 y（y = 0, 1, …, N – 1）（N：パイプライン段数）に対して、パイプラインレジスタ名 Py_xxx が対応する。

・ ワイヤ変数宣言（auto 型：非明示）（ビット幅数の昇順で列挙）：レジスタ属性のないワイヤ変数信号（ツールが生成する中間信号を含む）

・ LUT 読出し専用定数配列定義：LUT（Look up table：定数値の配列）を定数配列として定義

・ パイプラインステージ回路記述（各パイプラインステージでまとめて列挙、パイプラインステージの順番は後方から前方の順番で記述）：

- ワイヤ変数代入記述：ワイヤ変数への代入文（右辺式の信号はワイヤ変数、レジスタ変数、メモリ変数のいずれか）
- レジスタ変数代入記述：レジスタ変数への条件付き代入文

・ RTL 検証テストベクトル出力記述：RTL 等価 C コア関数内部で、すべての変数（信号）評価が終了した後に、RTL 検証テストベクトルをファイルに出力する。テストベクトル出力は 2 種類用意されており、2 つのマクロ定義によって制御されている。

```
#if !defined(DISABLE_PIXPIPE_LOG) && !defined(DISABLE_VECTOR_OUTPUT)
        if(fp_TCTProcFIFO_log != 0){ … } ///  テキスト表示用
#endif
```

```
#if !defined(DISABLE_VECTOR_OUTPUT)
        if(fp_TCTProcFIFO_dump != 0){ … } /// verilog の$readmemh 用
#endif
```

- DISABLE_VECTOR_OUTPUT：このマクロが定義されていると、テストベクトル出力はすべて無効化される（シミュレーション時間短縮のため）。RTL 等価 C モデルファイルでは、定義されていない。
- DISABLE_PIXPIPE_LOG：このマクロが定義されていると、「テキスト表示用テストベクトル出力」（テストベクトルが見易いように多少整形されている）が無効化される。RTL 等価 C モデルファイルの先頭部分で定義されているため、有効化するためには、この部分をコメントアウトするか、#undef によって定義を無効化する。

## 2.6 C2RTL合成ログファイル

＜今後追加予定＞

## 2.7 C2RTLツールエラーと対処法

＜今後追加予定＞

# 3. C2RTLフレームワークによるASIP開発メソドロジー

本章では、C2RTL フレームワークによる ASIP 開発メソドロジーについて説明する。尚、本章は TCT プロセッサをベースとした説明であるが、RISC-V ベースでも適用可能なメソドロジーである。＜本章の説明は、旧 C2RTL システムに基づいているため、今後 LLVM-C2RTL 環境用に情報を更新予定である。＞

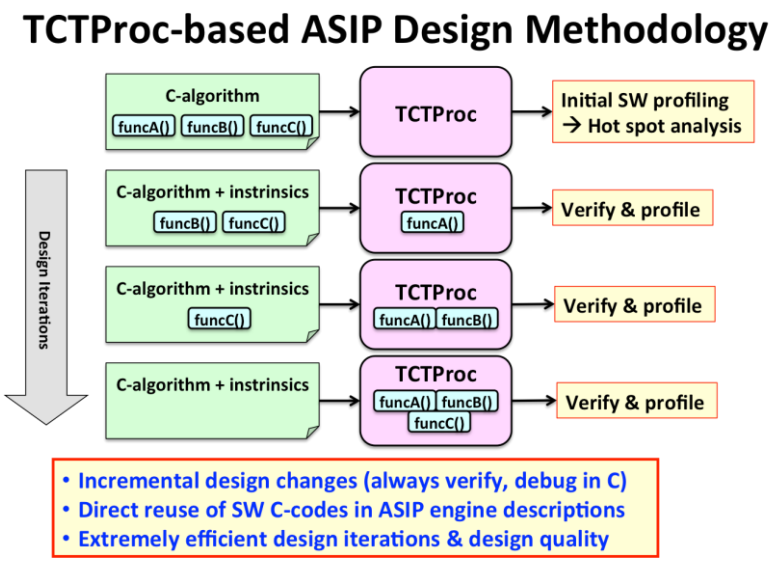
## 3.1　設計入力モデル

TCT-ASIP 開発における設計入力は、ターゲットアプリケーションの処理・機能を定義する「アプリ C リファレンスモデル」と、ASIP のベースとなる「TCT プロセッサ C モデル」である。

- アプリ C リファレンスモデル：ターゲットアプリケーションの動作要求仕様に対して出来るだけ広い範囲の動作検証が可能であること（多様なテスト入力が与えられること、多様な動作パラメータがテストできること、等）が望ましい。アプリ C リファレンスモデルは、C 開発環境の移植性を十分に考慮した設計になっているとは限らないので、C 開発環境・OS 等を変えるだけでアプリ挙動が変化する可能性を考慮する必要がある。
- TCT プロセッサ C モデル：特に入出力 IO 仕様については予め準備されたもの（例えば、2 章で説明した FIFO 付き TCT プロセッサによって画像データの入出力手段を確保する）であることが望ましい。

## 3.2　ASIP開発フロー概略

C2RTL フレームワークを活用した ASIP 開発アプローチは、アプリ記述とプロセッサ記述を C 言語で共通化したことのメリットをそのまま享受できるようにすることが基本的な考え方となる。下図にその ASIP 開発アプローチの概略を示す。




ASIP 開発の主要部分は、アプリ C リファレンスモデルの各コア関数（ホットスポット）を TCT プロセッサ C モデル内部に埋め込むことでこれらコア関数をハードウエア実装（コアエンジン化）し、専用命令でこれらコアエンジンを実行制御する機構を TCT プロセッサ C モデルに実装することである。具体的な ASIP 開発工程は以下の流れになる。

1. アプリ C リファレンスモデルの TCT プロセッサ実行検証環境の構築（アプリ C 実装モデル構築）：
   - TCT プロセッサ C モデルで実行検証可能なアプリ C リファレンスモデルのコード移植
   - アプリ C リファレンスモデルと TCT プロセッサ C モデルの実行結果の動作一致性の確認する仕組みの構築
   - TCT プロセッサ C モデルにおける実行プロファイル取得（実行サイクル数等）
2. TCT プロセッサ C モデルの ASIP 拡張命令用リソース定義：
   - アプリ固有拡張命令クラスコード定義、アプリ固有拡張命令 OP コード定義
   - 拡張レジスタ・メモリのリソース定義、リソース ID 定義
3. インクリメンタルな ASIP 拡張命令実装：
   - アプリ C 実装モデル：ASIP 拡張命令関数（TCT-intrinsic 関数）定義とアプリ C 実装モデルの ASIP 拡張命令への置き換え（TCT-intrinsic 関数による拡張命令プログラミングは、独自開発 TCT コンパイラによるプログラミング技法であり、gcc/clang 等の標準コンパイラ環境では、拡張命令用コンパイラ機能拡張の開発が必要である。Clang/LLVM ベースの拡張命令コンパイラ開発環境は今後取り組む予定である。）
   - TCT プロセッサ C モデル：ASIP 拡張命令の実装（命令デコード記述、命令実行動作記述）

- 検証項目：
  ① ASIP 拡張命令使用アプリ C 実装モデルとアプリ C リファレンスモデルの動作一致性
  ② ASIP 拡張命令使用アプリ C 実装モデルの実行プロファイル取得

## 3.3 アプリCリファレンスモデルのTCTプロセッサ実行検証環境の構築（アプリC実装モデルの構築）

アプリ C リファレンスモデルは、計算機環境で実行するシミュレーションモデルとして実装されている場合も多く、最終的な実装ターゲットである TCT-ASIP 実行環境への移植作業（アプリ C 実装モデル構築）が必要になる。3.4 節「C2RTL フレームワーク C コンパイラ」で説明したように、C 標準ライブラリ関数は TCT-ASIP 実行環境では使用できない（TCT プロセッサ実行ファイルではこれらライブラリ関数コールは NOP 命令に置き換えられる）ので、その部分の書き換え（ポーティング）が必要になる。

- ファイル IO 関数（fopen, fclose, fread, fwrite, fscan, fprintf）：処理データ入力、処理データ出力、パラメータ設定等のために、通常はファイル入出力を用いる場合が多いが、これらをファイル IO 無しで実現する必要がある。
- コンソール IO 関数（printf, scanf）：メッセージ入出力
- 動的メモリ管理（malloc, free）：様々なワークスペース確保で必要となる動的メモリ確保をすべて静的配列で実装する必要がある。

上記 C 標準ライブラリ関数のポーティング作業の他に、アプリ C リファレンスと TCT プロセッサ C モデルの動作一致性検証も実装する必要がある。

### 3.3.1 コンソールIO処理のプロセッサIOポート移植

アプリ C リファレンスモデルにおけるコンソール IO 処理は、プロセッサ IO ポート（例えば FIFO ポート）へ移植することができる。2.3.11～2.3.14 で説明したプロセッサ FIFO ポート機能と外部 FIFO テストベンチモデルは、以下の FIFO チャンネル ID 設定機能がある。

- FIFO チャンネル ID = 0：処理用データ転送チャンネル
- FIFO チャンネル ID = 1：文字データ転送チャンネル

また、3.5.3 で説明した TCT-ASIP 拡張命令関数定義の例では、void fifo_tx_1(int c){ … }が文字データ出力用拡張命令用 API 関数である。これらのプロセッサ IO 機能を活用した printf 関数の移植のための補助関数の例を以下に示す（/Applications/TCT_ProcModel/Src/TEST/myJPEG5_2/myJPEG_ext.c を参照）。

```
#define COUT          fifo_tx_1
void set_message_str(const char * str){ printf(str); while(*str){ COUT(*(str ++));}}
```

```
void set_message_str_w(const char * str, int w)
{
	const char * p = str;
	while(*p){ p ++; w --;}
	while((w --) > 0){ printf(" "); COUT(' ');}
	printf(str);
	while(*str){ COUT(*(str ++));}
}
```

```
void set_message_int(int val)
{
	char str[12], * p = str + 12;
	int isMinus;
	if((isMinus = (val < 0))){ val = -val;}
	*(-- p) = 0;///		termination
	while(val){
		*(-- p) = '0' + (val % 10);
		val /= 10;
	}
	if(isMinus)		{ *(-- p) = '-';}
	else if(*p == 0){ *(-- p) = '0';}
	set_message_str(p);
}
```

```
void set_message_hex(int val)
{
	unsigned int uval = (unsigned int) val;
	char str[12], * p = str + 12;
	*(-- p) = 0;///		termination
	while(uval){
		int h = uval & 0xf;
		*(-- p) = ((h <= 9) ? '0' : 'a' - 10) + h;
		uval >>= 4;
	}
	if(*p == 0){ *(-- p) = '0';}
	set_message_str(p);
}
```

```
void set_message_int_w(int val, int w)
{
	char str[12], * p = str + 12;
	int isMinus;
	if((isMinus = (val < 0))){ val = -val;}
	*(-- p) = 0;///		termination
	while(val){
		*(-- p) = '0' + (val % 10);
		val /= 10;
		w --;
	}
	if(isMinus)		{ *(-- p) = '-'; w --;}
	else if(*p == 0){ *(-- p) = '0'; w --;}
	while(w > 0)		{ *(-- p) = ' '; w --;}
	set_message_str(p);
}
```

```
void set_message_hex_w(int val, int w)
{
	unsigned int uval = (unsigned int) val;
	char str[12], * p = str + 12;
	*(-- p) = 0;///		termination
	while(uval){
		int h = uval & 0xf;
		*(-- p) = ((h <= 9) ? '0' : 'a' - 10) + h;
		uval >>= 4;
		w --;
	}
```

```
        if(*p == 0) { *(-- p) = '0'; w --;}
        while(w > 0){ *(-- p) = ' '; w --;}
        set_message_str(p);
}
```

- void set_message_str(const char * str){ … } : 文字列 str を FIFO 出力ポート（チャンネル ID = 1）に 1 文字ずつ出力（printf(“%s”, str);と等価）
  - printf() : TCT プロセッサ実行ファイルでは NOP 命令に置き換わる
- void set_message_str_w(const char * str, int w){ … } : printf(“%*s”, w, str);と等価（w : 文字数）
- void set_message_int(int val){ … } : printf(“%d”, val);と等価
- void set_message_hex(int val){ … } : printf(“%x”, val);と等価
- void set_message_int_w(int val, int w){ … } : printf(“%*d”, w, val);と等価（w : 桁数）
- void set_message_hex_w(int val, int w){ … } : printf(“%*x”, w, val);と等価（w : 桁数）

これらの関数によって printf 関数を以下のように移植することができる。

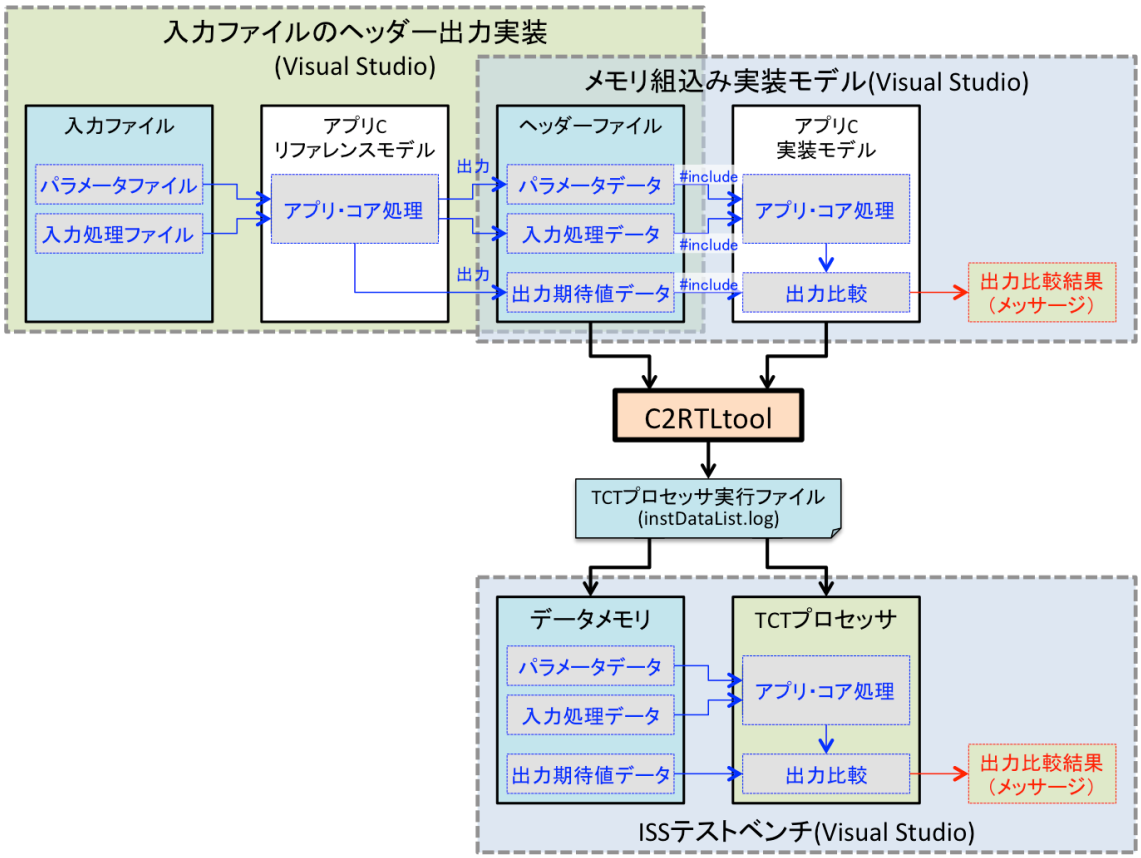
```
printf(“w = %4d¥n”, w);
            ↓
set_message_str(“w = “);
set_message_int_w(w, 4);
set_message_str(“¥n”);
```

### 3.3.2 ファイルIO処理のメモリ組込み変換実装

ファイル IO 処理を移植する第 1 の方法として、ファイル内部のデータをすべてヘッダーファイルに書き出し、これをインクルードファイルとしてアプリ C 実装モデルに組み込む方法である。ヘッダーファイルは、以下のように配列宣言に初期値を定義したものを生成する。

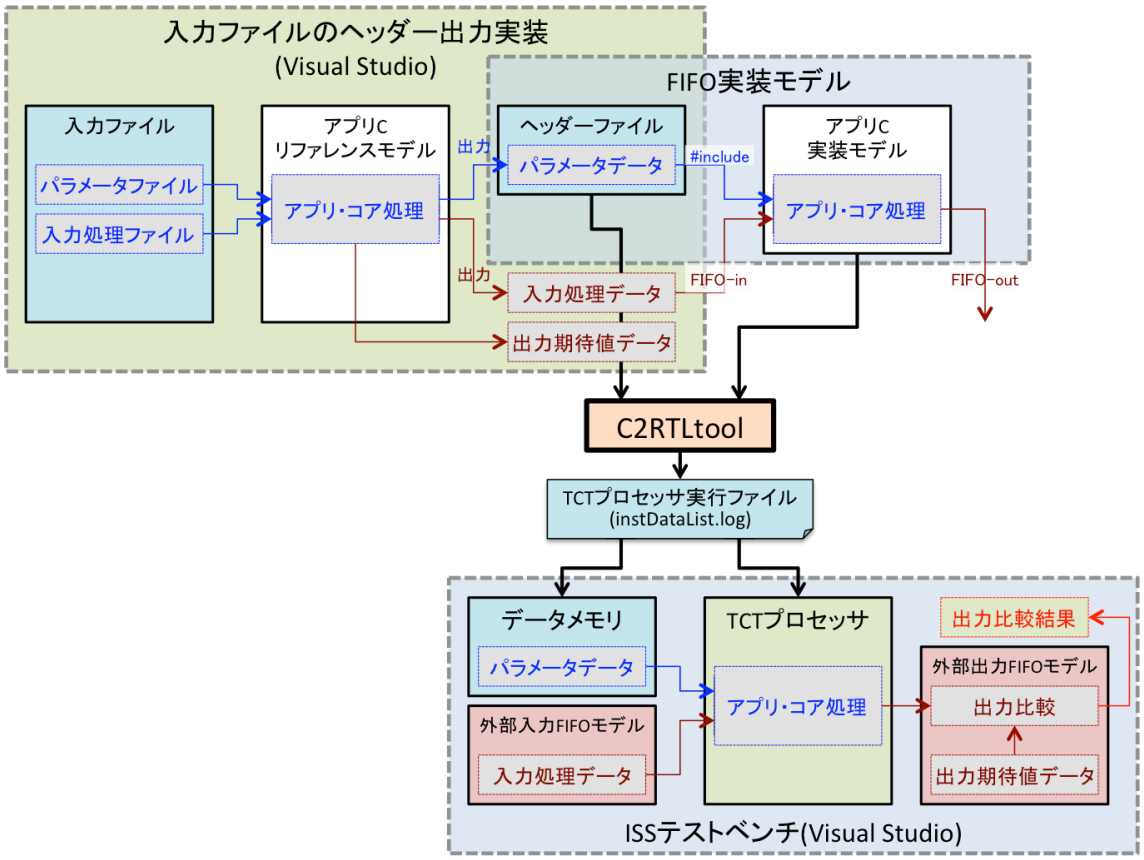
```
#define DATA_SIZE    1000
char data[DATA_SIZE] = {
51, 56, 80, 81, ... , 101
};
```

ファイルのデータ領域をすべてプロセッサのデータメモリ領域に取り込むことと等価である。この実装方式の場合、アプリ C リファレンスモデルとアプリ C 実装モデルの動作一致性検証は、プロセッサのデータメモリに格納された処理結果とアプリ C リファレンスモデルが生成する出力期待値データを格納したヘッダーファイルのデータとの比較処理をプロセッサ内部で行うことで実現する。




### 3.3.3 ファイルIO処理の外部FIFOモデル変換実装

ファイル IO 処理を移植する第 2 の方法として、ファイル内部のデータをすべて外部 FIFO モデルで実装し、プロセッサ FIFO 入出力ポートを介してデータを通信する方法である。例として、画像処理系アプリケーションを FIFO 入出力によるストリーミング処理として実装することを想定した場合、ファイル IO 処理を外部 FIFO モデルで実装することで、TCT-ASIP の実動作環境に近い実装が可能である。この実装方式の場合、アプリ C リファレンスモデルとアプリ実装モデルの動作一致性検証は、プロセッサの出力 FIFO ポートのデータとアプリ C リファレンスモデルが生成する出力期待値データを格納したヘッダーファイルのデータとの比較を外部出力 FIFO モデルで行うことで実現する。




## 3.4　TCTプロセッサCモデルのASIP拡張命令用リソース定義

本節では、TCT プロセッサ C モデル上での ASIP 拡張命令を開発する準備として、ASIP 専用リソースの定義方法について説明する。ASIP 専用リソース定義は、2.3.11 で説明した IO デバイスリソース定義記述に類似した形式となる。

- ASIP 命令クラスコード定義、ASIP 命令 OP コード定義
- ASIP 専用リソース（拡張レジスタ、メモリ）の識別 ID 定義
- ASIP 専用リソース構造体の定義（ユーザー定義構造体を階層的に定義する）

ASIP 拡張命令用リソース定義の例として、JPEG エンコーダ ASIP の記述を Appendix の 5.2.1「ASIP 拡張命令用リソース定義（TCTProc_jpg.h）」で説明している。

## 3.5　TCTプロセッサCモデルのASIP拡張命令実装

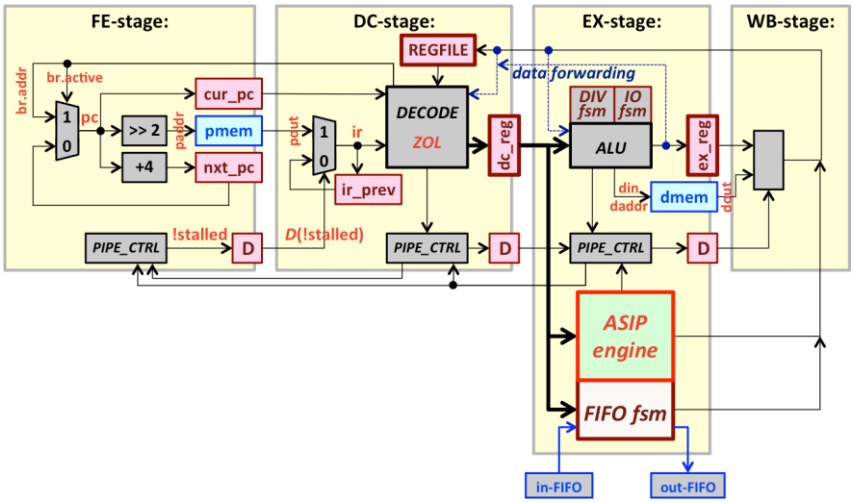
本節は、前節で説明した ASIP 拡張命令用リソース定義をアプリ C 実装モデルから制御可能にするための TCT プロセッサ C モデルの ASIP 拡張命令実装について説明する。なお、アプリ C 実装モデルにおける ASIP 拡張命令関数（TCT-intrinsic 関数）定義とアプリ C 実

装モデル上の ASIP 拡張命令への置き換えは、3.5 を参照のこと。また、具体的な記述例は Appendix「B. TCTProcJPG_FIFO の C モデル」を参照のこと。
TCT プロセッサ C モデルにおける ASIP 拡張命令実装は、下記の項目で行う。

- ASIP 拡張命令デコード記述：2.3.12 で説明した IO 命令デコード記述と同じコード構造で実装できる。具体的には、ASIP 固有命令クラスコードと対応させて、ASIP 固有命令デコード関数を準備し、その関数では、各命令フィールドを取り出し、対応する ASIP 固有命令 OP コード割当、レジスタ ID デコード等を実装する。
- ASIP 拡張命令実行動作記述：2.3.13 で説明した IO 制御 FSM 動作記述と同じコード構造で実装できる。具体的には、ASIP 拡張命令を実行する ASIP 実行制御 FSM 関数を準備し、その関数では、デコードされた ASIP 拡張命令の実行動作の他、ASIP エンジン内部の各 FSM 制御動作を記述する。

## 3.6 ASIP拡張命令実行動作記述のパイプライン設計手法

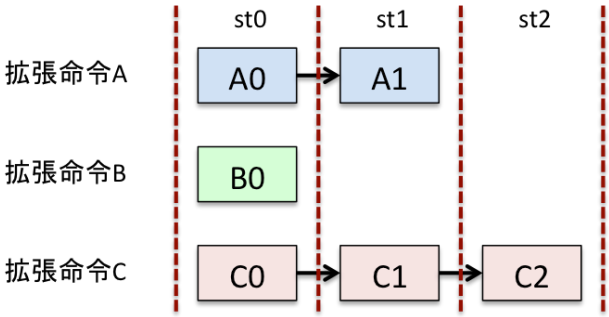
ASIP 拡張命令実行は、4 段パイプライン構造の TCT プロセッサにおける execute ステージで動作するような記述が必要になる。下図の EX-stage に埋込まれている「ASIP engine」が ASIP 拡張命令実行部であり、ASIP 処理性能を大幅に向上させるためには、この ASIP 拡張命令実行部でパイプライン構造・並列処理構造を適切に構築する必要がある。



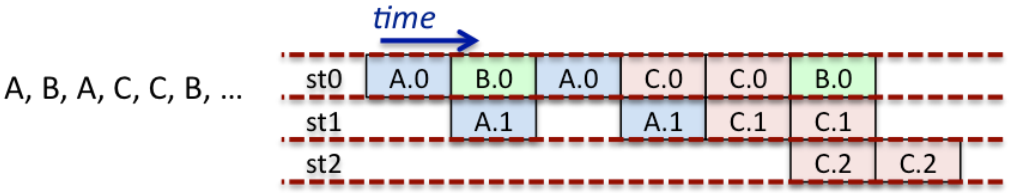

### 3.6.1 ASIP拡張命令のパイプライン動作記述

一つの ASIP 拡張命令を複数段のパイプライン構造として実装する必要があるケースとして以下が挙げられる。

- ASIP 内部の専用 SRAM のデータに対する処理を伴う場合：データ読出しに 1 サイクルかかるため、その読出しデータの処理は 2 サイクル目以降になる。
- ASIP 拡張命令の実行動作が大きな論理遅延を伴う場合：処理スループットを向上させるために、実行動作を数ステージに分割する。




ここで、上図のように 3 種類の拡張命令 A, B, C の実装を考える。A は 2 サイクル、B は 1 サイクル、C は 3 サイクルかかるとする。ただし、これら 3 命令は、いずれも異なった専用レジスタへの書込みをすると仮定する（汎用レジスタの書込みなし、同一の専用レジスタへの書込みもなし）。これら 3 命令を任意の順番で連続的に発行したときに、1 サイクルスループットを実現するためのパイプライン動作は、下図のようになる。ここで、横軸が時間で縦軸がパイプラインステージである。




これら 3 種類の拡張命令の 3 段パイプライン構造の C データフロー記述は、それぞれのパイプラインステージの動作を関数（F0_stage, F1_stage, F2_stage）でマージして下図のような記述で実現できる。

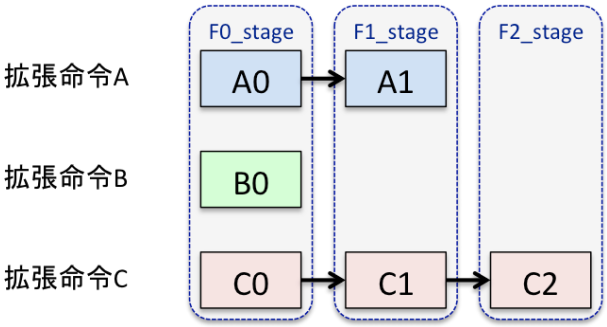
```
void F0_stage(ASIP * asip, UINT7 dc_reg_op)
{
   switch(dc_reg_op){
   case opA: A0(asip);      asip->op[0] = opA;       break;
   case opB: B0(asip);      asip->op[0] = opNULL;    break;
   case opC: C0(asip);      asip->op[0] = opC;       break;
   default:                 asip->op[0] = opNULL;    break;
   }
}
```

```
void F1_stage(ASIP * asip)
{
   switch(asip->op[0]){
   case opA: A1(asip);      asip->op[1] = opNULL;    break;
   case opC: C1(asip);      asip->op[1] = opC;       break;
   default:                 asip->op[1] = opNULL;    break;
   }
}
```

```
void Fasip_fsm(ASIP * asip, UINT7 dc_reg_op)
{
    F2_stage(asip);                  // asip->op[1] →
    F1_stage(asip);                  // asip->op[0] → asip->op[1]
    F0_stage(asip, dc_reg_op);       // dc_reg_op → asip->op[0]
}
```

```
void F2_stage(ASIP * asip)
{
   switch(asip->op[1]){
   case opC: C2(asip);      break;
   default:                 break;
   }
}
```

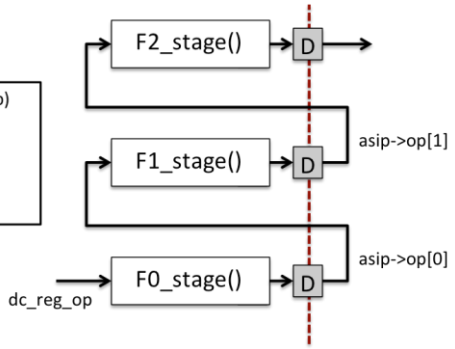
ここで、dc_reg_op は decode ステージ（execute ステージの前段）のレジスタ出力であり、asip->op[0], asip->op[1]は ASIP エンジン内部でパイプライン実行制御用に使われるレジスタであるとする。各拡張命令の動作に着目してこれらの関数記述を説明する（下図）。




- 拡張命令 A（2 ステージ動作）：
  - 第 1 ステージ（F0_stage）：dc_reg_op == opA の場合、A0(asip)を実行し、第 2 ステージに継続させるために asip->op[0]を opA に設定する。
  - 第 2 ステージ（F1_stage）：asip->op[0] == opA の場合、A1(asip)を実行し、第 3 ステージには継続しないため asip->op[1]を opNULL に設定する。
- 拡張命令 B（1 ステージ動作）：
  - 第 1 ステージ（F0_stage）：dc_reg_op == opB の場合、B0(asip)を実行し、第 2 ステージには継続しないため asip->op[0]を opNULL に設定する。
- 拡張命令 C（3 ステージ動作）：
  - 第 1 ステージ（F0_stage）：dc_reg_op == opC の場合、C0(asip)を実行し、第 2 ステージに継続させるために asip->op[0]を opC に設定する。
  - 第 2 ステージ（F1_stage）：asip->op[0] == opC の場合、C1(asip)を実行し、第 3 ステージに継続させるために asip->op[1]を opC に設定する。
  - 第 3 ステージ（F2_stage）：asip->op[1] == opC の場合、C2(asip)を実行する。

一方で、これら 3 つのステージの動作記述は Fasip_fsm 関数でパイプライン進行方向とは逆順に（F2_stage → F1_stage → F0_stage の順に）発行されることに注目する。これは、パイプラインレジスタ asip->op[0], asip->op[1]の参照が「逆方向参照」（1.5.6「C 記述上の変数参照方向と RTL 記述上の参照タイミング」）になっているためである。

```
void Fasip_fsm(ASIP * asip, UINT7 dc_reg_op)
{
    F2_stage(asip);
    F1_stage(asip);
    F0_stage(asip, dc_reg_op);
}
```




1.6.2「フロー制御付きパイプライン記述方式」で説明したように、逆方向参照を適切に使用することにより、プロセッサのパイプライン数を変更することなく、ロジック構造として複数パイプライン段での高スループット演算回路を構成することが可能である。

# Appendix

# A. TCTProcFIFOのCモデル

本節では、/Applications/TCT_ProcModel/Src/CORE/FifoVersion/の TCT プロセッサ C モデルについて説明する。＜本章の説明は、旧 C2RTL システムに基づいているため、今後 LLVM-C2RTL 環境用に情報を更新予定である。＞

## A.1　ISSテストベンチ関数（TCTProc.c）

下記に ISS テストベンチ関数を示す。

```
#define ISS_PARAMS          D_FIFO_PORT * in_fifo, D_FIFO_PORT * out_fifo
#define ISS_ARGS            in_fifo, out_fifo
#define ISS_TOP_ARGS        &in_fifo, &out_fifo

typedef int (*TCTProcFunc)(ISS_PARAMS);
```

```
int TCTProcModel(TCTProcFunc tct_proc_func, ISS_PARAMS) /// ISS ラッパー関数
{
   int val;
   fifo_ext_input(in_fifo, out_fifo);
   val = tct_proc_func(ISS_ARGS);                    /// ISS コア関数呼出し
   fifo_ext_output(in_fifo, out_fifo);
   return val;
}
```

```
int main(int argc, char * argv[])
{
   FIFO_PORT in_fifo = {0}, out_fifo = {0};          /// テストベンチ FIFO モデル定義
   Timer tm0 = {"TCTProc"}, tm1 = {"TCTProcPixPipe"}; /// タイマー定義
   OpenPixPipeTestbench();                           /// テストベンチ出力ファイル初期化
   install_program(cpu.dmem, cpu.pmem);              /// ISS メモリ初期化

   fifo_ext_init(&in_fifo, &out_fifo, &cpu.cycle);   /// テストベンチ FIFO モデル初期化
   get_timer(&tm0);                                  /// タイマー初期化
                                                     /// ISS（命令セットシミュレータ）ループ
   while(!TCTProcModel(TCTProcFIFO, ISS_TOP_ARGS)){ tm0.cycles ++;}
   get_timer(&tm0);                                  /// タイマー計測
   fifo_ext_print_result();                          /// テストベンチ FIFO モデル検証結果出力
   printf("hit return : ");
   getchar();

   fifo_ext_switch_mode(&in_fifo, &out_fifo);        /// テストベンチ FIFO モデルモード変更
#if !defined __TCT_COMPILER_USED__                   /// TCT コンパイラでは無視されるコード領域
   /// RTL 等価 C モデル（C2RTL で自動生成）用テストベンチ
   install_program(_v_cpu_dmem, _v_cpu_pmem);
   get_timer(&tm1);
   while(!TCTProcModel(PixPipe_TCTProcFIFO, ISS_TOP_ARGS)){ tm1.cycles ++;}
   get_timer(&tm1);
   fifo_ext_print_result();
   printf("hit return : ");
   getchar();
#endif
   ClosePixPipeTestbench();                  /// テストベンチ出力ファイル終了処理
   return 0;
```

}

- int TCTProcModel(TCTProcFunc tct_proc_func, ISS_PARAMS)：ISS コア関数へのラッパーであり、tct_proc_func が ISS コア関数ポインタである。ISS コア関数としては、RTL 合成対象トップ関数の他、C2RTL フレームワークが生成する RTL 等価 C モデルのトップ関数も使用可能である。
- int main(int argc, char * argv[])：ISS テストベンチメイン関数。
- FIFO_PORT in_fifo = {0}, out_fifo = {0};：FIFO 入出力付き ISS モデルへのデータ供給と出力検証を行うためのテストベンチ FIFO モデルをローカル変数として定義。
- Timer tm0 = {"TCTProc"}, tm1 = {"TCTProcPixPipe"};：ISS 実行時間計測タイマー。
- OpenPixPipeTestbench();：テストベンチ出力ファイル初期化。
- install_program(cpu.dmem, cpu.pmem);：ISS メモリ初期化（TCTProcUtils.c）。
- fifo_ext_init(&in_fifo, &out_fifo, &cpu.cycle);：テストベンチ FIFO モデル初期化。
- get_timer(&tm0);：タイマー初期化（初回呼出し）及びタイマー計測。
- while(!TCTProcModel(TCTProcFIFO, ISS_TOP_ARGS)){ tm0.cycles ++;}：ISS ループ処理（プロセッサが HALT するまで継続する）。
- fifo_ext_print_result();：テストベンチ FIFO モデル検証結果出力（ISS の FIFO 出力系列が期待値と一致するかを検証）。
- fifo_ext_switch_mode(&in_fifo, &out_fifo);：テストベンチ FIFO モデルの再初期化（この後の RTL 等価 C モデル検証のため）
- #if !defined __TCT_COMPILER_USED__：C2RTL フレームワークの C コンパイラ（TCT COMPILER）では、__TCT_COMPILER_USED__マクロ定義をコンパイラ内部で自動挿入する。この仕組みを使うことで、__TCT_COMPILER_USED__マクロが定義の有無によって、C2RTL フレームワーク C コンパイラ用（RTL 合成）とその他標準 C コンパイラ（ISS テストベンチ）の場合分けが C 記述上で可能となる。ここでは、C2RTL フレームワークによって自動生成される RTL 等価 C モデルの ISS テストベンチ記述部分を TCT COMPILER では無視するようにしている。
- RTL 等価 C モデル検証：ISS コア関数（RTL 合成対象トップ関数）の RTL 等価 C モデルの検証は、ISS コア関数の検証と同一のユーザー定義 API（install_program(), TCTProcModel()）で実行できる。
  - install_program(_v_cpu_dmem, _v_cpu_pmem);：RTL 等価 C モデル内部のメモリ配列を引数として引き渡す。
  - PixPipe_TCTProcFIFO：RTL 等価 C モデルのトップ関数名。RTL 合成対象トップ関数名が「XXX」の場合、その RTL 等価 C モデルのトップ関数名は「PixPipe_XXX」となる。

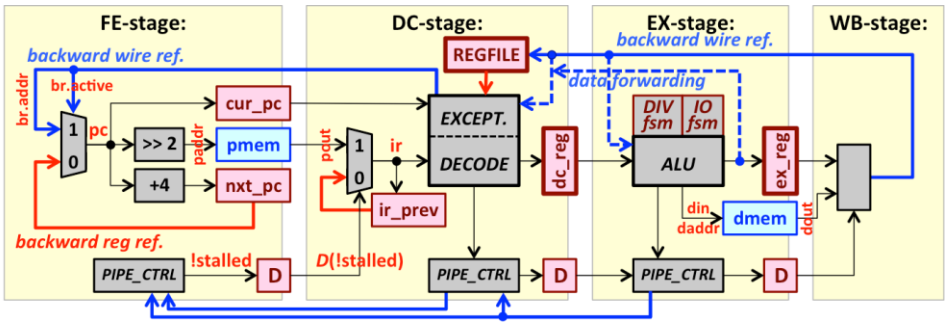
## A.2　ISSコア関数（TCTProc.c）

下記に ISS コア関数を示す。

```
// TCTProc.h:
#define UPDATE_PIPE_CTRL(pctl)                    pctl.stalled = pctl.stall_this
```

```
// TCTProc.c:
int TCTProcFIFO(ISS_PARAMS)
{
        set_fifo_input_ports(&cpu.io.fifo, in_fifo, out_fifo);
        fetch();
        decode();
        execute();
        writeback();
        UPDATE_PIPE_CTRL(cpu.fe_stt.pctl);
        UPDATE_PIPE_CTRL(cpu.dc_stt.pctl);
        UPDATE_PIPE_CTRL(cpu.ex_stt.pctl);
        UPDATE_PIPE_CTRL(cpu.wb_stt.pctl);
        set_fifo_output_ports(&cpu.io.fifo, in_fifo, out_fifo);
        cpu.cycle ++;
        return (cpu.halted == 1);
}
```

- set_fifo_input_ports(), set_fifo_output_ports()：テストベンチ FIFO モデルとプロセッサ内部 FIFO デバイスモデル（後述）の間の信号伝搬。
- fetch(), decode(), execute(), writeback()：プロセッサの 4 つのパイプラインステージ（FE, DC, EX, WB：下図）の動作記述。
- UPDATE_PIPE_CTRL：プロセッサのパイプラインフロー制御レジスタ更新。
- return (cpu.halted == 1);：プロセッサの HALT 状態を ISS の戻り値とする。




フロー制御付きパイプライン構造
（4段パイプラインRISCプロセッサ）

## A.3　プロセッサリソース定義記述（TCTProcArch.h）

プロセッサリソースとは、ISS コア関数で必要となるデータ構造を指す。以下に主要なプロセッサリソース記述を示す。

```
typedef struct ST_Insn
{
	UINT5	code, rf0, rf1, rf2, op_sc;
	UINT6	op;
	UINT3	dtype;
	UINT2	arf, op_io;
	SINT27	imm27;
	SINT17	imm17;
	SINT12	imm12;
	UINT15	uimm15;
	SINT32	imm;
	BIT	src_f[2], dst_f;
	UINT6	rid[3];
	UINT2	mmu_op;
} Insn;
```

```
typedef struct ST_PipeCtrl
{
	BIT	stall_prev;
	BIT	stall_this;
	ST_BIT	stalled;		// <- latch(stall_this)
	BIT	stalled_flag;		// <- wire(stalled | later_stages.stall_prev)
} PipeCtrl;
```

```
typedef struct ST_BranchStatus
{
	BIT	active;
	UINT32	addr;
} BranchStatus;
```

```
typedef struct ST_DstStatus
{
	BIT	valid, ldm_f;
	UINT6	id;
	UINT32	out;
} DstStatus;
```

- typedef struct ST_Insn { … } Insn;：命令デコード情報（DC ステージで生成・参照）
- typedef struct ST_PipeCtrl { … } PipeCtrl;：プロセッサパイプラインフロー制御情報（FE/DC/EX/WB 各ステージで生成、相互参照）
- typedef struct ST_BranchStatus { … } BranchStatus;：命令分岐制御用（DC ステージで生成、FE ステージで参照）
- typedef struct ST_DstStatus { … } DstStatus;:演算結果情報(EX/WB ステージで生成、DC/EX ステージで参照）

```
typedef struct ST_FEState
{
	PipeCtrl		pctl;
} FEState;
```

```
typedef struct ST_DCState
{
	PipeCtrl		pctl;
	BranchStatus		br;
	UINT32			src[2];
} DCState;
```

```
typedef struct ST_PendingOp
```

```
{
        BIT             is_pending;
        BIT             is_interruptible;
        BIT             resume_from_next_insn;
} PendingOp;
typedef struct ST_EXState
{
        PipeCtrl        pctl;
        PendingOp       pending_op;
        DstStatus       dst;
        UINT32          out2;
        UINT32          st_data, daddr;
        UINT2           wbflag;
} EXState;
typedef struct ST_WBState
{
        PipeCtrl        pctl;
        DstStatus       dst;
        UINT32          dout;
} WBState;
```

- typedef struct ST_FEState { … } FEState;：FE ステージ制御信号
- typedef struct ST_DCState { … } DCState;：DC ステージ制御信号
- typedef struct ST_EXState { … } EXState;：EX ステージ制御信号
- typedef struct ST_WBState { … } WBState;：WB ステージ制御信号

```
#pragma _TCT_verilog_state      FEReg DCReg EXReg DIVReg
typedef struct ST_FEReg
{
        UINT32  cur_pc, nxt_pc;
} FEReg;
typedef struct ST_DCReg
{
        UINT32  src[3], cur_pc;
        UINT3   dtype;
        UINT6   dst_id, src_id[2];
        BIT     src_f[2], dst_f, stm_f, ldm_f;
        UINT7   op;
        UINT2   mmu_op;
} DCReg;
typedef struct ST_EXReg
{
        UINT32  out;
        UINT2   wbflag;
} EXReg;
typedef struct ST_DIVReg
{
        BIT     pending, negate_out, modulo_flag, terminate_condition;
        UINT32  f, g, q, r;
} DIVReg;
```

- #pragma _TCT_verilog_state FEReg DCReg EXReg DIVReg : #pragma によるレジスタ属性記述。ここで、FEReg, DCReg, EXReg, DIVReg はそれぞれ構造体の typedef であり、構造体へのレジスタ属性記述は、それらの最下層メンバー変数全てに対して

レジスタ属性が付与される。

- typedef struct ST_FEReg { … } FEReg;：FE ステージパイプラインレジスタ
- typedef struct ST_DCReg { … } DCReg;：DC ステージパイプラインレジスタ
- typedef struct ST_EXReg { … } EXReg;：EX ステージパイプラインレジスタ
- typedef struct ST_DIVReg { … } DIVReg;：EX ステージ除算器 FSM 状態レジスタ

```
typedef struct ST_CPU
{
        ST_UINT32       cycle;
        RF_UINT32       gpr[GPR_COUNT];
        UINT32          ir;
        ST_UINT32       ir_prev;
        ST_BIT          halted;
        M_UINT32        pmem[PMEM_SIZE];
        M_UINT32        dmem[DMEM_SIZE];
        FIFO_PORT       dummy_fifo_port;
        Insn            insn;
        FEState         fe_stt;
        DCState         dc_stt;
        EXState         ex_stt;
        WBState         wb_stt;
        FEReg           fe_reg;
        DCReg           dc_reg;
        EXReg           ex_reg;
        DIVReg          div;
        IO              io;
} CPU;
```

- typedef struct ST_CPU { … } CPU;：CPU リソース全体のデータ構造。
- RF_UINT32 gpr[GPR_COUNT];：汎用レジスタファイル（レジスタファイル属性）。
- M_UINT32 pmem[PMEM_SIZE];：プログラムメモリ（メモリ属性）。
- M_UINT32 dmem[DMEM_SIZE];：データメモリ（メモリ属性）。
- FIFO_PORT dummy_fifo_port;：ダミーFIFO ポート（後述：ASIP 命令拡張実装において、拡張命令実行部が直接 FIFO ポートと結合する仕組みを構築する。ASIP 命令拡張が未実装の時にダミーFIFO ポートを準備しておく）。
- IO io;：IO デバイスモデル（後述）

## A.4 FEステージ動作記述（TCTProc.c）

下記に FE ステージ動作記述を示す。

```
// TCTProc.h:
#define INIT_PIPE_CTRL(pctl, stall)         pctl.stall_prev = 0; pctl.stall_this = 0; pctl.stalled_flag = stall
#define FE_STALL    ((cpu.dc_stt.pctl.stall_prev | cpu.ex_stt.pctl.stall_prev | cpu.wb_stt.pctl.stall_prev))
```

```
// TCTProc.c:
void fetch()
{
        INIT_PIPE_CTRL(cpu.fe_stt.pctl, FE_STALL);
```

```
        if(!cpu.fe_stt.pctl.stalled_flag){
                UINT32 pc = (cpu.dc_stt.br.active) ? cpu.dc_stt.br.addr : cpu.fe_reg.nxt_pc;
                cpu.fe_reg.cur_pc = pc;
                cpu.fe_reg.nxt_pc = (FC_UINT32)(pc + 4);
                cpu.ir = cpu.pmem[pc >> 2];
                cpu.ir_prev = cpu.ir;
        }
        else{ cpu.ir = cpu.ir_prev;}
}
```

- #define FE_STALL (cpu.dc_stt.pctl.stall_prev | cpu.ex_stt.pctl.stall_prev | cpu.wb_stt.pctl.stall_prev)：FE ステージのストール信号は、DC/EX/WB ステージからの逆方向参照の OR 結合である。
- INIT_PIPE_CTRL(cpu.fe_stt.pctl, FE_STALL);：パイプラインフロー制御信号初期化（DC/EX/WB の各後段ステージのストール信号によって設定される）。
- if(!cpu.fe_stt.pctl.stalled_flag){ … }：FE ステージのストール状態判定（ストールでない場合は括弧内を処理）。
- pc = (cpu.dc_stt.br.active) ? cpu.dc_stt.br.addr : cpu.fe_reg.nxt_pc;：現 PC の計算（DC ステージで分岐が発生した場合は分岐アドレスを設定し、それ以外では nxt_pc を設定する）。
- cpu.fe_reg.cur_pc = pc; cpu.fe_reg.nxt_pc = (FC_UINT32)(pc + 4);：現命令 PC（cur_pc）と次命令 PC（nxt_pc）の更新。
- cpu.ir = cpu.pmem[pc >> 2]; cpu.ir_prev = cpu.ir;：現命令フェッチ（ir_prev：命令一時レジスタ）
- else{ cpu.ir = cpu.ir_prev;}：FE ステージのストール時は、ir_prev 命令一時レジスタを現命令信号（ir）に代入。

## A.5 命令フォーマット定義記述（TCTProcArch.h）

命令デコードのための命令フォーマット定義記述を以下に示す。

```
enum FieldEnum
{
        F_code,
        F_rf0,
        F_dtype,
        F_arf,
        F_op,
        F_op_sc,
        F_rf1a,
        F_rf1,
        F_rf2,
        F_imm27,
        F_imm17,
        F_imm12,
        F_uimm15,
```

```
        F_op_io,
        F_count,
};
```

```
typedef struct ST_Field
{
        SINT32 msbPos, bitLength, codeOffset, isSigned;
} Field;
```

```
static const Field FLD[F_count] =
{
        {31, 5, 0, 0},          ///     F_code
        {26, 5, 0, 0},          ///     F_rf0
        {21, 3, 0, 0},          ///     F_dtype
        {18, 2, 28, 0},         ///     F_arf
        {21, 5, 0, 0},          ///     F_op
        {21, 5, 0, 0},          ///     F_op_sc
        {21, 5, 0, 0},          ///     F_rf1a
        {16, 5, 0, 0},          ///     F_rf1
        {11, 5, 0, 0},          ///     F_rf2
        {26, 27, 0, 1},         ///     F_imm27
        {16, 17, 0, 1},         ///     F_imm17
        {11, 12, 0, 1},         ///     F_imm12
        {14, 15, 0, 0},         ///     F_uimm15
        {16, 2, 0, 0},          ///     F_op_io
};
```

- enum FieldEnum { … }：命令フィールド enum 定義
- typedef struct ST_Field { SINT32 msbPos, bitLength, codeOffset, isSigned;}：命令フィールド構造体（MSB 位置、ビット長、オフセット値、符号属性）
- static const Field FLD[F_count] = { … };：命令フィールド構造体配列初期化

```
enum InsnEnum
{
        I_stm           = 0x12,
        I_stmr          = 0x14,
        I_ldm           = 0x1a,
        I_ldmr          = 0x1c,
        I_ldmr2         = 0x1d,
        I_comp          = 0x0c,
        I_compi         = 0x0d,
        I_compi2        = 0x0e,
        I_addi          = 0x0a,
        I_jmp           = 0x04,
        I_call          = 0x05,
        I_bz            = 0x06,
        I_bnz           = 0x07,
        I_jcr           = 0x17,
        I_nop           = 0x00,
        I_sc            = 0x02,
        I_spr           = 0x03,
        I_comm          = 0x01,
        I_ctrl          = 0x11,
        ///         unused: 0x09, 0x10, 0x13, 0x15, 0x16, 0x18, 0x19, 0x1b, 0x1e, 0x1f
};
```

```
enum DTypeEnum
{
        DT_UCHAR        = 0x1,      /// 8-bit unsigned
        DT_SCHAR        = 0x5,      /// 8-bit signed
        DT_USHORT       = 0x2,      /// 16-bit unsigned
```

```
        DT_SSHORT       = 0x6,      /// 16-bit signed
        DT_INT          = 0x0,      /// 32-bit
        DT_RTN          = 0x7,      /// RETURN: PC ← LKR, LKR ← dmem[SP – 4];
};
```

```
enum OpEnum
{
        O_nop     = 0x00,   /// NOP
        O_add     = 0x01,   /// dst ← src0 + src1;
        O_sub     = 0x02,   /// dst ← src0 – src1;
        O_shl     = 0x03,   /// dst ← src0 << src1;
        O_mova    = 0x05,   /// dst ← src0 + (src1 << 16);
        O_neq     = 0x06,   /// dst ← src0 != src1;
        O_equ     = 0x07,   /// dst ← src0 == src1;
        O_and     = 0x08,   /// dst ← src0 & src1;
        O_orr     = 0x09,   /// dst ← src0 | src1;
        O_xor     = 0x0a,   /// dst ← src0 ^ src1;
        O_inv     = 0x0b,   /// dst ← ~src0;
        O_not     = 0x0c,   /// dst ← !src0;
        O_mov     = 0x0e,   /// dst ← src0;
        O_movh    = 0x0f,   /// dst ← (src0 << 16);
        O_mul     = 0x0,    /// dst ← src0 * src1;
        O_div     = 0x1,    /// dst ← src0 / src1;
        O_mod     = 0x2,    /// dst ← src0 % src1;
        O_shr     = 0x3,    /// dst ← src0 >> src1;
        O_cgt     = 0x4,    /// dst ← src0 > src1;
        O_cge     = 0x5,    /// dst ← src0 >= src1;
        O_clt     = 0x6,    /// dst ← src0 < src1;
        O_cle     = 0x7,    /// dst ← src0 <= src1;
        O_ldm     = 0x20,   /// dst ← mem[addr + offset];
        O_stm     = 0x21,   /// mem[addr + offset] ← src0;
        O_call    = 0x22,   /// mem[SP – 4] ← LKR, LKR <- PC + 4;
        O_wait    = 0x24,   /// WAIT for interrupt
};
```

- enum InsnEnum { … };：命令クラスコード（5 ビット）
- enum DTypeEnum { … };：ロード・ストアデータ型コード（3 ビット）
  - 8 ビット：DT_UCHAR (unsigned), DT_SCHAR (signed)
  - 16 ビット：DT_USHORT (unsigned), DT_SSHORT (signed)
  - 32 ビット：DT_INT
  - リターン命令専用：RT_RTN（PC ← LKR, LKR ← dmem[SP – 4];）
- enum OpEnum { … };：命令演算コード（6 ビット）
  - 符号無し命令：0x00 – 0x0f
  - 符号付き命令：0x10 – 0x1f
    - 0_mul, O_div, …, O_cle：下位 3 ビットコードに対応
    - 4 ビット目：1（符号付き演算）、0（符号無し演算）
    - 5 ビット目：常に 1

    （補足：32 ビット乗算は符号付き・符号無しでも結果は同一であり、符号無し処理の分類でも構わないが、将来の拡張性を考慮して符号付き演算に分類している）
  - メモリアクセス命令：0x20 – 0x22（LDM, STM, CALL）

```
// TCTProc.h:
#define GET_BITS(data, msbPos, bitLength)                    ¥
                    (((data) >> ((msbPos) - (bitLength) + 1)) & ((1 << (bitLength)) - 1))
#define GET_FIELD(fid)                                       ¥
                    (GET_BITS(cpu.ir, FLD[fid].msbPos, FLD[fid].bitLength) + FLD[fid].codeOffset)
#define DEC_FLD(fid)                cpu.insn.fid = GET_FIELD(F_##fid)
#define DEC_FLD_ADD(fid)            cpu.insn.fid = GET_FIELD(F_##fid##a)
#define SIGN_EXT(data, len0, len1)  (((FC_SINT32)((data) << ((len1) - (len0)))) >> ((len1) - (len0)))
#define GET_IMM(fid)                (SIGN_EXT(cpu.insn.fid, FLD[F_##fid].bitLength, 32))
#define SET_IMM(fid)                DEC_FLD(fid); cpu.insn.imm = GET_IMM(fid)
```

- DEC_FLD(fid) : 命令フィールド(fid)のコードを cpu.ir から取り出し cpu.insn.fid に代入するマクロ。
- SET_IMM(fid) : 即値フィールド(fid)のコードを cpu.ir から取り出し、符号拡張して cpu.insn.imm に代入するマクロ。

## A.6　DC ステージ動作記述（TCTProc.c）

下記に DC ステージ動作記述を示す。

```
// TCTProc.h:
#define DC_STALL                    ((cpu.ex_stt.pctl.stall_prev | cpu.wb_stt.pctl.stall_prev))
#define SET_INSN_FLAG(fs0, fs1, fd) cpu.insn.src_f[0] = fs0; cpu.insn.src_f[1] = fs1; cpu.insn.dst_f = fd
#define BYPASS_DC(PIPE, ID)         ¥
        (cpu.##PIPE##_stt.dst.valid && cpu.##PIPE##_stt.dst.id == ID) ? cpu.##PIPE##_stt.dst.out
```

```
// TCTProc.c:
void dec_m17(){ DEC_FLD(rf0); DEC_FLD(dtype); DEC_FLD(arf); SET_IMM(imm17); SET_INSN(rid[0], rf0); SET_INSN(rid[1], arf);}
void dec_m1() { DEC_FLD(rf0); DEC_FLD(dtype); DEC_FLD(rf1); SET_IMM(imm12); SET_INSN(rid[0], rf0); SET_INSN(rid[1], rf1);}
void dec_m2() { DEC_FLD(rf0); DEC_FLD(dtype); DEC_FLD(rf1); DEC_FLD(rf2);   SET_INSN(rid[0], rf0); SET_INSN(rid[1], rf1); SET_INSN(rid[2], rf2);}
void dec_c3() { DEC_FLD(rf0); DEC_FLD(op);    DEC_FLD(rf1); DEC_FLD(rf2);   SET_INSN(rid[0], rf0); SET_INSN(rid[1], rf1); SET_INSN(rid[2], rf2);}
void dec_c2() { DEC_FLD(rf0); DEC_FLD(op);    DEC_FLD(rf1); SET_IMM(imm12); SET_INSN(rid[0], rf0); SET_INSN(rid[1], rf1);}
void dec_c1() { DEC_FLD(rf0); DEC_FLD(op);                  SET_IMM(imm17); SET_INSN(rid[0], rf0); SET_INSN(rid[1], rf0);}
void dec_add(){ DEC_FLD(rf0); DEC_FLD_ADD(rf1);             SET_IMM(imm17); SET_INSN(rid[0], rf0); SET_INSN(rid[1], rf1);}
void decode()
{
   BIT br_hazard = 0, halted = 0;
   cpu.insn.mmu_op  = MMUOp_null;
   cpu.insn.imm     = 0;
   INIT_PIPE_CTRL(cpu.dc_stt.pctl, DC_STALL);
   if(!cpu.dc_stt.pctl.stalled_flag){
            DEC_FLD(code);
            cpu.dc_stt.br.active  = 0;
            cpu.dc_stt.br.addr    = 0;
            switch(cpu.insn.code){
            case I_stm: dec_m17(); SET_INSN_FLAG(1, 1, 0); cpu.insn.op = O_stm; break;
            case I_stmr: dec_m1(); SET_INSN_FLAG(1, 1, 0); cpu.insn.op = O_stm; break;
            case I_ldm: dec_m17(); SET_INSN_FLAG(1, 0, 1); cpu.insn.op = O_ldm;
```

```
                if(cpu.insn.dtype == DT_RTN){
                        cpu.dc_stt.br.active  = 1;
                        cpu.dc_stt.br.addr = BYPASS_DC(wb, R_LKR) : cpu.gpr[R_LKR];
                }
                break;
        case I_ldmr: dec_m1();          SET_INSN_FLAG(1, 0, 1); cpu.insn.op = O_ldm; break;
        case I_ldmr2:dec_m2();          SET_INSN_FLAG(1, 1, 1); cpu.insn.op = O_ldm; break;
        case I_comp: dec_c3();          SET_INSN_FLAG(1, 1, 1);                      break;
        case I_compi:dec_c2();          SET_INSN_FLAG(1, 0, 1);                      break;
        case I_compi2:dec_c1();
                SET_INSN_FLAG(!(cpu.insn.op == O_mov || cpu.insn.op == O_movh), 0, 1); break;
        case I_addi: dec_add();         SET_INSN_FLAG(1, 0, 1); cpu.insn.op = O_add; break;
        case I_jmp:
        case I_call:
        case I_jcr:
        case I_bz:
        case I_bnz:         br_hazard = dec_jc();           break;
        case I_sc:          halted = dec_sc();              break;
        case I_spr:         dec_spr();                      break;
        case I_io:          dec_io();                       break;
        case I_nop:
        default:            dec_default();                  break;
        }
        {
          //////////// load gpr and imm
          UINT32 imm_val   = (FC_UINT32) cpu.insn.imm;
          BIT stm_f        = (cpu.insn.op == O_stm || cpu.insn.op == O_call);
          BIT ldm_f        = (cpu.insn.op == O_ldm);
          UINT6 src_id0    = cpu.insn.rid[1];
          UINT6 src_id1    = (stm_f) ? cpu.insn.rid[0] : cpu.insn.rid[2];
          UINT32 src_val0  = (src_id0 & 0x20) ? cpu.spr[src_id0 & 0x1f] : cpu.gpr[src_id0];
          UINT32 src_val1  = (src_id1 & 0x20) ? cpu.spr[src_id1 & 0x1f] : cpu.gpr[src_id1];
          if(halted || br_hazard){
            cpu.dc_stt.pctl.stall_prev  = 1;
            cpu.dc_stt.pctl.stall_this  = 1;
            cpu.dc_reg.op               = O_nop;
          }
          else{
            cpu.dc_reg.dst_id           = cpu.insn.rid[0];
            cpu.dc_reg.src_id[0]        = src_id0;
            cpu.dc_reg.src_id[1]        = src_id1;
            cpu.dc_stt.src[0]   = (cpu.insn.src_f[0]) ? (BYPASS_DC(wb, src_id0) : src_val0) : imm_val;
            cpu.dc_stt.src[1]   = (cpu.insn.src_f[1]) ? (BYPASS_DC(wb, src_id1) : src_val1) : imm_val;
            cpu.dc_reg.src[0]           = cpu.dc_stt.src[0];
            cpu.dc_reg.src[1]           = cpu.dc_stt.src[1];
            cpu.dc_reg.src[2]           = imm_val;
            cpu.dc_reg.op               = cpu.insn.op;
            cpu.dc_reg.dtype            = cpu.insn.dtype;
            cpu.dc_reg.stm_f            = stm_f;
            cpu.dc_reg.ldm_f            = ldm_f;
            cpu.dc_reg.dst_f            = cpu.insn.dst_f;
            cpu.dc_reg.src_f[0]         = cpu.insn.src_f[0];
            cpu.dc_reg.src_f[1]         = cpu.insn.src_f[1];
            cpu.dc_reg.mmu_op           = cpu.insn.mmu_op;
            cpu.dc_reg.cur_pc           = cpu.fe_reg.cur_pc;
          }
        }
  }
}
```

・ #define DC_STALL (cpu.ex_stt.pctl.stall_prev | cpu.wb_stt.pctl.stall_prev)：DC ステー

ジのストール信号は、EX/WB ステージからの逆方向参照の OR 結合である。

- #define BYPASS_DC(PIPE, ID)：後段ステージ（PIPE）から DC ステージへの data-forwarding（ID：レジスタ番号）を記述するためのマクロ
- dec_m17(), dec_m1(), dec_m2(), dec_c3(), dec_c2(), dec_c1(), dec_add()：各命令クラス固有のデコード処理関数。
- if(!cpu.dc_stt.pctl.stalled_flag){ … }：DC ステージのストール状態判定（ストールでない場合は括弧内を処理）。
- DEC_FLD(code);：F_code フィールド（31 – 27 ビット）を cpu.ir から取り出し cpu.insn.code に代入。
- switch(cpu.insn.code){ … }：5 ビットの cpu.insn.code で switch 文による各命令クラスのデコード処理分岐。
  - case I_ldm: if(cpu.insn.dtype == DT_RTN){ … }：リターン命令は LDM 命令クラスに含まれるため（LKR レジスタへのロード命令）、この場合は命令分岐フラブ（cpu.dc_stt.br.active）を 1 に設定し、LKR レジスタ値を命令分岐アドレス（cpu.dc_stt.br.addr）に設定する。
  - BYPASS_DC(wb, R_LKR);：LKR レジスタの WB ステージからの data-forwarding を実行するマクロ。WB ステージに LKR 書込みデータがある場合はこのデータを直接参照する。
  - case I_jmp: case I_call: case I_jcr: case I_bz: case I_bnz: br_hazard = dec_jc();：分岐命令クラス（I_jmp, I_call, I_jcr, I_bz, I_bnz）用デコード関数 dec_jc()の呼出し。戻り値は、分岐ハザードの有無を表す。
  - case I_io: dec_io(); break;：IO 命令デコード記述（後述）
- if(halted || br_hazard){ … }：HALT 命令実行または分岐ハザード発生の場合、ストール信号をパイプライン前段（FE）と後段（EX/WB）に通知する。
- else{ … }：デコードストール発生なしの場合は、命令デコード情報をパイプラインレジスタ（cpu.dc_reg）の各メンバー変数に代入する。
- cpu.dc_stt.src[0] = (cpu.insn.src_f[0]) ? (BYPASS_DC(wb, src_id0) : src_val0) : imm_val;：第 1 オペランドの WB ステージからの data-forwarding 記述。
- cpu.dc_stt.src[1] = (cpu.insn.src_f[1]) ? (BYPASS_DC(wb, src_id1) : src_val1) : imm_val;：第 2 オペランドの WB ステージからの data-forwarding 記述。

```
// TCTProc.h :
#define LDM_PENDING(PIPE, ID)                    ¥
        (cpu.##PIPE##_stt.dst.valid && cpu.##PIPE##_stt.dst.id == ID && cpu.##PIPE##_stt.dst.ldm_f)
```

```
// TCTProc.c :
BIT dec_jc()
{
        BIT r0Used = (cpu.insn.code == I_bz) || (cpu.insn.code == I_bnz) || (cpu.insn.code == I_jcr);
        UINT32    r0 = BYPASS_DC(ex, 0) : BYPASS_DC(wb, 0) : cpu.gpr[0];
        BIT       r0Hazard  = r0Used && LDM_PENDING(ex, 0);
```

```
        UINT32    r0IsZero   = (r0 == 0);
        DEC_FLD(imm27);
        switch(cpu.insn.code){
        case I_bz:          cpu.dc_stt.br.active = r0IsZero;          break;
        case I_bnz:         cpu.dc_stt.br.active = !r0IsZero;break;
        default:            cpu.dc_stt.br.active = 1;                 break;
        }
        switch(cpu.insn.code){
        case I_jcr:         cpu.dc_stt.br.addr = r0;                  break;
        default:    cpu.dc_stt.br.addr = (FC_UINT32)(cpu.fe_reg.cur_pc + (GET_IMM(imm27) << 2));
                    break;
        }
        if(cpu.insn.code == I_call || cpu.insn.code == I_jcr && ((cpu.insn.imm27 & 0x4000000) != 0)){
                    SET_INSN_FLAG(1, 1, 1);       /// M[SP(rid1) - 4] <- LKR(rid2), LKR(rid0) <- PC + 4
                    cpu.insn.rid[0]     = R_LKR;
                    cpu.insn.rid[1]     = R_SP;
                    cpu.insn.rid[2]     = R_LKR;
                    cpu.insn.imm        = -4;
                    cpu.insn.dtype      = DT_INT;
                    cpu.insn.op         = O_call;
        }
        else{ dec_default();}
        return r0Hazard;
}
```

- #define LDM_PENDING(PIPE, ID) : ：後段ステージ（PIPE）でレジスタ（ID：レジスタ番号）のロード命令が実行されているかを検知するためのマクロ
- BIT dec_jc(){ … }：分岐命令クラスのデコード関数。
  - 条件分岐命令（I_bz/I_bnz）：R0 が零かどうか（r0IsZero）で分岐条件が決定。
  - 無条件分岐命令（I_jmp/I_call/I_jcr）：コール型（LKR レジスタに次命令アドレスを代入）とジャンプ型（PC 以外に副作用なし）がある。
  - PC 相対分岐アドレス（I_bz/I_bnz/I_jmp/I_call）：27 ビット即値（符号付き）を現命令アドレスに対するオフセット値として分岐アドレスを計算。
  - レジスタ分岐アドレス（I_jcr）：レジスタ R0 が分岐アドレスとなる。
  - 分岐ハザード条件：
    - ✧ レジスタ R0 を使わない命令（I_jmp/I_call）：分岐ハザードは起きない。
    - ✧ レジスタ R0 を使う命令（I_bz/I_bnz/I_jcr）：直前命令が R0 ロード命令のときのみ分岐ハザードが発生する。この場合以外は、EX/WB からの R0 の data-forwarding によってハザードは回避される。
  - コール型分岐命令の副作用（I_call/I_jcr の一部）：コール命令は 2 つの副作用からなる。
    - ✧ LKR のスタック退避：EX ステージで、LKR のストア命令を実行する（アドレス：SP – 4）
    - ✧ 次命令アドレスの LKR 退避：次命令アドレス（現命令アドレス+4）を LKR にロードする。

## A.7　EXステージ動作記述（TCTProc.c）

下記に EX ステージ動作記述を示す。

```
// TCTProc.h :
#define EX_STALL                        (cpu.dc_stt.pctl.stall_this | cpu.wb_stt.pctl.stall_prev)
#define BYPASS_EX(PIPE, IDX)                                                                    ¥
        (cpu.##PIPE##_stt.dst.valid && cpu.dc_reg.src_f[IDX] &&                                 ¥
        cpu.##PIPE##_stt.dst.id == cpu.dc_reg.src_id[IDX]) ? cpu.##PIPE##_stt.dst.out
```

```
// TCTProc.c:
void execute()
{
   UINT32 src0bp = BYPASS_EX(wb, 0) : cpu.dc_reg.src[0];
   UINT32 src1bp = BYPASS_EX(wb, 1) : cpu.dc_reg.src[1];
   UINT32 ex_out = 0;
   INIT_PIPE_CTRL(cpu.ex_stt.pctl, EX_STALL);
   cpu.ex_stt.out2 = 0;
   cpu.ex_stt.st_data = 0;
   cpu.ex_stt.daddr = 0;
   cpu.ex_stt.wbflag = W_null;
   SET_EX_PENDING_OP(0, 0, 0);

#define DEV_FIFO_PORT &cpu.dummy_fifo_port
   io_fsm(&cpu.io, cpu.dc_reg.op, !cpu.ex_stt.pctl.stalled_flag, src0bp, src1bp, DEV_FIFO_PORT);
   if(cpu.io.active){
      ex_out                    = io_pipe_control();
      cpu.ex_reg.out            = ex_out;
      cpu.ex_reg.wbflag         = W_ex_out;
   }
   else if(cpu.div.pending){ ex_out = div_st_2();}
   else if(!cpu.ex_stt.pctl.stalled_flag){
      UINT32 src0               = src0bp;
      UINT32 src1               = ((cpu.dc_reg.stm_f) ? cpu.dc_reg.src[2] : src1bp);
      UINT32 src1b              = (cpu.dc_reg.op == O_sub) ? (FC_UINT32)(~src1 + 1) :
                                  (cpu.dc_reg.op == O_mova) ? (FC_UINT32)(src1 << 16) : src1;
      UINT32 add_out            = (FC_UINT32)(src0 + src1b);
      BIT eq_flag               = (src0 == src1);
      if(cpu.dc_reg.op == O_nop){          cpu.ex_stt.wbflag = W_null;}
      else if((cpu.dc_reg.op & 0x20) == 0x20){        ///         ldm, stm, call
         UINT32 daddr           = (add_out - DMEM_OFFSET) >> 2;
         UINT2 byte_pos         = (add_out & 0x3);
         switch(cpu.dc_reg.op){
         case O_ldm:  cpu.ex_stt.wbflag = W_dout;    break;
         case O_call:  cpu.ex_stt.wbflag = W_ex_out; ex_out = (FC_UINT32)(cpu.fe_reg.cur_pc + 4);
                                                     break;
         case O_stm:  cpu.ex_stt.wbflag = W_null;    break;
         }
         if(cpu.dc_reg.stm_f){
            cpu.ex_stt.st_data = src1bp;
            write_mem(&cpu.dmem[daddr], byte_enable(byte_pos),
                      format_write_data(cpu.ex_stt.st_data, byte_pos));
         }
         else{ cpu.wb_stt.dout = format_read_data(cpu.dmem[daddr], byte_pos);}
         cpu.ex_stt.daddr = daddr;
      }
      else{
         cpu.ex_stt.wbflag = W_ex_out;
         if(cpu.dc_reg.op & 0x10){           ///         signed/unsigned operators
            BIT isSigned            = (cpu.dc_reg.op & 0x8) != 0;
            BIT sign0               = isSigned & (src0 >> 31);
```

```
        BIT sign1               = isSigned & (src1 >> 31);
        BIT unsigned_lt         = (src0 < src1);
        BIT lt_flag             = sign0 ^ sign1 ^ unsigned_lt;
        switch(cpu.dc_reg.op & 0x7){
        case O_mul: ex_out   = (count_trailing_zeroes(src0) + count_trailing_zeroes(src1) < 32); break;
        case O_div:
        case O_mod:             div_st_1(src0, src1);                       break;
        case O_shr: ex_out   = (src0 >> src1) | ((sign0) ? (((FC_SINT32) 0x80000000) >> src1) : 0);
                                                                            break;
        case O_cgt: ex_out   = (!lt_flag) & (!eq_flag);                     break;
        case O_cge: ex_out   = (!lt_flag) | (eq_flag);                      break;
        case O_clt: ex_out   = (lt_flag) & (!eq_flag);                      break;
        case O_cle: ex_out   = (lt_flag) | (eq_flag);                       break;
        }
      }
      else{             ///         unsigned operators
        switch(cpu.dc_reg.op){
        case O_shl:ex_out = (FC_UINT32)(src0 << src1);                      break;
        case O_neq:             ex_out = !eq_flag;                          break;
        case O_equ:             ex_out = eq_flag;                           break;
        case O_and:             ex_out = src0 & src1;                       break;
        case O_orr:             ex_out = src0 | src1;                       break;
        case O_xor:             ex_out = src0 ^ src1;                       break;
        case O_inv:             ex_out = ~src0;                             break;
        case O_not:             ex_out = !src0;                             break;
        case O_mov:             ex_out = src0;                              break;
        case O_movh:            ex_out = (FC_UINT32)(src0 << 16);           break;
        default:                ex_out = add_out;                           break;
        }
      }
    }
    if((cpu.dc_reg.op & 0x17) == (0x10 | O_mul)){ cpu.ex_reg.out = (FC_UINT32)(src0 * src1);}
    else{ cpu.ex_reg.out = (FC_UINT32) ex_out;}
    cpu.ex_reg.wbflag           = cpu.ex_stt.wbflag;
  }
  SET_FB_DST(ex, (FC_UINT32) ex_out);
}
```

- #define EX_STALL (cpu.dc_stt.pctl.stall_this | cpu.wb_stt.pctl.stall_prev) : EX ステージのストール信号は、DC ステージからの順方向参照（パイプラインレジスタが自動挿入される）と WB ステージからの逆方向参照（実際には WB ステージのストールは発生しない）の OR 結合である。
- #define BYPASS_EX(PIPE, ID)：後段ステージ（PIPE）から EX ステージへの data-forwarding（ID：レジスタ番号）を記述するためのマクロ
- src0bp = BYPASS_EX(wb, 0) : cpu.dc_reg.src[0];：第 1 オペランドの WB ステージからの data-forwarding 記述。
- src1bp = BYPASS_EX(wb, 1) : cpu.dc_reg.src[1];：第 2 オペランドの WB ステージからの data-forwarding 記述。
- io_fsm(&cpu.io, cpu.dc_reg.op, !cpu.ex_stt.pctl.stalled_flag, src0bp, src1bp, DEV_FIFO_PORT);：IO 制御 FSM 動作記述。命令実行と独立した入力処理が必要なため、無条件（実行命令種に関わらず）で実行する。DEV_FIFO_PORT として &cpu.dummy_fifo_port（ダミーFIFO ポート）が定義されているが、FIFO ポートを直

接アクセスする ASIP 命令拡張エンジンがないためダミーFIFO ポートを接続する。

- if(cpu.io.active){ … }:IO 命令が実行された場合の処理(他の命令処理はスキップする)。
- else if(cpu.div.pending){ ex_out = div_st_2();}：除算 FSM 動作記述（除算第 2 状態）
- else if(!cpu.ex_stt.pctl.stalled_flag){ … }：EX ステージのストール状態判定（ストールでない場合は括弧内を処理)。
- add_out = (FC_UINT32)(src0 + src1b);：2 つのオペランドの加算値（加算命令、減算命令の他、メモリアクセス命令におけるアドレス計算（addr + offset）でも使用)。
- else if((cpu.dc_reg.op & 0x20) == 0x20){ … }：メモリアクセス命令動作記述
  - cpu.dc_reg.stm_f（ストアフラグ）：1（STM 命令/CALL 命令)、0（LDM 命令）
  - write_mem(&cpu.dmem[daddr], byte_enable(byte_pos), format_write_data(cpu.ex_stt.st_data, byte_pos));：メモリ書込み動作記述。
  - cpu.wb_stt.dout = format_read_data(cpu.dmem[daddr], byte_pos);:メモリ読出し動作記述。
- else { … }：その他演算命令動作記述
- if(cpu.dc_reg.op & 0x10){ … }：符号付き命令動作記述
- case O_mul: ex_out = (count_trailing_zeroes(src0) + count_trailing_zeroes(src1) < 32);：乗算命令結果の非零判定。DC ステージへの data-forwarding で参照
  - EX ステージ→DC ステージの data-forwarding は、分岐命令デコードにおけるレジスタ R0 の場合のみであり、零判定ができれば良い（乗算命令で分岐アドレスを計算することは想定しない)。
  - UINT8 count_trailing_zeroes(UINT32 n){ … }：n の LSB→MSB 方向で連続する 0 の個数を計算。
  - src0 と src1 の下位ワード部の 0 の個数の合計値が 32 以上の場合は、乗算結果（src0 * src1）が 0 になることを利用している。
  - 実際の乗算結果は別記述部分で行っているが（後述)、このように DC ステージへの data-forwarding 用（零判定用）に下位ワード部の 0 の個数を使う理由は、乗算器の大きな論理遅延を分岐判定処理（DC）と PC 計算（FE）まで伝搬させることを回避するためである。乗算器よりも下位ワード部の 0 個数計算ははるかに小さい論理遅延で実現できる（バレルシフタと同じ論理構造）ため、「ワイヤ逆方向参照」によって EX ステージの演算結果が DC ステージと FE ステージにまで遅延伝播する構造において、クロック動作周波数を高めるための重要な最適化技術である。
- case O_div: case O_mod: div_st_1(src0, src1);：除算 FSM 動作記述（除算初期状態）
  - void div_st_1(UINT32 src0, UINT32 src1){ … }：除算初期状態動作（内部で cpu.div.pending = 1 に設定される）
  - UINT32 div_st_2(){ … }：除算第 2 状態動作（cpu.div.pending == 1 の時に実行さ

れる）

- if((cpu.dc_reg.op & 0x17) == (0x10 | O_mul)){ cpu.ex_reg.out = (FC_UINT32)(src0 * src1);} : 乗算動作記述（EX ステージパイプラインレジスタに直接代入し、WB ステージに転送）。
- else{ cpu.ex_reg.out = (FC_UINT32) ex_out;} : 乗算以外のパイプラインレジスタ代入。

## A.8 WBステージ動作記述（TCTProc.c）

下記に WB ステージ動作記述を示す。

```
// TCTProc.h :
#define WB_STALL  (cpu.dc_stt.pctl.stall_this          | cpu.ex_stt.pctl.stall_this)
// TCTProc.c:
void writeback()
{
        UINT32 wb_data = (cpu.ex_reg.wbflag == W_dout) ? cpu.wb_stt.dout : cpu.ex_reg.out;
        INIT_PIPE_CTRL(cpu.wb_stt.pctl, WB_STALL);
        if(!cpu.wb_stt.pctl.stalled_flag && cpu.dc_reg.dst_f && (cpu.dc_reg.dst_id & 0x20) == 0){
                cpu.gpr[cpu.dc_reg.dst_id & 0x1f] = wb_data;
        }
        SET_FB_DST(wb, wb_data);
}
```

- #define WB_STALL (cpu.dc_stt.pctl.stall_this | cpu.ex_stt.pctl.stall_this) : WB ステージのストール信号は、DC/EX ステージからの順方向参照（パイプラインレジスタが自動挿入される）の OR 結合である。
- cpu.gpr[cpu.dc_reg.dst_id & 0x1f] = wb_data; : 汎用レジスタファイルへの書込み
  - 前節で、レジスタファイル属性について説明したが、汎用レジスタファイルの読出し動作を DC ステージで記述し、WB ステージで書込みを記述する（素直で直感的な）記述方式を許容するためにレジスタファイル属性の導入が（当初）必要であった。現状の C2RTL フレームワークでは、レジスタファイル属性ではなくレジスタ属性を使用し、汎用レジスタファイルへの書込み動作を DC ステージで記述する方式（具体的には、WB ステージでの書込みフラグを逆方向ワイヤ参照で DC ステージに通知する）をサポートしており、これによりレジスタファイル属性は不要となっているが、レジスタファイル属性による直感的記述方式もサポートを続けている。

## A.9 IOデバイスモデル

この節では、IO デバイスモデルの C 記述を説明する。

### A.9.1 IOデバイスリソース定義記述（TCTProc_io.h）

下記に IO デバイスリソース定義記述を示す。

```
enum IO_Enum
{
///         enum InsnEnum
            I_io                = 0x08,

            SO_io_r             = 0x0,
            SO_io_s             = 0x1,
            SO_io_w             = 0x2,
            SO_io_c             = 0x3,
///         enum OpEnum
            O_io_r              = 0x30,
            O_io_s              = 0x31,
            O_io_w              = 0x32,
            O_io_c              = 0x33,

///         device ID
            IO_UART_0           = 0x0,
            IO_UART_1           = 0x1,
            IO_FIFO_0           = 0x2,
};
```

```
#define MAX_IO_BUF_SIZE    2
typedef struct ST_IOBuf
{
            ST_BIT          not_empty, full;
            ST_UINT4        rp, wp;
            RF_UINT32       buf[MAX_IO_BUF_SIZE];       ///         reg-file
} IOBuf;
```

```
typedef UINT32              FIFO_DATA;
typedef ST_UINT32           ST_FIFO_DATA;

typedef struct ST_FIFO_PORT
{
            BIT             ready;      ///     0: busy,                1: ready
            BIT             strobe;     ///     0: no-data,             1: data-valid
            BIT             chanID;     ///     0: raw-data,            1: char-data
            UINT4           bytes;      ///     # of bytes to transmit (may be ignored)
            FIFO_DATA       data;       ///     24-bit data
} FIFO_PORT;
```

```
typedef   FIFO_PORT   D_FIFO_PORT;  ///         direct port signals (not latched)
#ifdef   __TCT_COMPILER_USED__
#pragma  _TCT_verilog_direct_signal             D_FIFO_PORT
#endif
```

```
typedef struct ST_FIFO_RX
{
            FIFO_PORT       port;
            BIT             stalled;
            BIT             ready_out;
            ST_UINT4        bytes_out;
} FIFO_RX;
```

```
typedef struct ST_FIFO_TX
{
            FIFO_PORT       port;
            BIT             stalled;
```

```
        ST_BIT          strobe_out;         ///     (output) 0: no-data,   1: data-valid
        ST_BIT          chanID_out;         ///     (output) 0: raw-data, 1: char-data
        ST_BIT          chanID_out_default;
        ST_FIFO_DATA    data_out;           ///     (output) data-output

} FIFO_TX;
```

```
typedef struct ST_FIFO
{
        FIFO_RX         rx;
        FIFO_TX         tx;
        ST_BIT          activated;
        ST_UINT3        pending_op;
        UINT32          data_in;
        BIT             active, stalled, data_latched;
} FIFO;
```

- enum IO_Enum{ … } : IO デバイス関連 ID
  - I_io = 0x08 : IO 命令クラスコード
  - SO_io_r = 0x0, SO_io_s = 0x1, SO_io_w = 0x2, SO_io_c = 0x3 : 2 ビット命令フィールド F_op_io の ID
    - SO_io_r: IOread (mode = 0 : data input)
    - SO_io_s: IOread (mode = 1 : status read)
    - SO_io_w : IOwrite（mode = 0 : data output）
    - SO_io_c : IOwrite（mode = 1 : command write）
  - O_io_r = 0x30, O_io_s = 0x31, O_io_w = 0x32, O_io_c = 0x33 : 命令演算コード
    - enum OpEnum で定義されている命令演算コード（0x00 – 0x0f : 符号無し、0x10 – 0x1f : 符号付き、0x20 – 0x22 : メモリアクセス）と重複してはならない。
    - EX ステージでの命令演算コード処理を簡単化するため、上位 2 ビットだけで IO 命令演算コードが判別できるようにコードを割り当てている。
  - IO_UART_0 = 0x0, IO_UART_1 = 0x1, IO_FIFO_0 = 0x2 : IO デバイス ID
- typedef struct ST_IOBuf { … } IOBuf; : IO デバイス内臓 FIFO モデル（プロセッサ・IO デバイス間のインターフェース用）
- typedef struct ST_FIFO_PORT { … } FIFO_PORT; : FIFO ポート構造体
  - BIT ready; : FIFO 入力側の状態信号（0 : 入力不可、1 : 入力可）
  - BIT strobe; : FIFO 出力側のストローブ信号（0 : データ有効、1 : データ無効）
  - BIT chanID; : FIFO チャンネル ID（0 : データ、1 : 文字列メッセージ）
  - UINT4 bytes; : FIFO データ転送幅（FIFO 入力側が指定、省略可）
  - FIFO_DATA data; : FIFO データポート
- typedef struct ST_FIFO_RX { … } FIFO_RX; : 入力 FIFO ポート構造体
- typedef struct ST_FIFO_TX { … } FIFO_TX; : 出力 FIFO ポート構造体
- typedef struct ST_FIFO { … } FIFO; : FIFO デバイス構造体

### A.9.2 IO命令デコード記述（TCTProc_io.c）

IO 命令デコードは 2.3.8「DC ステージ動作記述」で説明したように、cpu.insn.code（32 ビット命令の上位 5 ビット）が I_io（= 0x08）の時に dec_io()関数を呼ぶことで実行される。下記が IO 命令デコード記述である。

```
void dec_io()
{
        DEC_FLD(rf0); DEC_FLD(op_sc); DEC_FLD(op_io); DEC_FLD(uimm15);
        cpu.insn.imm = (cpu.insn.op_sc) | (cpu.insn.op_io << 5) | (cpu.insn.uimm15 << 7);
        switch(cpu.insn.op_io){
        case SO_io_r:       cpu.insn.op = O_io_r; break;    ///         io_read_data
        case SO_io_s:       cpu.insn.op = O_io_s; break;    ///         io_read_status
        case SO_io_w:       cpu.insn.op = O_io_w; break;    ///         io_write_data
        case SO_io_c:       cpu.insn.op = O_io_c; break;    ///         io_write_command
        }
        switch(cpu.insn.op_io){
        case SO_io_r:       ///         io_read_data
        case SO_io_s:       ///         io_read_status
                SET_INSN(rid[0], rf0); SET_INSN_FLAG(0, 0, 1);
                break;
        case SO_io_w:       ///         io_write_data
        case SO_io_c:       ///         io_write_command
                SET_INSN(rid[1], rf0); SET_INSN_FLAG(1, 0, 0);
                break;
        }
}
```

- DEC_FLD(op_sc); : [21:17]ビット範囲の 5 ビットフィールドを cpu.insn.op_sc に取り出す。
- DEC_FLD(op_io); : [16:15]ビット範囲の 2 ビットフィールドを cpu.insn.op_io に取り出す。
- DEC_FLD(uimm15); : [14:0]ビット範囲の 15 ビットフィールドを cpu.insn.uimm15 に取り出す。

上記のように、IO 命令クラス（32 ビット命令の上位 5 ビットが 0x8 である命令クラス）のデコード記述は他の命令グラスのデコード記述と独立しているため、上位 5 ビット以外の命令フィールドは、IO 命令クラス固有の形式をとることができる。

### A.9.3 IO制御FSM動作記述（TCTProc_io.c）

下記に IO 制御 FSM 動作記述を示す。

```
void io_fsm(IO * io, UINT7 dc_reg_op, BIT active, UINT32 src0, UINT32 io_com, FIFO_PORT *
dev_fifo_port)
{
   UINT5  io_id;
   UINT3  io_op;
   UINT12 io_param;
   int is_io_insn        = (active && (dc_reg_op & 0x30) == 0x30);
   int cancel_io_insn = (!active && (dc_reg_op & 0x30) != 0x30);
   if(is_io_insn){
```

```
    io_id = (io_com & 0x1f);
    io_op = ((io_com >> 5) & 0x3) | 0x4;
    io_param = ((io_com >> 7) & 0x7fff);
  }
  else{
    io_id = 0;
    io_op = IO_Idle;
    io_param = 0;
  }
  io->stalled          = 0;
  io->data_in          = 0;
  io->data_latched     = 0;
  io->active           = 0;
  fifo_fsm(io, &io->fifo, dev_fifo_port, cancel_io_insn, (io_id == IO_FIFO_0) ? io_op : IO_Idle, io_param,
src0);
}
```

- is_io_insn = (active && (dc_reg_op & 0x30) == 0x30);：命令発行時 IO 命令判定
- cancel_io_insn = (!active && (dc_reg_op & 0x30) != 0x30);：ストール時非 IO 命令判定
  - EX ステージがストール状態（!active）であり、発行命令が IO 命令でない。
- ここでは、内部で fifo_fsm()を呼出しているが、他の IO デバイスがある場合（UART など）もここからその FSM 動作記述を呼び出すことになる。

```
UINT32 io_pipe_control()
{
        if(cpu.io.stalled){
                cpu.ex_stt.pctl.stall_this = 1;
                cpu.ex_stt.pctl.stall_prev = 1;
                SET_EX_PENDING_OP(1, 1, 0);
        }
        return cpu.io.data_in;
}
```

- UINT32 io_pipe_control(){ … } : execute()内部で、cpu.io.active == 1 の時に呼び出される。
- if(cpu.io.stalled){ … }：いずれかの IO デバイスがストールした場合、前後のパイプラインステージにストール通知を行う。
- return cpu.io.data_in;：IO デバイスの読出し結果（データ読出し、状態読出し）を EX ステージ出力に戻す。

### A.9.4 FIFO制御FSM動作記述（TCTProc_io.c）

下記に FIFO 制御 FSM 動作記述を示す。

```
void fifo_fsm(IO * io, FIFO * fifo, FIFO_PORT * dev_fifo_port, BIT cancel_io_insn, UINT3 io_op_in, UINT15
io_param, UINT32 src0)
{
   UINT32 status = 0, rx_data = 0;
   UINT3 io_op = (cancel_io_insn) ? IO_Idle : (io_op_in | fifo->pending_op);
   fifo->stalled = 0;
   if(io_op == IO_WriteCommand)          { fifo_write_command(fifo, src0);}
   else{
      if(io_op == IO_ReadStatus)         { status = fifo_read_status(fifo);}
      if(fifo->activated){
         fifo_tx(fifo, dev_fifo_port, io_op, io_param & 0x1, src0);
         rx_data       = fifo_rx(fifo, dev_fifo_port, io_op);
         fifo->stalled = fifo->tx.stalled | fifo->rx.stalled;
      }
      else{ fifo->rx.ready_out = 0;}
   }
   if(fifo->stalled){
      fifo->pending_op          = io_op;
      fifo->data_latched        = 0;
   }
   else{
      fifo->pending_op          = IO_Idle;
      fifo->data_in             = status | rx_data;
      fifo->data_latched        = (io_op == IO_ReadStatus) | (io_op == IO_ReadData);
   }
   fifo->active                 = (io_op != IO_Idle);
   fifo->rx.port.ready          = fifo->rx.ready_out;
   fifo->rx.port.bytes          = fifo->rx.bytes_out;
   fifo->tx.port.strobe         = fifo->tx.strobe_out;
   fifo->tx.port.chanID         = fifo->tx.chanID_out;
   fifo->tx.port.data           = fifo->tx.data_out;
   io->stalled                  |= fifo->stalled;
   io->data_in                  |= fifo->data_in;
   io->data_latched             |= fifo->data_latched;
   io->active                   |= fifo->active;
}
```

- if(io_op == IO_WriteCommand){ fifo_write_command(fifo, src0);}：FIFO コマンド書込み動作記述。
- if(io_op == IO_ReadStatus){ status = fifo_read_status(fifo);}：FIFO 状態読出し動作記述。
- if(fifo->activated){ … }：FIFO が「アクティベート」状態の場合のデータ入出力動作記述（FIFO コマンド書込み命令によって FIFO のアクティベート・非アクティベート状態を設定できる）。
  - fifo_tx(fifo, dev_fifo_port, io_op, io_param & 0x1, src0);：FIFO データ出力動作記述。
  - rx_data = fifo_rx(fifo, dev_fifo_port, io_op);：FIFO データ入力動作記述。

## A.10 外部FIFOテストベンチモデル動作記述（TCTProc_io.c）

下記に外部 FIFO テストベンチモデル（RTL 合成対象外）と FIFO ポートインターフェースモデルの動作記述を示す。

```
void set_fifo_FW(FIFO_PORT * fifo_src, FIFO_PORT * fifo_dst)
{
        fifo_dst->strobe    = fifo_src->strobe;
        fifo_dst->chanID    = fifo_src->chanID;
        fifo_dst->data      = fifo_src->data;
}
void set_fifo_BW(FIFO_PORT * fifo_src, FIFO_PORT * fifo_dst)
{
        fifo_src->ready     = fifo_dst->ready;
        fifo_src->bytes     = fifo_dst->bytes;
}
void set_fifo_input_ports(FIFO * fifo, FIFO_PORT * in_fifo, FIFO_PORT * out_fifo)
{
        set_fifo_FW(in_fifo, &fifo->rx.port);
        set_fifo_BW(&fifo->tx.port, out_fifo);
}
void set_fifo_output_ports(FIFO * fifo, FIFO_PORT * in_fifo, FIFO_PORT * out_fifo)
{
        set_fifo_BW(in_fifo, &fifo->rx.port);
        set_fifo_FW(&fifo->tx.port, out_fifo);
}
```

- void set_fifo_FW(FIFO_PORT * fifo_src, FIFO_PORT * fifo_dst){ … }：FIFO ポートインターフェースの順方向信号伝搬（FIFO 出力ソース→FIFO 入力ターゲット）。
- void set_fifo_BW(FIFO_PORT * fifo_src, FIFO_PORT * fifo_dst){ … }：FIFO ポートインターフェースの逆方向信号伝搬（FIFO 出力ソース←FIFO 入力ターゲット）。
- void set_fifo_input_ports(FIFO * fifo, FIFO_PORT * in_fifo, FIFO_PORT * out_fifo){ … }：プロセッサ FIFO ポートとテストベンチ FIFO モデルの信号伝搬（ISS コア関数（TCTProcFIFO）のプロセッサ動作記述の直前に呼び出される。
- void set_fifo_output_ports(FIFO * fifo, FIFO_PORT * in_fifo, FIFO_PORT * out_fifo){ … }：プロセッサ FIFO ポートとテストベンチ FIFO モデルの信号伝搬（ISS コア関数（TCTProcFIFO）のプロセッサ動作記述の直後に呼び出される。

```
#define EXT_FIFO_BUF_SIZE 1000
struct EXT_FIFO
{
        int cpu_cycle, cpu_mode;
        UINT32 * p_cpu_cycle;
        FIFO_PORT in, out;
        char rx_buf[EXT_FIFO_BUF_SIZE];
        int rx_pnt;
        int in_count, out_count, match_count, no_match_count, in_fifo_terminated, out_fifo_terminated;
        int in_error_count, out_error_count;
        int in_interval, out_interval;
        int prev_in_cycle, prev_out_cycle;
        FILE * src_img_fp, * out_jpg_fp;
```

```
        int in_bytes, in_residue_bytes, in_residue_data;
} x_fifo = {0};
```

- struct EXT_FIFO{ … } x_fifo = {0};：テストベンチ FIFO モデル用構造体。
  - FIFO_PORT in, out;：プロセッサ（RTL 合成対象）の FIFO 入出力ポートとのインターフェース。構造体自体はプロセッサの FIFO ポートと共通。
  - char rx_buf[EXT_FIFO_BUF_SIZE];：プロセッサ FIFO 出力ポートからの文字列出力用バッファ。
  - その他メンバー変数：アプリケーション依存部分。（プロセッサ FIFO 入力ポートへのデータ供給方法やプロセッサ FIFO 出力ポートのデータ検証方法はアプリケーション依存の実装となる）

```
#define MAX_INTERVAL                1//5
#define RAND_SEED                   0
#define IN_RAND_INTERVAL            (abs(rand()) % MAX_INTERVAL)
#define OUT_RAND_INTERVAL           (abs(rand()) % MAX_INTERVAL)
#define MAX_MSG_COUNT               20

void fifo_port_reset(FIFO_PORT * fifo)
{
   fifo->chanID         = 0;
   fifo->data           = 0;
   fifo->ready          = 0;
   fifo->strobe         = 0;
   fifo->bytes          = 0;
}
```

```
void fifo_ext_set_in_interval()
{ x_fifo.in_interval   = IN_RAND_INTERVAL;   x_fifo.in.strobe = (x_fifo.in_interval   == 0);}
```

```
void fifo_ext_set_out_interval()
{ x_fifo.out_interval = OUT_RAND_INTERVAL; x_fifo.out.ready = (x_fifo.out_interval == 0);}
```

```
#include "../../TEST/myJPEG5_2/image_info.h"
void fifo_ext_init(FIFO_PORT * in_fifo, FIFO_PORT * out_fifo, UINT32 * p_cpu_cycle)
{
   srand(RAND_SEED);
   x_fifo.p_cpu_cycle           = p_cpu_cycle;
   fifo_port_reset(&x_fifo.in);
   fifo_ext_set_in_interval();
   x_fifo.in.chanID             = 0;
   x_fifo.in.data               = 0;
   fifo_port_reset(&x_fifo.out);
   fifo_ext_set_out_interval();
   x_fifo.in_count              = 0;
   x_fifo.in_count ++;
   x_fifo.out_count             = 0;
   x_fifo.match_count           = 0;
   x_fifo.no_match_count        = 0;
   x_fifo.in_fifo_terminated    = 0;
   x_fifo.out_fifo_terminated   = 0;
   x_fifo.cpu_cycle             = 0;
   x_fifo.cpu_mode              = 0;
   x_fifo.in_error_count        = 0;
   x_fifo.out_error_count       = 0;
   x_fifo.prev_in_cycle         = 0;
   x_fifo.prev_out_cycle        = 0;
   x_fifo.in_bytes              = 0;
   x_fifo.in_residue_bytes      = 0;
```

```
   x_fifo.in_residue_data          = 0;

   if(x_fifo.src_img_fp){ fclose(x_fifo.src_img_fp);}
   if(x_fifo.out_jpg_fp){ fclose(x_fifo.out_jpg_fp);}
   x_fifo.src_img_fp = fopen(src_img_file_name, "rb");
   x_fifo.out_jpg_fp = fopen(out_jpg_file_name, "rb");
}
```

```
void fifo_ext_print_result()
{
   printf("x_fifo: match_count = %d, no_match_count = %d, out_fifo_count = %d, in_fifo_count = %d,
out_error_count = %d¥n",
      x_fifo.match_count, x_fifo.no_match_count, x_fifo.out_count, x_fifo.in_count, x_fifo.out_error_count);
}
```

- void fifo_port_reset(FIFO_PORT * fifo) { … };：FIFO_PORT 各信号のリセット。
- void fifo_ext_set_in_interval(){ … }：テストベンチ入力 FIFO モデルのデータ供給インターバル設定（プロセッサモデル外部の入力側 FIFO モデルのストール状態をエミュレートすることが目的）。
- void fifo_ext_set_out_interval(){ … }：テストベンチ出力 FIFO モデルのデータ消費インターバル設定（プロセッサモデル外部の出力側 FIFO モデルのストール状態をエミュレートすることが目的）。
- #include "../../TEST/myJPEG5_2/image_info.h"：テストベンチ FIFO モデルのデータファイル定義の読込み（予めアプリケーション側で準備する）。
- void fifo_ext_init(FIFO_PORT * in_fifo, FIFO_PORT * out_fifo, UINT32 * p_cpu_cycle){ … }：テストベンチ FIFO モデル初期化。
- x_fifo.src_img_fp = fopen(src_img_file_name, "rb");：FIFO データ供給用ファイル名 src_img_file_name は、"../../TEST/myJPEG5_2/image_info.h"で定義されている。
- x_fifo.out_jpg_fp = fopen(out_jpg_file_name, "rb");：FIFO データ検証用ファイル名 out_jpg_file_name は、"../../TEST/myJPEG5_2/image_info.h"で定義されている。
- void fifo_ext_print_result(){ … }：テストベンチ FIFO モデルの内部状態出力（データ検証結果等）。

```
void x_out_fifo_update()
{
   int cpu_cycle = (x_fifo.cpu_mode == 1) ? x_fifo.cpu_cycle : *x_fifo.p_cpu_cycle;
   if(x_fifo.out.chanID == 1){     ///            char-data
      if(x_fifo.rx_pnt == EXT_FIFO_BUF_SIZE - 3){
         x_fifo.rx_buf[x_fifo.rx_pnt ++]        = '¥n';
         x_fifo.rx_buf[x_fifo.rx_pnt]           = 0;
         printf("[%10d] <FIFO_X.RX.pending> msg = %s", cpu_cycle, x_fifo.rx_buf);
         x_fifo.rx_pnt = 0;
      }
      x_fifo.rx_buf[x_fifo.rx_pnt ++]           = x_fifo.out.data;
      x_fifo.rx_buf[x_fifo.rx_pnt]  = 0;
      if(x_fifo.out.data == '¥n'){
         printf("[%10d] <FIFO_X.RX> msg = %s", cpu_cycle, x_fifo.rx_buf);
         x_fifo.rx_pnt = 0;
      }
   }
```

```
    else if(!x_fifo.out_fifo_terminated){        ///            x_fifo.out.chanID == 0
        if(x_fifo.out_jpg_fp){
            int c = getc(x_fifo.out_jpg_fp);
            if(c == EOF){ x_fifo.out_fifo_terminated = 1;}
            else{
                int matched = (c == x_fifo.out.data);
                x_fifo.match_count      += matched;
                x_fifo.no_match_count           += !matched;
            }
        }
        else{ x_fifo.out_fifo_terminated = 1;}
        x_fifo.prev_out_cycle = cpu_cycle;
        x_fifo.out_count ++;
    }
    else{ x_fifo.out_error_count ++;}
}
```

```
void x_in_fifo_update()
{
    int cpu_cycle = (x_fifo.cpu_mode == 1) ? x_fifo.cpu_cycle : *x_fifo.p_cpu_cycle;
    int c;
    if(!x_fifo.in_fifo_terminated){
        x_fifo.prev_in_cycle = cpu_cycle;
        x_fifo.in.strobe   = 1;
        x_fifo.in.chanID  = 0;
        if(x_fifo.src_img_fp){
            int rx_bytes = x_fifo.in.bytes, i, d = x_fifo.in_residue_data;
            for(i = x_fifo.in_residue_bytes; i < rx_bytes; i ++){
                c = getc(x_fifo.src_img_fp);
                if(c == EOF){
                    x_fifo.in_fifo_terminated = 1;
                    break;
                }
                else{ d |= (c & 0xff) << (i * 8);}
            }
            x_fifo.in.data = d;
            x_fifo.in_residue_bytes   = 0;
            x_fifo.in_residue_data    = 0;
        }
        if(!x_fifo.in_fifo_terminated){ x_fifo.in_count ++;}
    }
    else{
        if(x_fifo.in_fifo_terminated == 1){
            x_fifo.in.strobe              = 1;
            x_fifo.in.chanID              = 1;
            x_fifo.in.data                = 0;
            x_fifo.in_fifo_terminated = 2;
            printf("[%10d] <FIFO_X.TX> : in_fifo_terminated (sending data to chan[1])¥n", cpu_cycle);
        }
        else{ x_fifo.in.strobe          = 0;}
    }
}
```

```
void fifo_ext_input(FIFO_PORT * in_fifo, FIFO_PORT * out_fifo)
{
    set_fifo_FW(&x_fifo.in, in_fifo);
    set_fifo_BW(out_fifo, &x_fifo.out);
    if(x_fifo.in_interval > 0)        { x_fifo.in.strobe = 0; x_fifo.in_interval --;}
    else                                                                { x_fifo.in.strobe = 1;}
    if(x_fifo.out.strobe)             { x_out_fifo_update(); fifo_ext_set_out_interval();}
    if(x_fifo.cpu_mode == 1)          { x_fifo.cpu_cycle ++;}
}
```

```
void fifo_ext_output(FIFO_PORT * in_fifo, FIFO_PORT * out_fifo)
{
    set_fifo_BW(&x_fifo.in, in_fifo);
```

```
    set_fifo_FW(out_fifo, &x_fifo.out);
    if(x_fifo.out_interval > 0)       { x_fifo.out.ready = 0; x_fifo.out_interval --;}
    else                                                                { x_fifo.out.ready = 1;}
    if(x_fifo.in.ready)               { x_in_fifo_update(); fifo_ext_set_in_interval();}
    if(x_fifo.in.bytes != x_fifo.in_bytes){
      if(x_fifo.in.bytes > x_fifo.in_bytes){
        int i, d = x_fifo.in.data << ((x_fifo.in.bytes - x_fifo.in_bytes) * 8);
        if(x_fifo.src_img_fp){
          for(i = 0; i < (x_fifo.in.bytes - x_fifo.in_bytes); i ++){
            int c = getc(x_fifo.src_img_fp);
            if(c == EOF){
              x_fifo.in_fifo_terminated = 1;
              break;
            }
            else{ d |= (c & 0xff) << (i * 8);}
          }
        }
        x_fifo.in.data = d;
      }
      else{
        x_fifo.in_residue_bytes   = x_fifo.in_bytes - x_fifo.in.bytes;
        x_fifo.in_residue_data    = x_fifo.in.data >> ((x_fifo.in_bytes - x_fifo.in_residue_bytes) * 8);
        x_fifo.in.data            &= 0xffffffff >> (x_fifo.in_residue_bytes * 8);
      }
      x_fifo.in_bytes = x_fifo.in.bytes;
    }
}
```

```
void fifo_ext_switch_mode(FIFO_PORT * in_fifo, FIFO_PORT * out_fifo)
{
    if(x_fifo.rx_pnt){ x_fifo.out.chanID = 1; x_fifo.out.data = '¥n'; x_out_fifo_update();}
    fifo_ext_init(in_fifo, out_fifo, x_fifo.p_cpu_cycle);
    x_fifo.cpu_mode = 1;
}
```

- void x_out_fifo_update(){ … } : テストベンチ出力 FIFO モデル更新記述（FIFO 出力値検証）
- void x_in_fifo_update(){ … } : テストベンチ入力 FIFO モデル更新記述（FIFO 入力データ供給）
- void fifo_ext_input(FIFO_PORT * in_fifo, FIFO_PORT * out_fifo){ … } : ISS ラッパー関数内部における ISS コア関数処理前のテストベンチ FIFO 状態更新処理。
- void fifo_ext_output(FIFO_PORT * in_fifo, FIFO_PORT * out_fifo){ … } : ISS ラッパー関数内部における ISS コア関数処理後のテストベンチ FIFO 状態更新処理。

# B. TCTProcJPG_FIFOのCモデル

本節では、/Applications/TCT_ProcModel/Src/CORE/JPGFifoVersion/の JPEG エンコーダ ASIP の C モデルについて説明する。

## B.1　ASIP拡張命令用リソース定義（TCTProc_jpg.h）

```
enum JPG_Enum
{
        I_jpg       = 0x09,

        SO_jpg_reg_read     = 0x01,   /// jpg_reg_ld          rf0, rf1     : gpr[rf0]   <- jpg.reg[gpr[rf1]]
        SO_jpg_reg_write    = 0x02,   /// jpg_reg_st          rf0, rf1     : jpg.reg[gpr[rf1]]      <- gpr[rf0]
        SO_jpg_inc_frame    = 0x03,   /// jpg_inc_frame
        SO_jpg_inc_line     = 0x04,   /// jpg_inc_line
        SO_jpg_in_rgb2ycbcr = 0x05,   /// jpg_in rf0                       : gpr[rf0]   <- rgb2ycrcb(in_rgb)
        SO_jpg_inc_block_line = 0x08, /// jpg_inc_block_line
        SO_jpg_dct          = 0x0a,   /// jpg_dct rf1: dct(rf1)            [rf1:0(horizontal), 1(vertical)]
        SO_jpg_quant        = 0x0c,   /// jpg_quant           rf0          : gpr[rf0]   <- quant()
        SO_jpg_emit_byte    = 0x07,   /// jpg_emit_byte       rf0          : emit_byte(gpr[rf0])
        SO_jpg_emit_bits    = 0x09,   /// jpg_emit_bits       rf0, rf1     : emit_bits(gpr[rf0], gpr[rf1])
        SO_jpg_enc_coef     = 0x0d,   /// jpg_enc_coef rf0, rf1: gpr[rf0]<- enc_coef(rf1)[rf1: 0(dc), 1(ac)]
        O_jpg_mask          = 0x50,
};
```

```
#define DCTBL_ADDR_BITS             4
#define ACTBL_ADDR_BITS             8
#define DCTBL_SIZE                  (1 << DCTBL_ADDR_BITS)    // 16//256
#define ACTBL_SIZE                  (1 << ACTBL_ADDR_BITS)    // 256
#define DCTBL_ADDR_MASK             (DCTBL_SIZE - 1)
#define ACTBL_ADDR_MASK             (ACTBL_SIZE - 1)
```

```
enum JPG_reg_size_enum
{
        JPG_SZ_pixBytes             = 1,
        JPG_SZ_imageSizeX           = 1,
        JPG_SZ_imageSizeY           = 1,
        JPG_SZ_emit_byte_count      = 1,
        JPG_SZ_emit_byte_check      = 1,
        JPG_SZ_quantTableY          = 64,
        JPG_SZ_quantTableCbCr       = 64,
        JPG_SZ_dctbl                = DCTBL_SIZE * 2,
        JPG_SZ_actbl                = ACTBL_SIZE * 2,
};
```

```
#define JPG_IDX0(n)         JPG_##n = 0
#define JPG_IDX(n, m)       JPG_##n = (JPG_##m + JPG_SZ_##m)
#define JPG_R(m)            (JPG_##m + JPG_SZ_##m)
```

```
enum JPG_pipe_command_enum
{
        JPG_IDX0(pixBytes),                                 ///     1
        JPG_IDX(imageSizeX,         pixBytes),              ///     1
        JPG_IDX(imageSizeY,         imageSizeX),            ///     1
        JPG_IDX(emit_byte_count,    imageSizeY),            ///     1
        JPG_IDX(emit_byte_check,    emit_byte_count),       ///     1
        JPG_IDX(quantTableY,        emit_byte_check),       ///     64
        JPG_IDX(quantTableCbCr,     quantTableY),           ///     64
        JPG_IDX(dctbl,              quantTableCbCr),        ///     512
```

```
        JPG_IDX(actbl,          dctbl),         ///     512
};
```

- enum JPG_Enum { … };：JPEG エンコーダ ASIP 拡張命令のコード定義
  - I_jpg = 0x09：JPEG エンコーダ ASIP の命令クラスコード
  - SO_jpg_xx = yy：JPEG エンコーダ ASIP の命令 OP コード
- enum JPG_reg_size_enum{ …. };：JPEG エンコーダリソース（レジスタ、メモリ）のサイズ定義（メモリリソースの場合、メモリワード数がサイズに対応する）
- enum JPG_pipe_command_enum{ … };：JPEG エンコーダリソースの識別 ID（メモリの場合、識別 ID からアドレス値が計算できるように、各リソースのサイズがオフセットになるように識別 ID を割り当てている）

```
#define MAX_JPG_WIDTH   1600
#define JPG_PIPE_COUNT  1
```

```
typedef struct ST_JPG_PIPE_REG
{
        ST_UINT7        op;
        ST_UINT32       out;
        ST_BIT          out_pending;
} JPG_PIPE_REG;
```

```
typedef struct ST_JPG_PIPE_SIG
{
        BIT             stall, pending, out_ready;
        UINT32          out;
} JPG_PIPE_SIG;
```

```
typedef struct ST_JPG_DCT_LINE
{
        ST_SINT14       word[8];
} JPG_DCT_LINE;
```

```
typedef struct ST_PIXState
{
        ST_UINT16       pos_x, pos_y, Y_pos, CbCr_pos, edge_Y_pos, edge_CbCr_pos;
        ST_UINT8        edge_Y, edge_Cb, edge_Cr;
} PIXState;
```

```
typedef struct ST_BLKState
{
        ST_BIT          new_frame, out_of_bounds;
        ST_UINT3        ID;             ///     0 - 5
        ST_UINT16       pos_x, pos_y;
        ST_UINT3        pix_x, pix_y;
        UINT32          out;
} BLKState;
```

```
typedef struct ST_DCTState
{
        JPG_DCT_LINE    data[8];
        SINT16          in_data[8], out_data[8];
        ST_BIT          dir[2], pending;
        UINT3           pix_x;
} DCTState;
```

```
typedef struct ST_QUANT
{
        M_UINT15        quantTable[64 * 2];
        ST_UINT3        ID, ID2;        ///     0 - 5
        ST_UINT6        pos, addr, buf_idx, last_nz_idx;
        BIT             div_pending[2];
        ST_BIT          quant_pending, qtable_latched, neg_flag;
```

```
        ST_UINT15       qtable_data;
        UINT15          qtable_out;
        BIT             qtable_out_active;
        UINT6           qtable_addr;
        ST_UINT6        zig_zag_pos;
        ST_UINT4        run_length;
        ST_UINT4        cnt;
        ST_UINT15       nom, q;
        ST_UINT18       denom, rem;
        ST_SINT16       DCy, DCcb, DCcr, curDC;
} QUANT;
```

```
typedef struct ST_HUFF
{
        M_UINT21        dctbl[2][DCTBL_SIZE], actbl[2][ACTBL_SIZE];
        ST_UINT32       emit_byte_count, emit_byte_check;
        ST_UINT24       buf;
        ST_UINT6        bits;
        ST_BIT    stuff_flag, bits_pending, tbl_pending, code_pending, end_of_block, enc_pending;
        ST_UINT16       code;
        ST_UINT6        code_bits, coef_cnt;
        UINT21          tbl_out;
        ST_SINT16       ac;
        ST_UINT4        run_length;
        ST_UINT6        Q_idx;
        UINT3           ID;
} HUFF;
```

```
typedef struct ST_JPG
{
   M_UINT32 Y[2][MAX_JPG_WIDTH * 2], Cb[2][MAX_JPG_WIDTH/2], Cr[2][MAX_JPG_WIDTH/2];
        ST_UINT24       Y_tmp, Cb_tmp, Cr_tmp;
        M_UINT8         Cb0[2][MAX_JPG_WIDTH/2], Cr0[2][MAX_JPG_WIDTH/2];
        M_SINT20        Qbuf[64];
        SINT20          Q_out;
        UINT16          CbCr_out;
        UINT32          Y_out;
        PIXState        pix;
        DCTState        dct;
        BLKState        blk;
        QUANT           q;
        HUFF            h;
        ST_UINT16       pixBytes, imageSizeX, imageSizeY;
        ST_UINT8        curCb0[2], curCr0[2];
        UINT8           prevCb0[2], prevCr0[2];
        ST_BIT          edge_fsm_state, dsamp_fsm_state, dct_fsm_state, quant_fsm_state;
        ST_BIT          new_frame;
        ST_UINT8        i_param[2];
        JPG_PIPE_SIG    pipe_sig[JPG_PIPE_COUNT];
        JPG_PIPE_REG    pipe_reg[JPG_PIPE_COUNT];
        BIT             active, in_fifo_stall, out_fifo_stall, any_stall;
        BIT             pending;
        ST_BIT          activated, jpg_stalled, in_fifo_stalled, out_fifo_stalled;
        UINT32          out;
        ST_UINT32       out_pix;
        FIFO_PORT       fifo_port;
} JPG;
```

・ typedef struct ST_JPG_PIPE_REG { … } JPG_PIPE_REG;：JPEG エンコーダエンジンのパイプラインレジスタ

- typedef struct ST_JPG_PIPE_SIG { … } JPG_PIPE_SIG;：JPEG エンコーダエンジンの出力信号（ワイヤ変数）
- typedef struct ST_JPG_DCT_LINE { … } JPG_DCT_LINE;：8 点 DCT 処理を行うための内部レジスタ
- typedef struct ST_PIXState { … } PIXState;：JPEG エンコーダの入力段処理（RGB-YCbCr 変換）の位置情報を格納したレジスタ群
- typedef struct ST_BLKState { … } BLKState;：JPEG エンコーダの 8x8 ブロック処理の位置情報を格納したレジスタ群
- typedef struct ST_DCTState { … } DCTState;：DCT 変換処理 FSM 状態レジスタ群
- typedef struct ST_QUANT { … } QUANT;：量子化処理 FSM 状態レジスタ群
- typedef struct ST_HUFF { … } HUFF;：Huffman 符号化処理 FSM 状態レジスタ群
- typedef struct ST_JPG { … } JPG;：JPEG エンコーダ全体のリソース定義

このようにして定義した JPEG エンコーダリソースは、最終的にプロセッサ全体のリソース構造体 CPU のメンバー変数として組み込む。

```
// TCTProcArch.h
typedef struct ST_CPU
{
        …
        JPG     jpg;
        …
} CPU;
```

## B.2 ASIP拡張命令デコード記述

ASIP 拡張命令デコード記述は、2.3.12 で説明した IO 命令デコード記述と同じコード構造で実装できる。

- TCT プロセッサの最上位デコード関数 decode()内部で、ASIP 固有命令クラスコード（例：I_jpg = 0x09）の検知時に、ASIP 固有命令デコード関数にジャンプしてデコード処理を行う。
- ASIP 固有命令デコード関数では、各命令フィールドを取り出し、対応する ASIP 固有命令 OP コード割当、レジスタ ID デコード等を行う。

下記に JPEG エンコーダ拡張命令のデコード関数 dec_jpg()を示す。ここでは、比較的簡単な命令フォーマットを採用していおり、op_sc フィールド（[21:17]ビット範囲の 5 ビットフィールド）と 3 つのレジスタフィールド（rf0, rf1, rf2）のみによって命令デコードを行っている。

```
void dec_jpg()
{
        DEC_FLD(rf0); DEC_FLD(op_sc); DEC_FLD(rf1); DEC_FLD(rf2);
        switch(cpu.insn.op_sc){
        case SO_jpg_reg_read:           /// jpg_reg_ld rf0, rf1 : gpr[rf0]   <- jpg.reg[gpr[rf1]]
                cpu.insn.op = O_jpg_mask | SO_jpg_reg_read; SET_INSN_FLAG(1, 0, 1);
```

```
                cpu.insn.rid[0] = cpu.insn.rf0;              ///          dstID      <- rf0
                cpu.insn.rid[1] = cpu.insn.rf1;              ///          src0ID     <- rf1
                break;
        case SO_jpg_reg_write:          /// jpg_reg_st rf0, rf1  : jpg.reg[gpr[rf1]]      <- gpr[rf0]
                cpu.insn.op = O_jpg_mask | SO_jpg_reg_write; SET_INSN_FLAG(1, 1, 0);
                cpu.insn.rid[1] = cpu.insn.rf1;              ///          src0ID     <- rf1
                cpu.insn.rid[2] = cpu.insn.rf0;              ///          src1ID     <- rf0
                break;
        case SO_jpg_inc_frame:
                cpu.insn.op = O_jpg_mask | SO_jpg_inc_frame; SET_INSN_FLAG(0, 0, 0);
                break;
        case SO_jpg_inc_line:
                cpu.insn.op = O_jpg_mask | SO_jpg_inc_line; SET_INSN_FLAG(0, 0, 0);
                break;
        case SO_jpg_in_rgb2ycbcr:
                cpu.insn.op = O_jpg_mask | SO_jpg_in_rgb2ycbcr;   SET_INSN_FLAG(0, 0, 1);
                cpu.insn.rid[0] = cpu.insn.rf0;              ///          dstID      <- rf0;
                break;
        case SO_jpg_inc_block_line:    /// jpg_inc_block_line
                cpu.insn.op = O_jpg_mask | SO_jpg_inc_block_line;   SET_INSN_FLAG(0, 0, 0);
                break;
        case SO_jpg_dct:                /// jpg_dct rf1          : dct(rf1)   [rf1:0(horizontal), 1(vertical)]
                cpu.insn.op = O_jpg_mask | SO_jpg_dct;  SET_INSN_FLAG(0, 0, 0);
                cpu.jpg.i_param[0] = cpu.insn.rf1;           /// rf1:0(horizontal), 1(vertical)
                break;
        case SO_jpg_quant:              /// jpg_quant            : quant()
                cpu.insn.op = O_jpg_mask | SO_jpg_quant; SET_INSN_FLAG(0, 0, 0);
                break;
        case SO_jpg_emit_byte:          /// jpg_emit_byte       rf0 : emit_byte(gpr[rf0])
                cpu.insn.op = O_jpg_mask | SO_jpg_emit_byte; SET_INSN_FLAG(1, 0, 0);
                cpu.insn.rid[1] = cpu.insn.rf1;    ///          src0ID     <- rf1
                break;
        case SO_jpg_emit_bits: /// jpg_emit_bits   rf0, rf1      : emit_bits(gpr[rf0], gpr[rf1])
                cpu.insn.op = O_jpg_mask | SO_jpg_emit_bits; SET_INSN_FLAG(1, 1, 0);
                cpu.insn.rid[1] = cpu.insn.rf1;              ///          src0ID     <- rf1
                cpu.insn.rid[2] = cpu.insn.rf2;              ///          src1ID     <- rf2
                break;
        case SO_jpg_enc_coef: /// jpg_enc_coef   rf0, rf1 : gpr[rf0] <- enc_coef(rf1) [rf1: 0(dc), 1(ac)]
                cpu.insn.op = O_jpg_mask | SO_jpg_enc_coef; SET_INSN_FLAG(0, 0, 1);
                cpu.insn.rid[0] = cpu.insn.rf0;              ///          dstID      <- rf0
                cpu.jpg.i_param[0]    = cpu.insn.rf1;        ///          rf1: 0(dc), 1(ac)
                break;
        }
}
```

なお、上記 JPEG エンコード拡張命令の TCT-intrinsic 関数定義を下記に示す。

（/Applications/TCT_ProcModel/Src/TEST/myJPEG5_2/myJPEG_ext.c 参照）

```
int JPG_reg_read(int jpgRegID){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_reg_read))}

void JPG_reg_write(int jpgRegID, int data){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg,
SO_jpg_reg_write)) TCT_SWAP_REG_FIELD;}

void JPG_inc_frame(){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_inc_frame))}

void JPG_inc_line(){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_inc_line))}

int JPG_in_rgb2ycbcr(){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_in_rgb2ycbcr))}

void JPG_inc_block_line(){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_inc_block_line))}

void JPG_dct(){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_dct))}
```

```
void JPG_quant(){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_quant))}

void JPG_emit_byte(int data){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_emit_byte))}

void JPG_emit_bits(unsigned int code, int bits){ INTRINSIC_CODE(TCT_CODE_GEN(I_jpg,
SO_jpg_emit_bits))}

int JPG_enc_coef()  { INTRINSIC_CODE(TCT_CODE_GEN(I_jpg, SO_jpg_enc_coef))}
```

## B.3　ASIP拡張命令実行動作記述

ASIP 拡張命令実行動作記述は、2.3.13 で説明した IO 制御 FSM 動作記述と同じコード構造で実装できる。

- TCT プロセッサの最上位命令実行関数 execute()内部で、ASIP 拡張命令を実行する ASIP 実行制御 FSM 関数を呼び出す。
- ASIP 実行制御 FSM 関数では、デコードされた ASIP 拡張命令の実行動作の他、ASIP エンジン内部の各 FSM 制御動作を記述する。

下記に、JPEG エンコーダ実行制御 FSM 関数呼出しを追加した execute()関数を示す。
（//Applications/TCT_ProcModel/Src/CORE/JPGFifoVersion/TCTProc.c 参照）

```
// TCTProc.c:
void execute()
{
   ...
   jpg_fsm(&cpu.jpg, cpu.dc_reg.op, !cpu.ex_stt.pctl.stalled_flag, src0bp, src1bp, &cpu.io.fifo);
   io_fsm(&cpu.io, cpu.dc_reg.op, !cpu.ex_stt.pctl.stalled_flag, src0bp, src1bp, &cpu.jpg.fifo_port);
   if(cpu.jpg.active){
      ex_out                  = jpg_pipe_control();
      cpu.ex_reg.out          = ex_out;
      cpu.ex_reg.wbflag       = W_ex_out;
   }
   else if(cpu.io.active){
      ex_out                  = io_pipe_control();
      cpu.ex_reg.out          = ex_out;
      cpu.ex_reg.wbflag       = W_ex_out;
   }
   else ...   /// その他の命令実行動作記述
}
```

- jpg_fsm(&cpu.jpg, cpu.dc_reg.op, !cpu.ex_stt.pctl.stalled_flag, src0bp, src1bp, &cpu.io.fifo);：JPEG エンコーダ FSM の動作記述（プロセッサの FIFO 入出力ポート &cpu.io.fifo を引数として渡しており、JPEG エンコーダ FSM が直接 FIFO 入出力ポートとデータ送受信ができるようになっている）
- io_fsm(&cpu.io, cpu.dc_reg.op, !cpu.ex_stt.pctl.stalled_flag, src0bp, src1bp, &cpu.jpg.fifo_port);：IO 制御 FSM の動作記述（2.3.9 の EX ステージ動作記述では、

&cpu.dummy_fifo_port を接続していたが、ここでは、JPEG エンコーダ FSM が FIFO ポートとの通信で使用する&cpu.jpg.fifo を接続する）

- if(cpu.jpg.active){ … }：JPEG エンコーダ拡張命令が実行された場合の処理

次に、JPEG エンコーダ拡張命令実行制御 FSM 関数 jpg_fsm()を示す。

```
void jpg_fsm(JPG * jpg, UINT7 dc_reg_op, BIT active, UINT32 src0, UINT32 src1, FIFO * fifo)
{
  int jpg_insn_active  = 0, jpg_pipe_stall   = 0;
  int i;
  for(i = 0; i < JPG_PIPE_COUNT; i ++){
    jpg->pipe_sig[i].stall        = 0;
    jpg->pipe_sig[i].pending      = 0;
    jpg->pipe_sig[i].out_ready    = 0;
    jpg->pipe_sig[i].out          = 0;
  }
  jpg->pending                    = 0;
  jpg->in_fifo_stall              = 0;
  jpg->out_fifo_stall             = 0;
  jpg->fifo_port.chanID           = 0;
  jpg->fifo_port.strobe           = 0;
  jpg->fifo_port.ready            = 0;
  jpg->out                        = 0;
  if(jpg->out_fifo_stalled){
    jpg->fifo_port.data           = jpg->out_pix;
    jpg->fifo_port.strobe         = 1;
    jpg->active                   = 1;
    jpg->in_fifo_stall            = jpg->in_fifo_stalled;
    jpg->out_fifo_stall           = !fifo->tx.port.ready;
    jpg->pipe_sig[0].stall        = jpg->jpg_stalled;
    jpg_pipe_stall                = jpg->pipe_sig[0].stall;
  }
  else{
    int jpg_any_active = 0, jpg_pipe_stall0 = 0;
    UINT32 jpg_out = 0;
    jpg_insn_active  = JPG0_stage(jpg, dc_reg_op, active, src0, src1, &fifo->rx.port, &fifo->tx.port);
    jpg_any_active   |= jpg_insn_active;
    for(i = 0; i < JPG_PIPE_COUNT; i ++){
      jpg_pipe_stall0             |= jpg->pipe_sig[i].pending;
      jpg_out                     |= jpg->pipe_sig[i].out;
    }
    jpg_pipe_stall                = jpg_pipe_stall0;
    jpg->jpg_stalled              = jpg_pipe_stall0;
    jpg->in_fifo_stalled          = jpg->in_fifo_stall;
    jpg->active                   = jpg_any_active | jpg->in_fifo_stall | jpg->out_fifo_stall;
    jpg->out                      = jpg_out;
  }
  jpg->out_fifo_stalled           = jpg->out_fifo_stall;
  jpg->any_stall                  = jpg_pipe_stall | jpg->in_fifo_stall | jpg->out_fifo_stall | jpg->pending;
}
```

```
UINT32 jpg_pipe_control()
{
  if(cpu.jpg.any_stall){
    cpu.ex_stt.pctl.stall_this    = 1;
    cpu.ex_stt.pctl.stall_prev    = 1;
  }
  return cpu.jpg.out;
}
```

- if(jpg->out_fifo_stalled){ … } : 出力 FIFO ポートがストール状態の処理（JPEG エンコーダ FSM が出力 FIFO からデータ送信を実行する時に、外部 FIFO がビジー状態出会った時にストール状態になる）
- else{ … } : 出力 FIFO ポートがストール状態でない通常処理
  - jpg_insn_active = JPG0_stage(jpg, dc_reg_op, active, src0, src1, &fifo->rx.port, &fifo->tx.port); : JPEG エンコーダ FSM 動作記述

## C. C2RTLオプションファイルパラメータ

## D. C2RTLツールエラーメッセージ